Design Like a Robot! (FreeRTOS)

[v0_05, door Marius Versteegen, 2026]

Over dit document

Dit document neemt je bij de hand tijdens het uitwerken van een simpele casus. Vanuit gegeven requirements kom je via de vaste stappen tot overzichtelijke code. 95% van het proces bestaat uit het eenvoudig volgen van vuistregels. Dus zodra je de vuistregels kent, gaat het design proces al bijna automatisch. Om te zorgen dat niets uit de lucht komt vallen, worden veel kleine muizenstapjes (in belachelijk veel paginas) beschreven. Het is dus als een soort film. Zodra je de stapjes in de smiezen hebt, kun je vlugger doorbladeren.

Het fijne van deze muizenstapjes aanpak is dat je niet van tevoren alles hoeft te overzien. Je kunt op gewoon muizestapje voor muizestapje nemen, en uiteindelijk toch een complex probleem oplossen. Bovendien minimaliseer je met die aanpak de kans op een “writers block”. Een muizestapje zetten lukt immers altijd wel.

Tip: Oefen na het doornemen van dit document het design-proces een aantal keer zelf op klad papier voor wat kleine casussen. Oefening is de enige weg om software architectuur goed in de vingers te krijgen.

Theorie en Casus

Voorafgaand aan het uitvoeren van elke design-fase worden nog even de bijbehorende theorie/vuistregels op een rijtje gezet. De theorie is te herkennen aan blauwe kopjes. De uitvoering van de casus in groene kopjes.

De requirements

Het uitgangspunt voor het maken van een software design bestaat uit requirements. Dat zijn:

- Een use case diagram
- Use case beschrijvingen
- Eventueel een activity diagram ter verduidelijking van de use case beschrijvingen. (niet in dit document)

De design fasen

Vanuit de requirements zijn de vaste stappen om te komen tot een oplossing:

1. Object model met object beschrijvingen
2. Taakstructurering
3. Klassediagram
4. Stds
5. Code
6. Concurrency Diagram

Zoals gezegd, in den beginne was er een Casus met bijbehorende requirements.

De Casus: een Trilapparaat

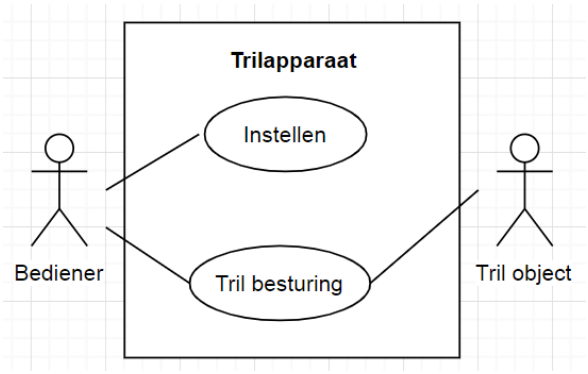
Toepassing:

Het gaat om een all-round ontwerp voor een trilapparaat. Het kan gebruikt worden om een trilplaat te laten bewegen om zand te scheiden op korrelgrootte, om zand van aardappels te schudden, en er zijn vast nog andere functies voor te bedenken.

Opbouw:

Het apparaat heeft een tril-toets, een plus-toets, een min-toets, een nood-knop, een display en een electromagneet met daarbinnen een metalen pin met veer. De toetsen veren terug als je je vinger terug trekt. De noodknop moet je actief weer terug omhoog trekken. Als de electromagneet aan staat, gaat de metalen pin naar het midden van de electromagneet. Als de electromagneet wordt uitgezet, trekt de veer de pin weer terug uit de electromagneet.

Use Case Diagram



Use Case Beschrijvingen

Naam	Instellen
Beschrijving	De ingestelde trilfrequentie wordt op de display weergegeven. Met de plus en min knop kan dat getal aangepast worden. Het getal moet tussen de 1 en de 100 (Hz) blijven. Als na de laatste trilfrequentie-aanpassing een minuut lang geen nieuwe trilfrequentie-aanpassing volgde, wordt die laatste nieuwe trilfrequentie gelogd naar een remote database. Dat laatste kan enige seconden duren.
Uitzondering	Zodra de noodknop is ingeslagen kan de frequentie niet meer worden aangepast, en toont het display “Emergency Stop”. Zodra de noodknop weer is omhooggetrokken kan de bovenstaande use case weer hervat worden.

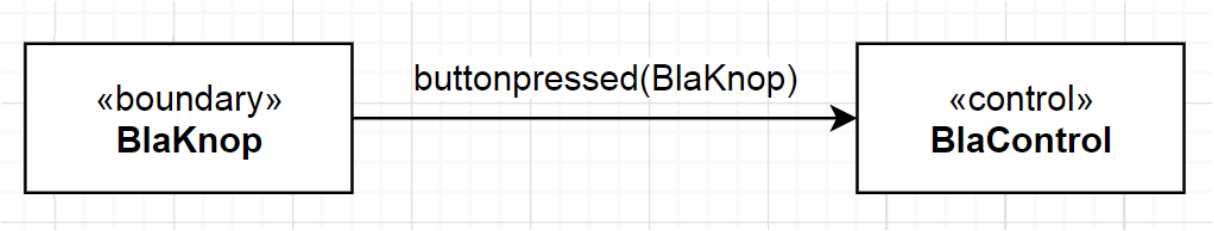
Naam	Tril besturing
Preconditie	Als deze use case start, trilt het apparaat niet.
Beschrijving	Met het indrukken van de tril knop wordt het trillen in de actueel ingestelde trilfrequentie getoggeld.
Uitzondering	Zodra de noodknop is ingeslagen stopt het trillen, en wordt de tril knop genegeerd. Zodra de noodknop weer is omhooggetrokken kan deze use case weer hervat worden.

Design Fase1:
Object model met object beschrijvingen

De eerste fase bestaat uit het maken van een object model en parallel daaraan een lijst met object beschrijvingen.

Wat is een Object model?

Voorbeeld van een Object Model



Een object model is een weergave van objecten en de berichten die ze met elkaar uitwisselen. De objecten worden gerepresenteerd door vierkanten met namen er in (BlaKnop en BlaControl). Boven de naam kan een “categorie”/stereotype (bijvoorbeeld <<boundary>>) staan aangegeven. De vierkanten worden met pijlen met elkaar verbonden. De pijlen geven de richting van de berichten aan. Naast de pijlen staan de namen van de berichten.

Wat is een object?

Een object is een stuk software met een bepaalde verantwoordelijkheid. Een softwarematig “ding”.

Welke stereotypen / categorieen objecten zijn er?

Binnen CSM hoef je alleen te verifieren of een object van een van onderstaande stereotypen is:

- <<boundary>> Een boundary object is de grens van de software, en praat met de “buitenwereld”, dwz met de hardware. Voorbeelden zijn sensoren (knoppen, meetinstrumenten) ,actuatoren (motors, displays, speakers etc) en proxy objecten (remote database proxy: een stuk software dat communiceert met een remote database).
- <<control>> Een control object coordineert een groep andere objecten. Het is een soort “dirigent”.
- <<entity>> Een entity object encapsuleert data (in het algemeen via setters en getters of read en write functies). Het is in het algemeen mooi om bij mekaar horende data te groeperen in een entity object. Vooral als die data als pakket zijn weg door het systeem vindt (bijvoorbeeld samen wordt opgeslagen) of op een centrale plek moet staan die voor meerdere objecten toegankelijk is.
- Iets anders Als een object niet in een van bovengenoemde categorien valt, hoef je er geen stereotype voor in te vullen.

Wat is een bericht?

Een bericht is iets wat het ene object tegen het andere object “zegt”. In het uiteindelijke klassediagram wordt het bericht een functie. Een bericht kan een “payload” hebben. Die staat tussen haakjes. In bovenstaand bijvoorbeeld, zegt het het object BlaKnop “buttonpressed(BlaKnop)“ tegen BlaControl. Dus “er is een knop ingedrukt” en het ging om “BlaKnop”. Meerdere berichten kun je langs dezelfde pijl weergeven door ze met comma te scheiden.

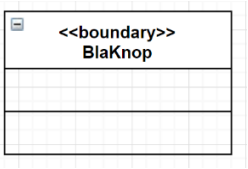
Zo maak je een object model

Het maken van een object model bestaat uit de volgende stappen:

1. Maak een controller object aan voor elke use case.
    a. Het aanmaken van een object gaat in 3 stappen:
        i. Voeg het toe aan het object model.
        ii. Geef het een eventueel geschikt stereotype (zie even verderop).
        iii. Voeg een beschrijving ervan toe aan de lijst met objecten.
2. Verwerk de use case beschrijvingen zin voor zin:
    a. Lees een zin.
    b. Bedenk welke “dingen” je systeem nodig heeft om te kunnen doen wat er staat. (vaak staan ze al in de zin als zelfstandige naamwoorden)
    c. Maak voor elk van die “dingen” een object aan (weer in 3 stappen):
        i. Voeg het toe aan het object model.
        ii. Geef het een eventueel geschikt stereotype (zie even verderop).
        iii. Voeg een beschrijving ervan toe aan de lijst met objecten.
    d. Ga na welk “ding” iets tegen welk “ander ding” “zegt” om de zin waar te maken.
    e. Zet wat er “wordt gezegd” als bericht langs een pijl vanuit het sprekende object naar het luisterende object.

Praktische Tips

- Als je opdrachtgever het toelaat, gebruik dan tijdens het tekenen van je object model geen vierkanten, maar gebruik klasse placeholders (dat is eigenlijk niet officieel, maar bespaart dubbel werk wanneer je in een latere design fase je object model omzet in een klassediagram):



  (in drawIO kun je in het member-veld en het functie-veld een spatie invullen, dan kun je daar later altijd nog - na het omzetten van object model naar klassediagram – de members en functies invullen).
- Gebruik geen spaties of speciale karakters in de namen voor je objecten. Later worden die namen immers de uitgangspunten voor de klasse namen.
- Gebruik geen spaties of speciale karakters in de namen voor je berichten of hun payload. Later worden die namen immers de uitgangspunten van publieke operaties van je klassen.
- Voor use cases waarvan een activity diagram beschikbaar is, kun je de bovenstaande stappen (nog makkelijker-) uitvoeren door stap voor stap door dat activity diagram te lopen (in plaats van zin voor zin door de use case beschrijving. In dit document gaan we even uit van het worst case geval – dus dat er geen activity diagram beschikbaar is.

Timers

Wel of geen object aanmaken voor een timer?

Timers zijn waitables zoals flags en channels. In het algemeen voegt het niet veel toe om waitable objecten expliciet in het object model op te nemen. Tijdens het omzetten van het STD naar Code volgt al op logische wijze welke waitables je moet aanmaken.

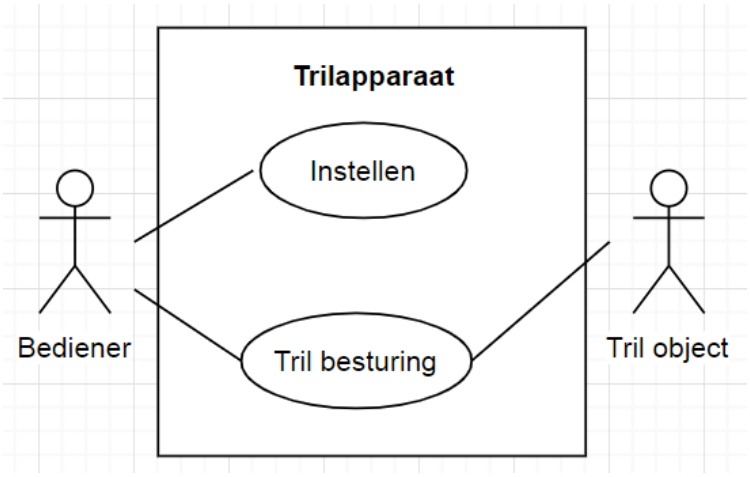
Okee, tot zover de theorie voor het maken van een object model. Now, let's Due it!

Object model met object beschrijvingen voor het Trilapparaat

We lopen voor het trilapparaat gewoon de bovengenoemde stappen voor het opstellen van een object model door.

Stap 1: Maak een controller object aan voor elke use case.

Nog even kijken naar het use case diagram:



Toevoegen aan het object model

Aha, we hadden twee use cases: “Instellen” en “Trilbesturing”. Voor elke use case moesten we een controller object aanmaken. Mooie compacte namen voor de bijbehorende controller objecten lijken me “InstelControl” en “TrilControl”. Er mogen geen spaties of speciale karakters in de namen zitten.

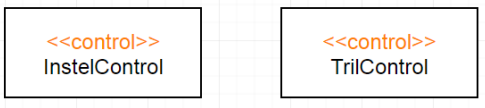
Die voegen we toe aan het object model:



Eventueel stereotype aangeven

Voor objecten uit een van de categorien <<control>>, <<boundary>> en <<entity>> moeten we nog een stereotype weergeven boven de naam.

Bij onze controller objecten moeten we dus het stereotype <<control>> boven de naam noteren:



Beschrijving toevoegen aan de lijst met objecten

Elk van de gevonden objecten voegen we toe aan de lijst met objecten, en we voegen een beschrijving toe. In dit stadium weten we alleen dat deze twee objecten de bijbehorende use case voor hun rekening moeten nemen. Voor nu vullen we dat dus in:

Object Naam	**Stereotype**	**Beschrijving**
InstelControl	Control	“Dirigent” van use case “Instellen” (zie use case beschrijving)
TrilControl	Control	“Dirigent” van use case “Trillen” (zie use case beschrijving)

Stap 2: Verwerk de use case beschrijvingen zin voor zin

We beginnen met de use case “Instellen”.

Lees een zin

We lezen de eerste zin van de use case:
“De ingestelde trilfrequentie wordt op de display weergegeven.”

Bedenk welke “dingen” je systeem nodig heeft om te kunnen doen wat er staat.

In deze zin bedenk ik dat de volgende dingen van belang zijn:

- een “ingestelde trilfrequentie”
- een display

In dit geval is dat extra makkelijk, omdat deze “dingen” al voorkomen in de zin als zelfstandige naamwoorden. Dat is typisch het geval als de usecase is geformuleerd vanuit het **perspectief van het apparaat**.

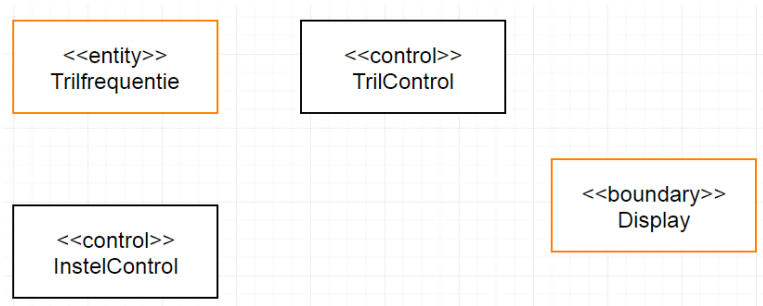
Stel nu dat de usecase zou zijn gespecificeerd vanuit het perspectief van de persoon. Dan had er bijvoorbeeld kunnen staan: “de operator verifieert de ingestelde trilfrequentie visueel”. In zo’n vager geformuleerd geval zul je zelf moeten bedenken met welke “dingen” je dat mogelijk kunt maken. Je kunt dan kiezen voor een display, maar bijvoorbeeld ook voor een analoge wijzer die uitslaat. Misschien bedenk je in het geval van een display dat je het alleen wilt als er een knop wordt ingedrukt. In dat geval heb je er ook een knop voor nodig.

Maak voor elk van de “dingen” een object aan

- Een data-gegeven als trilfrequentie kunnen we modelleren als entity object. Ik noem het “Trilfrequentie”.
- Voor de display maken we een boundary object aan. Ik noem het “Display”. Het is een software object dat hardware (in dit geval een display) aanstuurt.

Zoals altijd voegen we de nieuwe objecten toe in 3 stappen:

Toevoegen aan het object model + stereotype erbij



Beschrijving toevoegen aan de lijst met objecten

Object Naam	Stereotype	Beschrijving
InstelControl	Control	“Dirigent” van use case “Instellen” (zie use case beschrijving).
TrilControl	Control	“Dirigent” van use case “Trillen” (zie use case beschrijving).
Trilfequentie	Entity	Bevat de ingestelde trilfrequentie.

Display	Boundary	Stuurt de display hardware aan.

Databeheer

Publieke data

NB: we hebben het hier over "**publieke data**" die wordt **uitgewisseld** tussen objecten. Als het slechts binnen een object gebruikt wordt, kan het verborgen kunnen blijven, en zien we het niet in het objectmodel terug.

Waar "zit" die publieke data nu?

In het algemeen probeer je voor elke publieke data in je systeem te laten beheren door een enkele "**(hoofd-) eigenaar**". Als je de data wilt opvragen, kun je die eigenaar raadplegen. Wil je de data veranderen, dan kun je dat bij die eigenaar aanvragen. Je kunt ervoor kiezen om een entity object (hoofd-) eigenaar te maken, of een ander object.

Wat is netjes? Data beheer door entity object of ander object?

Het is in het algemeen netjes dat data wordt beheerd in het object **waar het het meest mee verbonden is / wordt "gebruikt"**. In dat geval hoef je er geen apart entity object voor te maken, en encapsuleer je de data in het betreffende object.

De trilfrequentie wordt weliswaar ingesteld in InstelControl, maar wordt echt gebruikt in TrilControl, je zou het dus prima kunnen encapsuleren in TrilControl.

Maar wanneer gebruik je dan een entity object?

Dat kun je gebruiken in de volgende gevallen:

- **Datapakket**
- **Geen logische eigenaar**

Datapakket

Het had ook kunnen zijn dat trilfrequentie data is die in **samenhang met andere data als een geheel** kan worden **gezien**, **behandeld**, **weggeschreven of verzonden**, als **datapakket**. In dat geval is het netter om die bij mekaar horende data te bundelen in een apart "**entity object**".

Bijvoorbeeld als de data zich bevindt in traag geheugen, en/of om bij mekaar horende data logisch te groeperen en/of als de data om de een of andere reden geencapsuleerd moet worden. Bijvoorbeeld bij een database of bij data die via encryptie wordt opgeslagen en opgevraagd.

Bij ons trilapparaat is er alleen een Trilfrequentie die ingesteld moet worden. In een wat ingewikkelder, realistischer voorbeeld wil je misschien meerdere data instellen, die tesamen een entiteit "programma" vormen. Dat bevat dan bijvoorbeeld Trilfrequentie, Trilintensiteit en Loopduur. Je kunt zo'n entiteit dan bijvoorbeeld in zijn geheel aanpassen met InstelControl en eventueel lezen of schrijven naar disk met een DiskControl.

Stel nu dat je in zo'n geval DiskControl weglaat, en de entiteit "programma" als geheel **alleen** wordt gebruikt door InstelControl. Het gaat dan alleen nog om een conceptuele groepering van data. In dat geval zou je er ook voor kunnen kiezen om die data toch te **encapsuleren** in InstelControl (in plaats van in een apart entity object. Dat is een kwestie van smaak.

Geen logische eigenaar

In het geval van de Trilfrequentie is het wel duidelijk dat die het meest verbonden is met TrilControl. Het kan ook voorkomen dat je data tegenkomt die door meerdere objecten wordt gebruikt, maar waarvan het niet duidelijk het meest bij een enkel object hoort.

Ook in zo'n geval is het netter om een apart entity object eigenaar voor die data te laten zijn.

Databeheer

Voor het Trilapparaat kies ik voor een apart entity object, puur om ook een entity object in het design te laten voorkomen. Als extra argument zou ik kunnen gebruiken dat ik voorzie dat het aantal instellingen in de toekomst wellicht zal worden uitgebreid, en dat het dan conceptueel overzichtelijk is als de instellingen in een aparte entity gebundeld zijn.
Maar zoals je misschien al uit het bovenstaande verhaal hebt opgemerkt, was het bij dit minimale Trilapparaat eigenlijk niet nodig.

Ga na welk "ding" iets tegen welk "ander ding" "zegt"

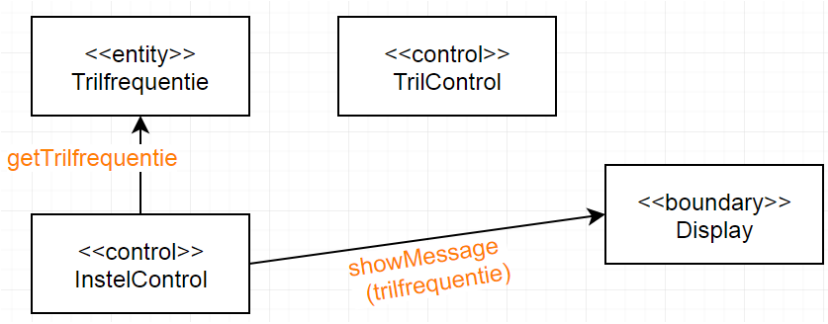
De zin uit de use case "Instellen" was:
"De ingestelde trilfrequentie wordt op de display weergegeven."

Die zin wordt uitgevoerd (zoals de meeste zinnen in de use case instellen) door het object InstelControl. Dus: InstelControl zegt "geef trilfrequentie weer" tegen het Display object.
Maar goed, een Display is een display, en weet niets van trilfrequenties. We passen dit bevel dus een beetje aan, naar: "showMessage(trilfrequentie)". Dus zonder spaties, en met bericht en "payload" netjes gescheiden.

Even verder denken: InstelControl geeft dus de trilfrequentie door aan de Display, maar hoe komt InstelControl daar aan? We hadden besloten om de trilfrequentie in een TrilFrequentie entity op te slaan, dus InstelControl zal de trilfrequentie daar op moeten vragen. InstelControl zegt dus tegen TrilFrequentie "geefTrilFrequentie", of gebruikelijker, met het engelse get: "getTrilFrequentie".

Zet wat er "wordt gezegd" als bericht langs een pijl vanuit het sprekende object naar het luisterende object.

Onderstaand is dat gedaan voor de berichten showMessage(trilfrequentie) en getTrilFrequentie.



Met dezelfde stappen kunnen we de andere zinnen van deze use case verwerken.
Op naar de volgende zin dus!

Lees een zin

We lezen de tweede zin van de use case Instellen:

"Met de plus en min knop kan dat getal aangepast worden."

Identificeer de "dingen"

In deze zin zie ik de volgende dingen (zelfstandig naamwoorden):

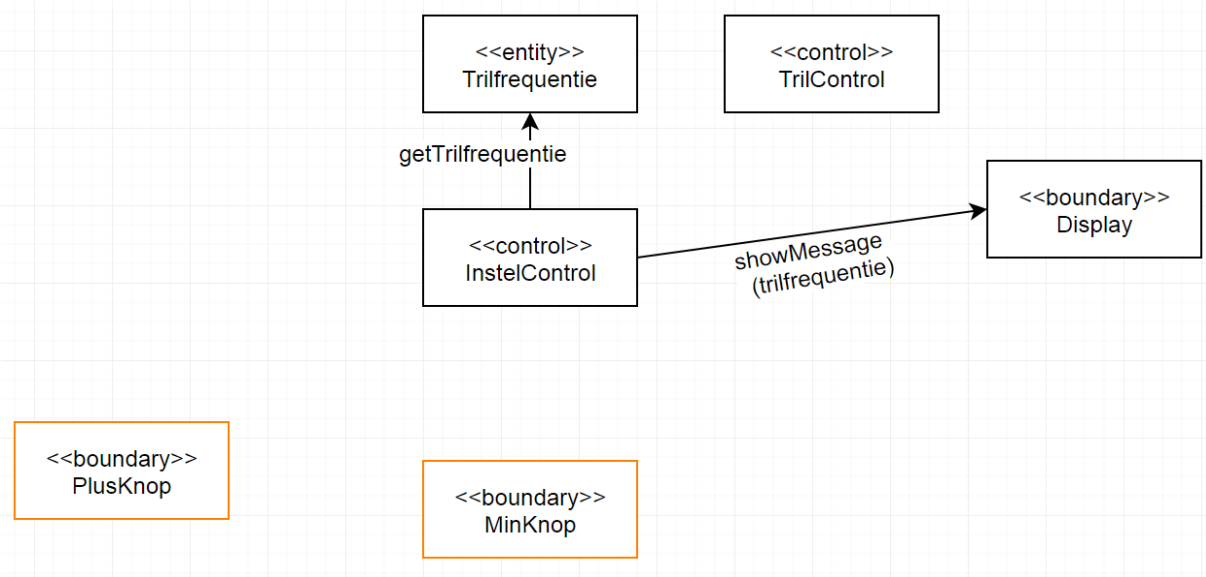
- Een plus knop
- Een min knop

Maak voor elk van de "dingen" een object aan

- Voor elk van beide knoppen maken we een boundary object aan. Het zijn software objecten die hardware (in dit geval een pin van de chip waar een button aanvast zit) uitlezen.

Zoals altijd voegen we de nieuwe objecten toe in 3 stappen:

Toevoegen aan het object model + stereotype erbij



Beschrijving toevoegen aan de lijst met objecten

Object Naam	Stereotype	Beschrijving
InstelControl	control	"Dirigent" van use case "Instellen" (zie use case beschrijving).
TrilControl	control	"Dirigent" van use case "Trillen" (zie use case beschrijving).
Trilfequentie	entity	Bevat de ingestelde trilfrequentie.
Display	boundary	Stuurt de display hardware aan.
PlusKnop	boundary	Een knop die terugveert.
MinKnop	boundary	Een knop die terugveert.

Ga na welk "ding" iets tegen welk "ander ding" "zegt"

De zin uit de use case "Instellen" was:

"Met de plus en min knop kan dat getal aangepast worden."

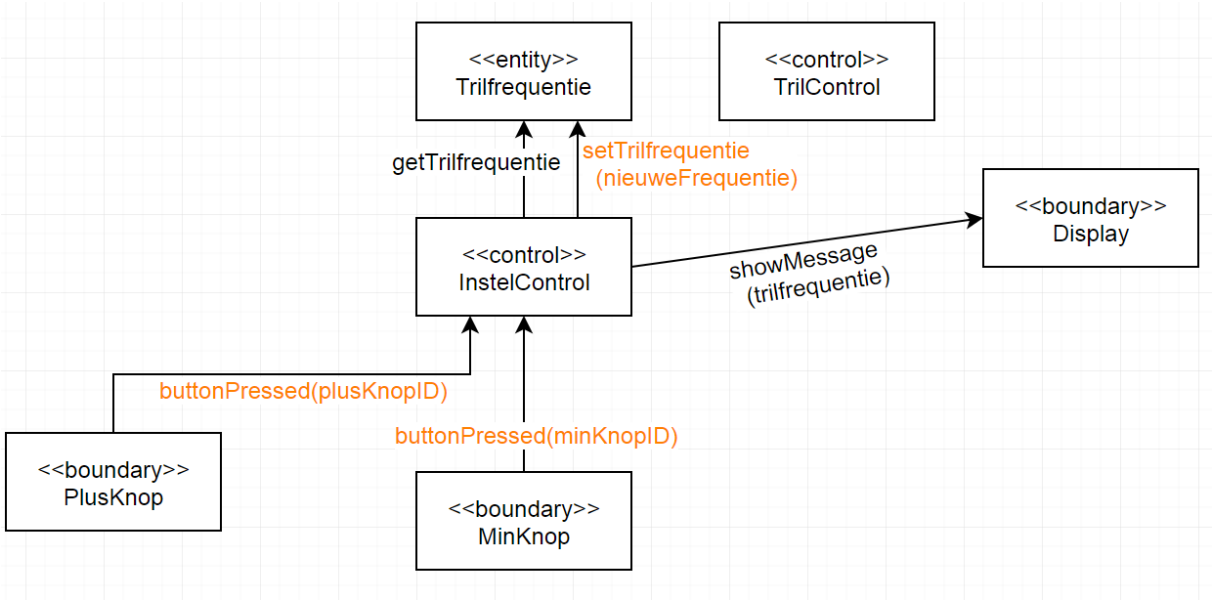
We hebben het over de use case Instellen, dus het object dat dat doet moet InstelControl zijn.

- De plus en min knoppen moeten dus melden aan InstelControl dat ze ingedrukt zijn.
- Vervolgens kan InstelControl de frequentie uitlezen uit TrilFrequentie, aanpassen, en er weer naar terugschrijven.

Kortom:

- PlusKnop zegt "meldKnopIngedrukt(plusKnopID)" tegen InstelControl. plusKnopID is daarbij een int of enum die aangeeft om welke knop het gaat. Gebruikelijker is de engelse variant: "buttonpressed(plusKnopID)". Voor de MinKnop geldt iets soortgelijks.
- InstelControl zegt "getTrilfrequentie" en "setTrilfrequentie(nieuweFrequentie)" tegen de TrilFrequentie entity.

Zet wat er "wordt gezegd" als bericht langs een pijl vanuit het sprekende object naar het luisterende object.



Mooi, volgende zin!

Lees een zin

We lezen de volgende zin van de use case Instellen:
"Het getal moet tussen de 1 en de 100 (Hz) blijven."

Identificeer de "dingen"

Dat "getal" is de trilfrequentie. Die hebben we al. Verder geen (nieuwe-) "dingen" hier. We lezen we de beschrijving van use case instellen, dus dit is iets waar InstelControl voor moet gaan zorgen in zijn Std.

Volgende zin!

Lees een zin

We lezen de volgende zin van de use case Instellen:
"Als na de laatste trilfrequentie-aanpassing een minuut lang geen nieuwe trilfrequentie-aanpassing volgde, wordt die laatste nieuwe trilfrequentie gelogd naar een remote database."

Identificeer de "dingen"

In deze zin zie ik de volgende nieuwe dingen (zelfstandig naamwoorden):

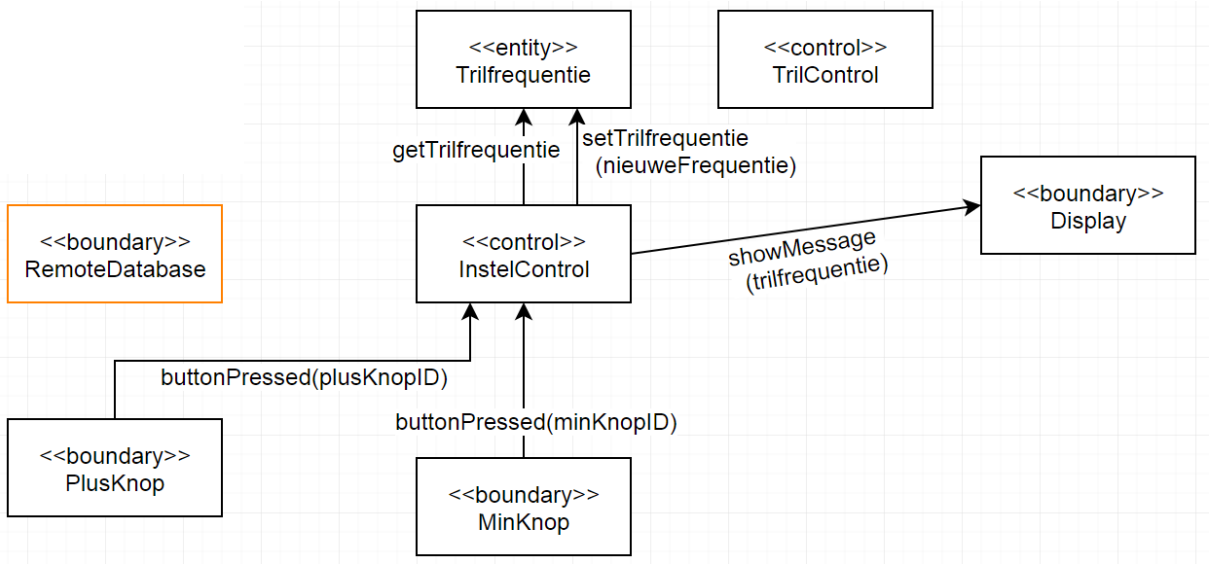
- Een minuut
- Een remote database

Maak voor elk van de "dingen" een object aan

- Er moet een tijd worden gemeten (een minuut), in een soort van "time out" situatie. In de theorie hebben we gezien dat we voor waitables zoals timers geen object in het object model hoeven aan te maken.
- Voor de remote database maken we een boundary object aan. We zouden het nauwkeuriger kunnen specificeren als "proxy" object (maar laten dat hier achterwege).

Zoals altijd voegen we de nieuwe objecten toe in 3 stappen:

Toevoegen aan het object model + stereotype erbij



Beschrijving toevoegen aan de lijst met objecten

Object Naam	Stereotype	Beschrijving
InstelControl	control	"Dirigent" van use case "Instellen" (zie use case beschrijving).
TrilControl	control	"Dirigent" van use case "Trillen" (zie use case beschrijving).
Trilfequentie	entity	Bevat de ingestelde trilfrequentie.
Display	boundary	Stuurt de display hardware aan.
PlusKnop	boundary	Een knop die terugveert.
MinKnop	boundary	Een knop die terugveert.
RemoteDatabase	boundary	Een object via welke data uitgewisseld kan worden met een database elders.

Ga na welk "ding" iets tegen welk "ander ding" "zegt"

De zin uit de use case "Instellen" was:

"Als na de laatste trilfrequentie-aanpassing een minuut lang geen nieuwe trilfrequentie-aanpassing volgde, wordt die laatste nieuwe trilfrequentie gelogd naar een remote database."

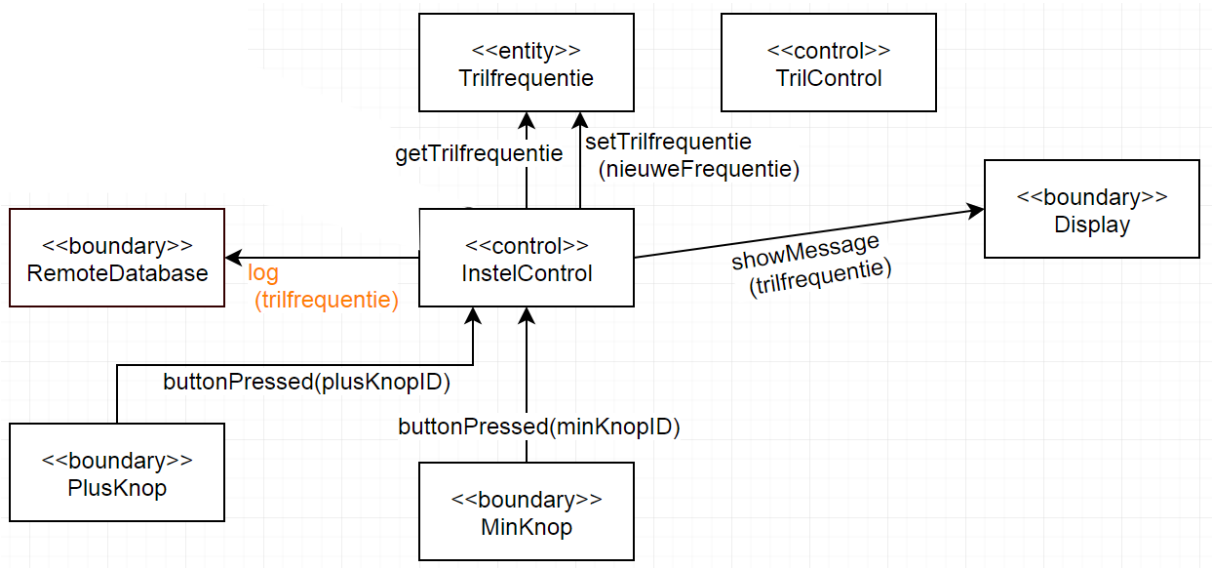
We hebben het over de use case Instellen, dus InstellenControl moet (een bepaalde tijd na voor het laatst indrukken van een knop) de nieuwe trilfrequentie loggen naar de remote database.

- InstelControl zegt tegen de RemoteDatabase kan zeggen “log deze trilfrequentie”. Mooi opgesplitst in bericht en payload zou dat worden: log(trilfrequentie).

Kortom:

- InstelControl zegt log(trilfrequentie) tegen RemoteDatabase.

Zet wat er “wordt gezegd” als bericht langs een pijl vanuit het sprekende object naar het luisterende object.



Mooi, volgende zin!

Lees een zin

We lezen de volgende zin van de use case Instellen:
“Dat laatste kan enige seconden duren.”

Identificeer de “dingen”

In deze zin zie ik de volgende dingen (zelfstandig naamwoorden): seconden. Maar dit zijn geen seconden die we zelf willen maken. We hoeven voor waitables zoals timers ueberhaupt geen objecten aan te maken.

Volgende zin!

Lees een zin

We lezen de volgende zin van de use case Instellen:
“Zodra de noodknop is ingeslagen kan de frequentie niet meer worden aangepast, en toont het display “Emergency Stop”.”

Identificeer de “dingen”

In deze zin zie ik de volgende nieuwe dingen (zelfstandig naamwoorden):

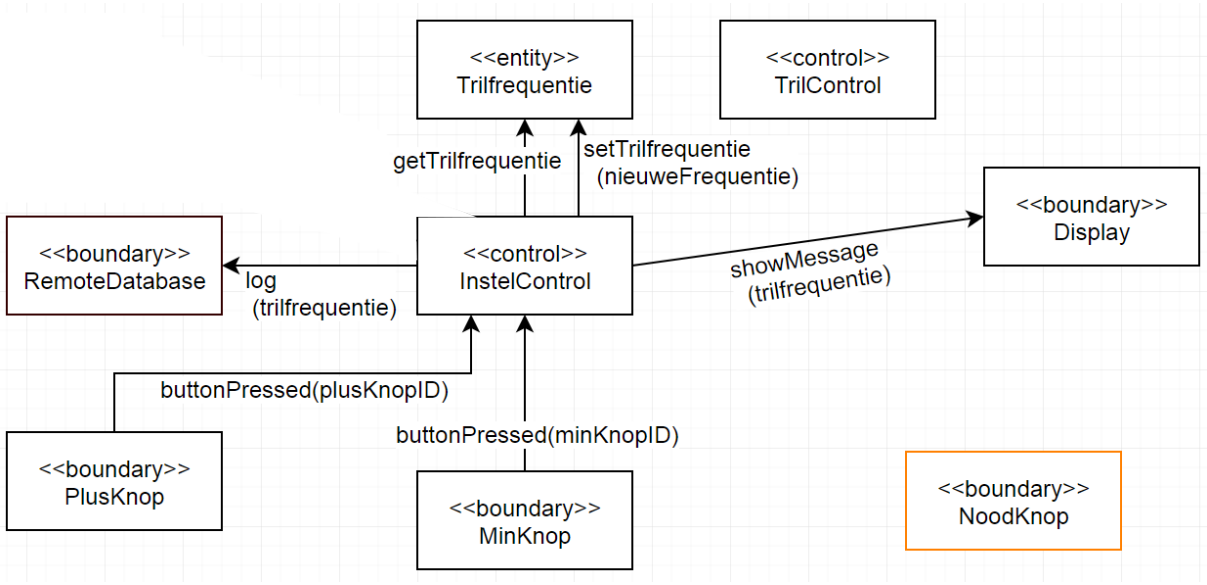
- Een noodknop

Maak voor elk van de "dingen" een object aan

- Voor de noodknop maken we een boundary object aan.

Zoals altijd voegen we de nieuwe objecten toe in 3 stappen:

Toevoegen aan het object model + stereotype erbij



Beschrijving toevoegen aan de lijst met objecten

Object Naam	Stereotype	Beschrijving
InstelControl	Control	"Dirigent" van use case "Instellen" (zie use case beschrijving).
TrilControl	Control	"Dirigent" van use case "Trillen" (zie use case beschrijving).
Trilfequentie	Entity	Bevat de ingestelde trilfrequentie.
Display	boundary	Stuurt de display hardware aan.
PlusKnop	boundary	Een knop die terugveert.
MinKnop	boundary	Een knop die terugveert.
NoodKnop	boundary	Een knop die kan worden ingeslagen en weer uitgetrokken.
RemoteDatabase	boundary	Een object via welke data uitgewisseld kan worden met een database elders.

Ga na welk "ding" iets tegen welk "ander ding" "zegt"

De zin uit de use case "Instellen" was:
"Zodra de noodknop is ingeslagen kan de frequentie niet meer worden aangepast, en toont het display "Emergency Stop"."

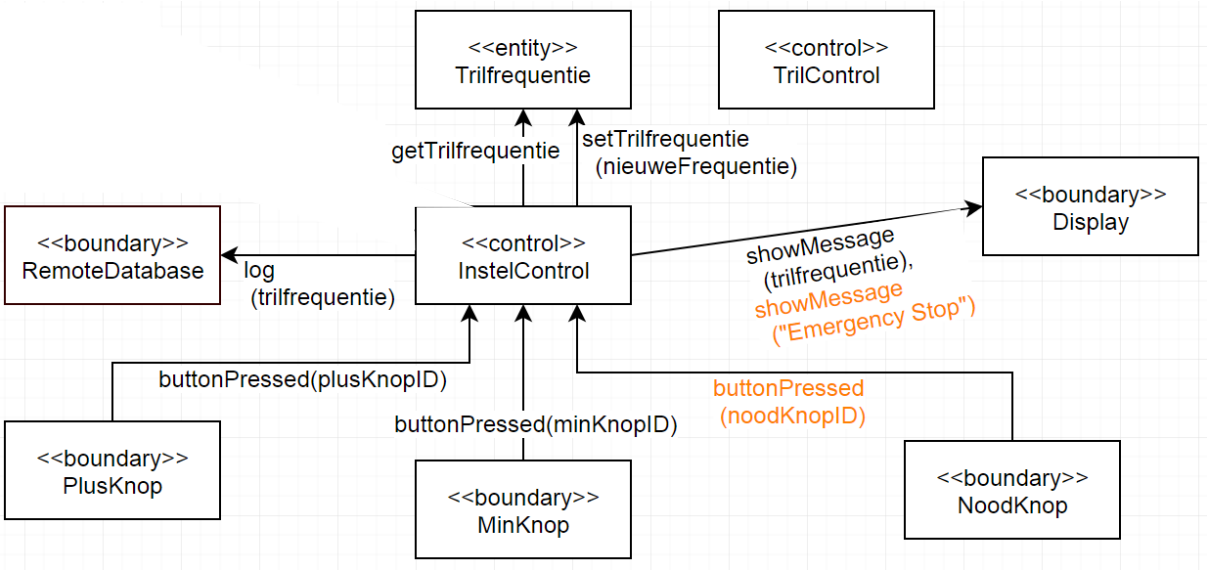
We hebben het over de use case Instellen, dus InstellenControl moet het object zijn dat luistert naar het de noodknop en moet zorgen dat er iets op het display verschijnt.

Kortom:

- Noodknop zegt buttonpressed(noodKnopId) tegen InstelControl.

- InstelControl zegt showmessage(“Emergency Stop”) tegen Display.

Zet wat er “wordt gezegd” als bericht langs een pijl vanuit het sprekende object naar het luisterende object.



Mooi, volgende zin!

Lees een zin

We lezen de volgende zin van de use case Instellen:
“Zodra de noodknop weer is omhooggetrokken kan de bovenstaande use case weer hervat worden.”

Identificeer de “dingen”

In deze zin staan geen nieuwe “dingen”. Wel staat er iets over een al toegevoegd ding, de Noodknop. We gaan daarom door naar de volgende stap:

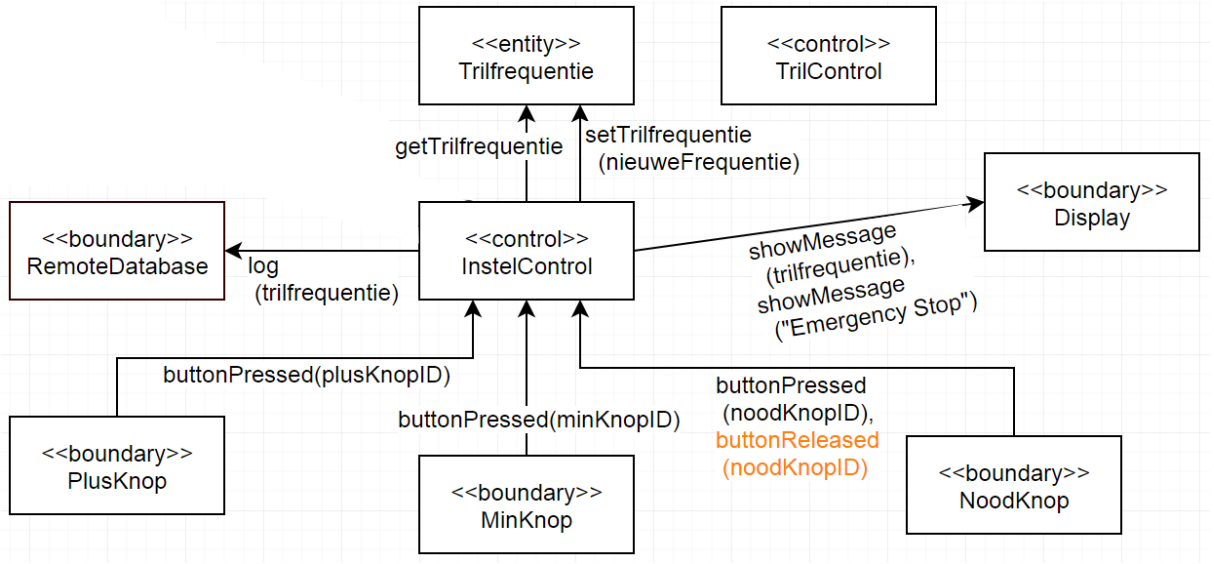
Ga na welk “ding” iets tegen welk “ander ding” “zegt”

We hebben het over de use case Instellen, dus InstelControl moet het object zijn dat luistert naar het de noodknop en moet zorgen dat het frequentie-instellen wordt hervat zodra de noodknop weer is omhooggetrokken. De Noodknop moet dus tegen InstelControl zeggen dat hij “weer omhoog getrokken is”. We kunnen daar een mooi algemeen engels bericht met payload voor maken: buttonReleased(buttonId).

Kortom:

- Noodknop zegt buttonReleased(noodKnopId) tegen InstelControl.

Zet wat er "wordt gezegd" als bericht langs een pijl vanuit het sprekende object naar het luisterende object.



Mooi, we we hebben nu alle zinnen van de beschrijving van de use case "Instellen".
Stoiceins gaan we verder met de volgende use case: "Trillen".

Lees een zin

We lezen de eerste zin van de use case Trillen:
"Als deze use case start, trilt het apparaat niet."

Identificeer de "dingen"

In deze zin zie ik de volgende nieuwe dingen (zelfstandig naamwoorden):

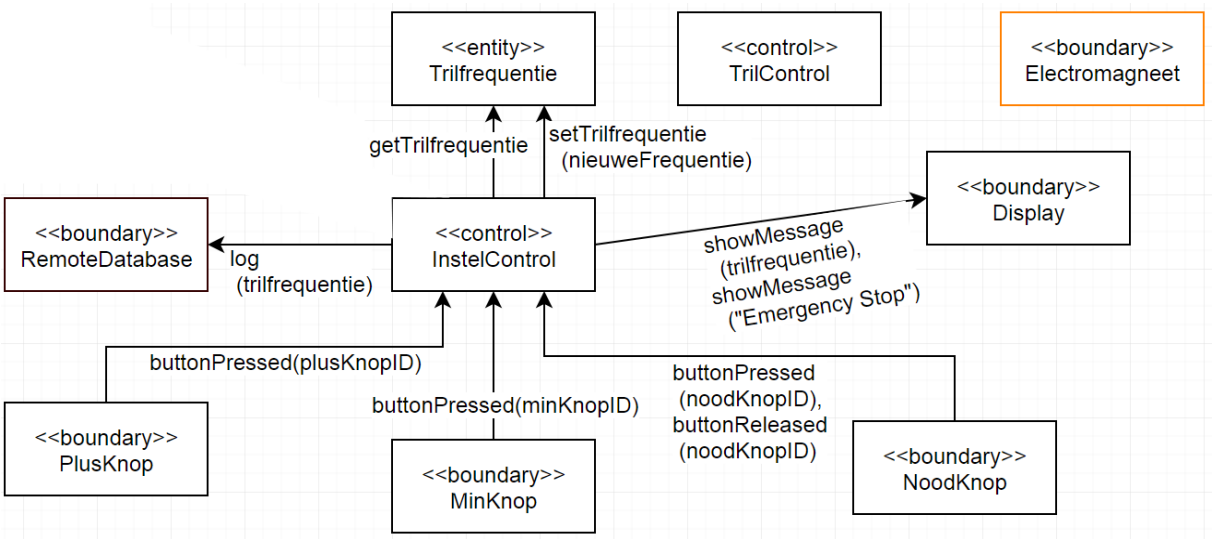
- Het apparaat. "Het apparaat" is kennelijk iets dat al dan niet kan trillen. In de casus hebben we gezien dat dat gebeurt door een electromagneet aan of uit te schakelen. Het ligt dus voor de hand om een object aan te maken dat de electromagneet aan of uit kan schakelen.

Maak voor elk van de "dingen" een object aan

- We maken een boundary object Electromagneet aan. Dat is een software object dat kan zorgen dat er stroom wordt gezet – of niet – op een hardwarematige electromagneet.

Zoals altijd voegen we de nieuwe objecten toe in 3 stappen:

Toevoegen aan het object model + stereotype erbij



Beschrijving toevoegen aan de lijst met objecten

Object Naam	**Stereotype**	**Beschrijving**
InstelControl	Control	"Dirigent" van use case "Instellen" (zie use case beschrijving).
TrilControl	Control	"Dirigent" van use case "Trillen" (zie use case beschrijving).
Trilfequentie	Entity	Bevat de ingestelde trilfrequentie.
Display	Boundary	Stuurt de display hardware aan.
PlusKnop	Boundary	Een knop die terugveert.
MinKnop	Boundary	Een knop die terugveert.
NoodKnop	Boundary	Een knop die kan worden ingeslagen en weer uitgetrokken.
Electromagneet	Boundary	Een electromagneet die kan worden aan en uitgezet.
RemoteDatabase	Boundary	Een object via welke data uitgewisseld kan worden met een database elders.

Ga na welk "ding" iets tegen welk "ander ding" "zegt"

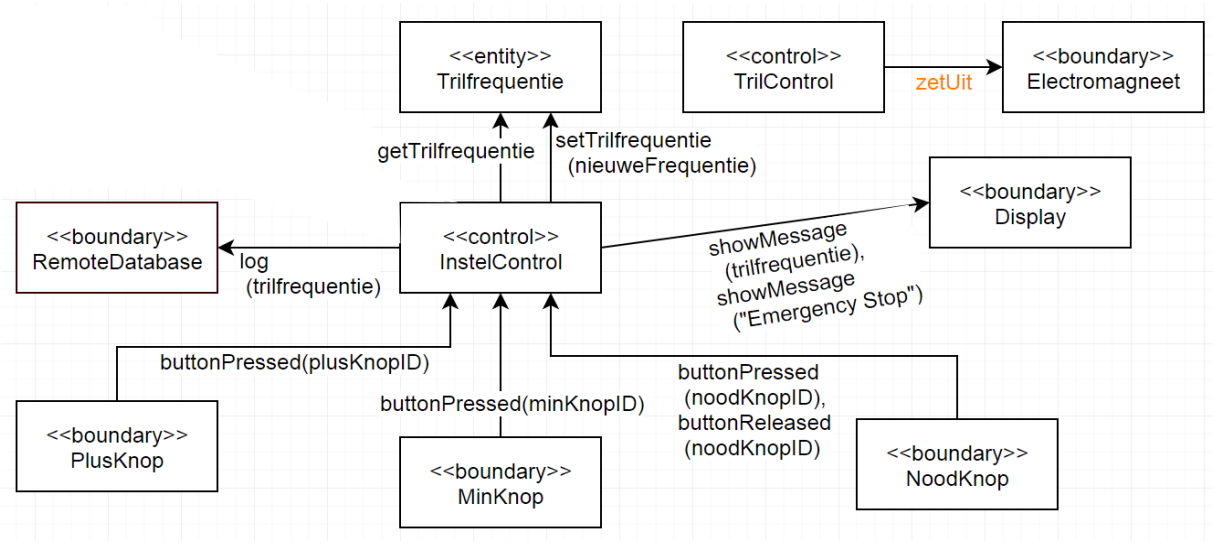
De zin uit de use case "Trillen" was:
"Als deze use case start, trilt het apparaat niet."

We hebben het over de use case Trillen, dus TrilControl moet het object zijn dat zorgt dat de electromagneet na opstarten uit staat.

Kortom:

- TrilControl zegt zetUit tegen Electromagneet.

Zet wat er "wordt gezegd" als bericht langs een pijl vanuit het sprekende object naar het luisterende object.



Volgende zin!

Lees een zin

We lezen de volgende zin van de use case Trillen:

"Met het indrukken van de tril knop wordt het trillen in de actueel ingestelde trilfrequentie getoggeld."

Identificeer de "dingen"

In deze zin zie ik de volgende nieuwe dingen:

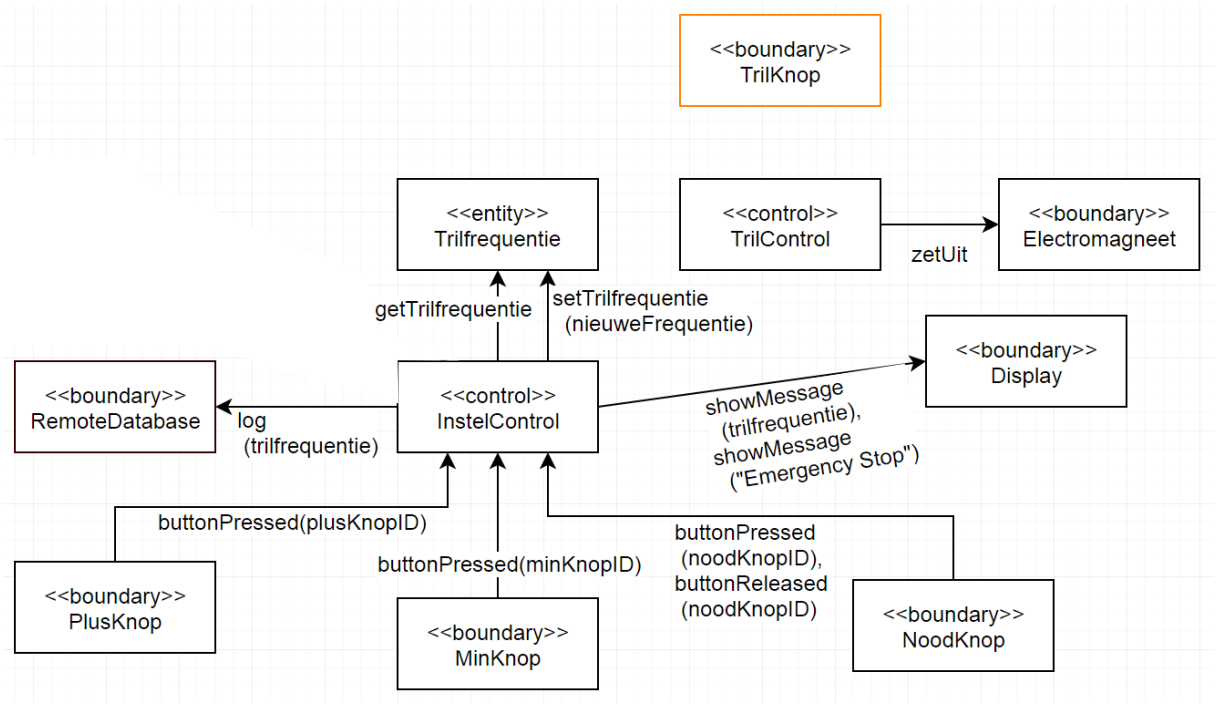
- Een trilknop
- Er moet ook iets gebeuren in een bepaalde frequentie, ofwel met bepaalde tussentijdsintervallen. Die tijdsintervallen zou je kunnen maken met een timer.
  Net als andere waitables hoeven we daar geen object voor aan te maken in het object model.

Maak voor elk van de "dingen" een object aan

- Voor de trilknop maken we een boundary object TrilKnop aan.

Zoals altijd voegen we de nieuwe objecten toe in 3 stappen:

Toevoegen aan het object model + stereotype erbij



Beschrijving toevoegen aan de lijst met objecten

Object Naam	Stereotype	Beschrijving
InstelControl	Control	"Dirigent" van use case "Instellen" (zie use case beschrijving).
TrilControl	Control	"Dirigent" van use case "Trillen" (zie use case beschrijving).
Trilfequentie	Entity	Bevat de ingestelde trilfrequentie.
Display	Boundary	Stuurt de display hardware aan.
PlusKnop	Boundary	Een knop die terugveert.
MinKnop	Boundary	Een knop die terugveert.
NoodKnop	boundary	Een knop die kan worden ingeslagen en weer uitgetrokken.
TrilKnop	Boundary	Een knop die terugveert.
RemoteDatabase	Boundary	Een object via welke data uitgewisseld kan worden met een database elders.

Ga na welk "ding" iets tegen welk "ander ding" "zegt"

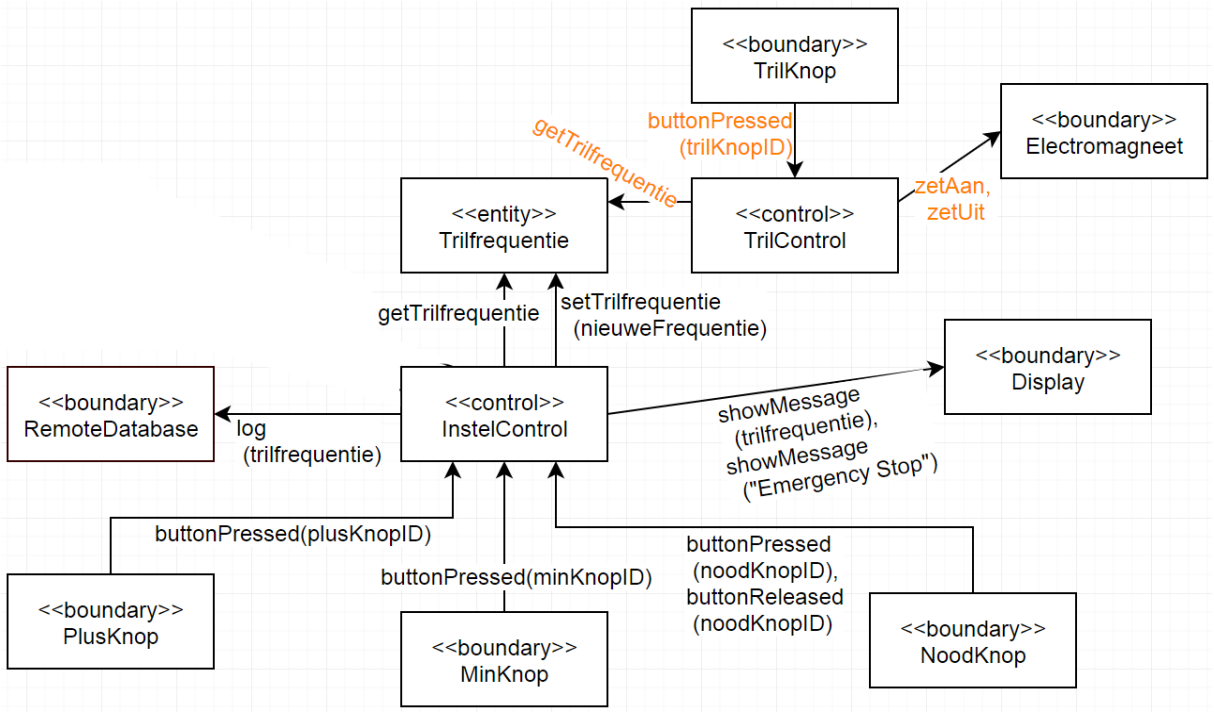
De zin uit de use case "Trillen" was:
"Met het indrukken van de tril knop wordt het trillen in de actueel ingestelde trilfrequentie getoggeld."

- We hebben het over de use case Trillen, dus TrilControl moet het object zijn dat luistert naar het de TrilKnop en moet zorgen dat het trillen aan of uit wordt gezet. Maar wat is trillen? Dat is het aan en weer uitzetten van de electromagneet. In het huidige object model kan TrilControl al ZetUit zeggen tegen de electromagneet. Om te kunnen laten trillen moet hij dus ook "zet aan" kunnen zeggen.
- Verder moet het trillen gebeuren met de actueel ingestelde frequentie. Het zou dus handig zijn als TrilControl tegen de Trilfrequentie entity "getTrilFrequentie" zou kunnen zeggen.

Kortom:

- TrilKnop zegt buttonPressed(trilKnopID) tegen TrilControl
- TrilControl zegt zetAan tegen Electromagneet.
- TrilControl zegt getTrilfrequentie tegen Trilfrequentie.

Zet wat er "wordt gezegd" als bericht langs een pijl vanuit het sprekende object naar het luisterende object.



Mooi, volgende zin!

Lees een zin

We lezen de volgende zin van de use case Trillen:
"Zodra de noodknop is ingeslagen stopt het trillen, en wordt de tril knop genegeerd."

Identificeer de "dingen"

Alle dingen in die zin: noodknop, tril knop, trillen (van de electromagneet en TrilControl) staan al in het object model. We hoeven dus geen object toe te voegen.

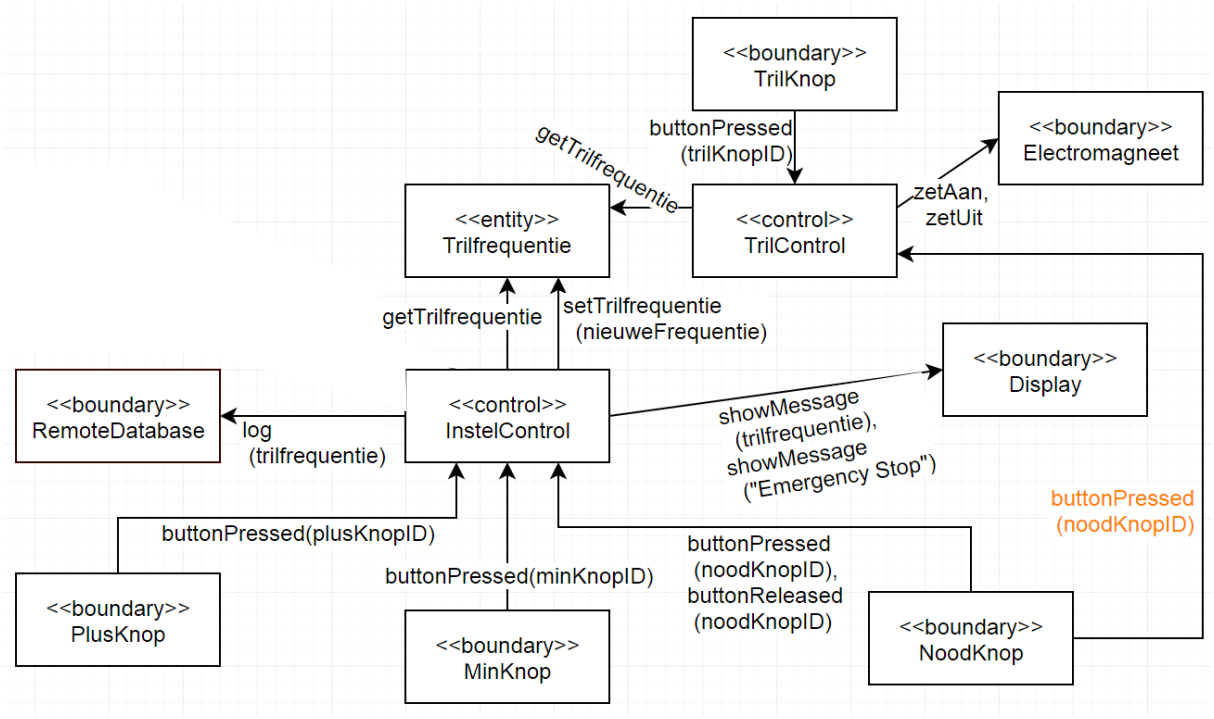
Ga na welk "ding" iets tegen welk "ander ding" "zegt"

- We hebben het over de use case Trillen, dus TrilControl moet het object zijn dat in deze zin luistert naar het de NoodKnop en moet zorgen dat alles tot stilstand komt.
  NoodKnop moet TrilControl dus laten weten dat hij is ingedrukt.

Kortom:

- NoodKnop zegt buttonPressed(noodKnopID) tegen TrilControl.

Zet wat er "wordt gezegd" als bericht langs een pijl vanuit het sprekende object naar het luisterende object.



Mooi, volgende zin!

Lees een zin

We lezen de volgende zin van de use case Trillen:

"Zodra de noodknop weer is omhooggetrokken kan deze use case weer hervat worden."

Identificeer de "dingen"

Alle dingen in die zin staan al in het object model. We hoeven dus geen object toe te voegen.

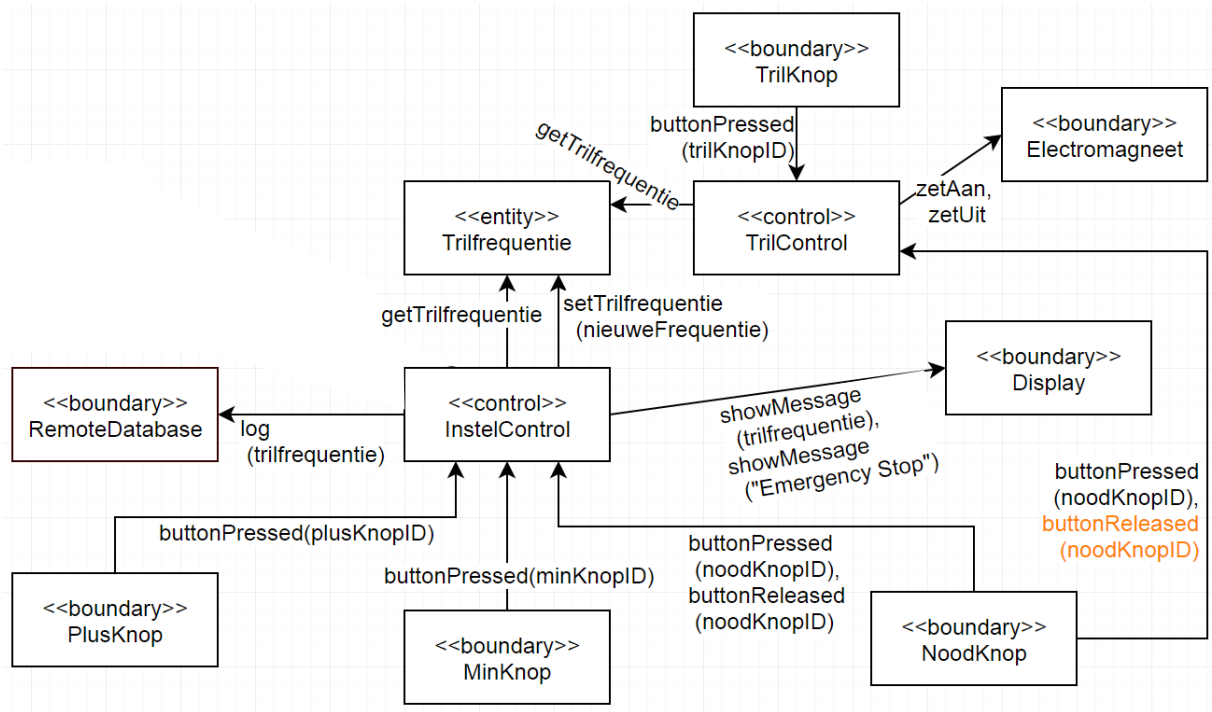
Ga na welk "ding" iets tegen welk "ander ding" "zegt"

- We hebben het over de use case Trillen, dus TrilControl moet het object zijn dat in deze zin luistert naar het de NoodKnop en moet zorgen dat de TrilControl zijn normale werk weer kan gaan doen.
  NoodKnop moet TrilControl dus laten weten dat hij is uitgetrokken.

Kortom:

- NoodKnop zegt buttonReleased(noodKnopID) tegen TrilControl.

Zet wat er "wordt gezegd" als bericht langs een pijl vanuit het sprekende object naar het luisterende object.



Dat was de laatste zin van de laatste use case.

Taakstructurering

Zodra de lijst van objecten en hun beschrijvingen compleet is, moet aan elk van de objecten een van de volgende “taaksoorten” worden toegekend:

- Periodical
  -> wanneer iets met vast interval moet worden gedaan, zoals pollen.
- Demand driven
  -> overig: gaat pas iets doen als er een bericht binnenkomt.
- Geen taak
  Soms kun je direct al besluiten dat een object geen eigen taak nodig heeft, bijvoorbeeld als hetgeen wat hij doet vrijwel instantaan is. Voorbeelden: entitiy objecten, leds.

Taakstructurerings-tabel

De objecten en hun klassificering zet je op een rijtje in een tabel:

Objecten	Taaksoort	Periode	Deadline	Prioriteit

Behalve de objectnaam en de taaksoort vul je per object ook een deadline en eventueel een periode in.

Periode

Objecten met als taaksoort “Periodical” komen met de regelmaat van een klok in actie om iets te doen.

Deadline

Een deadline is de tijd die het rtos erover mag doen om een taak de beurt te geven nadat een event waar hij op wacht heeft gevuurd.

Hoe de deadline te kiezen?
-> Geef iets eerst een belachelijk grote deadline, en bedenkt wat er dan mis gaat, of wat er ergerlijk is. Verlaag vervolgens de deadline, tot het nog net acceptabel is.

NB:

- Als iets niet heel tijd-kritisch is (bijvoorbeeld het doorgeven van de hoeveelheid zon voor een zonnescherm), dan mag daar best lang (een minuut?) over gedaan worden. Dat het in praktijk binnen een microseconde afgehandeld wordt, speelt daarbij geen rol.
- Vaak is van belang hoe snel wordt gereageerd op een treintje van berichten.
  Bijvoorbeeld Noodrem->NoodstopControl->Schijfremmen
  Als je wilt dat na het trekken van de Noodrem niet langer dan 200ms gewacht mag worden voordat de Schijfremmen hun werk kunnen gaan doen, dan betekent dat dat het gezamenlijk totaal van de deadlines voor de objecten Noodrem, NoodstopControl en Schijfremmen 200ms moet zijn. Stel dat de Schijfremmen passief (instantaan) reageren, dan kan op basis van dit criterium de 200ms dus verdeeld worden over NoodstopControl en Noodrem. Bijvoorbeeld elk 100ms (tenzij er nog een ander pad is, waaruit volgt dat een van beide kritischer moet zijn).

- Het object met de kleinste deadline moet immers ook zijn deadline kunnen halen. Om dat mogelijk te maken (helemaal in een cooperative rtos), mag als vuistregel **elke** taak maximaal onafgebroken code draaien gedurende **1/5e van de kortste deadline**.

  Bijvoorbeeld, je hebt:
  - Een BitSend object met een deadline van 1ms
  - Een display met een deadline van 200ms.

    De display is dan kennelijk niet zo tijdkritisch, maar hij moet desondanks vaak genoeg zijn thread afstaan (middels een vTaskYield callbijvoorbeeld, elke 100us) om ervoor te zorgen dat het BitSend object vaak genoeg de kans krijgt om zijn deadline te halen.

Prioriteit

Nadat alle deadlines zijn ingevuld, geef je de objecten prioriteiten. Hogere prioriteiten hebben lagere nummers. Dus prioriteit 1 is een hogere prioriteit dan prioriteit 2. Vuistregel: des te korter de deadline, des te hoger de prioriteit.

Taakstructurering

In het voorgaande hebben we op basis van de gegeven use cases en hun beschrijvingen een object model gemaakt, compleet met een lijst van objecten en hun beschrijvingen:

Taaksoort en deadline

Eerst vullen we per object de taaksoort en de deadline in:

Objecten	Taaksoort	Periode	Deadline
InstelControl	Intern, Demand Driven		100ms
TrilControl	Intern, Demand Driven		50ms
Trilfequentie	Geen		
Display	IO, Demand Driven		100ms
PlusKnop	IO, Periodical		50ms
MinKnop	IO, Periodical		50ms
NoodKnop	IO, Periodical		50ms
TrilKnop	IO, Periodical		50ms
RemoteDatabase	IO, Demand Driven		3 minuten

Het waarom:

- Respons op Display
  Als op een knop wordt gedrukt, zit het net in mijn irritatiezone als het scherm pas een kwart seconde later een reactie daarop laat zien. Dat betekent dat het treintje PlusKnop->InstelControl->Display in totaal niet meer dan 250 ms mag verbruiken.
- Reactiesnelheid op Noodknop
  Als op de noodknop wordt gedrukt, moet het apparaat snel, laten we zeggen binnen 100ms stoppen, anders wordt er te veel schade gedaan. Dat betekent dat het treintje NoodKnop->TrilControl->Magneet niet meer dan 100ms mag gebruiken. Het treintje NoodKnop->InstelControl->Display toont alleen de noodstop. Het is niet erg(erlijk) als dat in totaal 250 ms duurt.
- PlusKnop, MinKnop, Knoppen
  Ik wil een knop snel achtereenvolgens kunnen indrukken, en dat elk van mijn indrukkingen worden geregistreerd. Met een deadline van 50ms moet dat lukken. Sneller dan dat achtereenvolgens indrukken kan fysiologisch niet.
- Display en InstelControl
  Met 50ms voor de PlusKnop blijft er 200ms over voor de rest van het treintje voor voldoende snelle respons op display. Ik verdeel die gelijkelijk over InstelControl en de Display, dus ieder 100ms.
- TrilControl
  Voor de reactiesnelheid op de NoodKnop hadden we bedacht dat dat binnen 100ms moet kunnen. Als we uitgaan voor 50ms budget van de Noodknop (net als de andere knoppen), blijft 50ms over voor TrilControl.

- TrilFrequentie
  TrilFrequentie is een entiteit. Een ding met getters en setters. Vaak werken die functies bijna instantaan, en is de overhead van een aparte taak niet de moeite waard. Dus: geen taak.
- RemoteDatabase
  De aansturing van de RemoteDatabase is “fire and forget”. Je stuurt er iets naartoe, en het wordt wel afgehandeld. Dat mag best een tijdje duren, zolang er maar een voldoende groot channel wordt gebruikt om de log-opdrachten te bufferen.

Periodetijden

Vervolgens kunnen we de voor de periodieke taken de periodetijden invullen:

Objecten	Taaksoort	Periode	Deadline
InstelControl	Intern, Demand Driven		100ms
TrilControl	Intern, Demand Driven		50ms
Trilfequentie	Geen		
Display	IO, Demand Driven		100ms
PlusKnop	IO, Periodical	50ms	50ms
MinKnop	IO, Periodical	50ms	50ms
NoodKnop	IO, Periodical	50ms	50ms
TrilKnop	IO, Periodical	50ms	50ms
RemoteDatabase	IO, Demand Driven		3 minuten

Vuistregel: kies een periodetijd die gelijk is aan de deadline. Dan is de taak binnen een periode afgehandeld.

Prioriteiten

Vervolgens kiezen we de prioriteiten. De hoogste prioriteit, 1 (sommigen kiezen 0) is voor de taak met de kortste deadline (50ms). Daarna volgen de taken met een dealine van 100ms, daarna de taak met een deadline van 3 minuten.

Objecten	Taaksoort	Periode	Deadline	Prioriteit
InstelControl	Intern, Demand Driven		100ms	2
TrilControl	Intern, Demand Driven		50ms	1
Trilfequentie	Geen			
Display	IO, Demand Driven		100ms	2
PlusKnop	IO, Periodical	50ms	50ms	1
MinKnop	IO, Periodical	50ms	50ms	1
NoodKnop	IO, Periodical	50ms	50ms	1
TrilKnop	IO, Periodical	50ms	50ms	1
RemoteDatabase	IO, Demand Driven		3 minuten	3

Taken samenvoegen

Het laatste deel van de taakstructurering bestaat uit het eventueel samenvoegen van taken. Dat samenvoegen kan op grond van verschillende soorten “cohesie” (dat is een ander woord voor “samenhang”).

Je kunt die samenhang op vele fancy wijzen ontdekken en toepassen maar wij adviseren om het alleen te doen volgens deze vuistregels
(reden: het moet zoden zetten en dingen niet veel complexer maken):

- ### Temporele Cohesie:
  Periodieke taken met dezelfde periode, deadline en prioriteit kun je samennemen.
- ### Control cohesie:
  Demand Driven taken die alleen worden aangestuurd door een enkel ander object kun je soms samenvoegen met dat andere object. Dat kan alleen als wat ze moeten doen “acceptabel kort is”.
  Voorbeeld: in het object model zien we dat Display alleen door InstelControl wordt aangestuurd. In de taakstructureringstabel zien we dat de Display een deadline heeft van 100ms. Als het acceptabel zou zijn dat InstelControl 100ms moet wachten tot de display update klaar is voordat het kan reageren op nieuwe frequentie-aanpassingen, dan zouden we de taak van de display dus kunnen samenvoegen met die van InstelControl.
  Het lijkt mij echter fijner dat de trilfrequentieaanpassing zo snel mogelijk wordt verwerkt na het indrukken van een knop. Vandaar dat ik ervoor heb gekozen om display zijn eigen taak te laten houden.
- ### Sequentiele cohesie:
  Als duidelijk is dat twee zekere taken nooit tegelijkertijd “actief” zijn / buiten hun “Idle state” werken, kun je ze in een noodsituatie (geheugengebrek oid) samenvoegen op basis van sequentiële cohesie . In deze cursus ben ik dat tot nog toe niet tegengekomen.

Wat betekent “Samenvoegen”

Dat kan twee dingen betekenen:

1. Een van de objecten verliest zijn taak. De dingen van het andere object worden (ook) in de thread van de taak van de aanroeper uitgevoerd.
2. In het geval van temporele cohesie:
   Alle betrokken objecten verliezen hun eigen taak. Er komt een extra “Handler” object met een taak. Vanuit dat object worden periodiek “update” functies van de betrokken objecten aangeroepen. Zodoende voeren alle betrokken objecten periodiek hun “ding” uit zonder dat elk van hen een eigen taak nodig heeft.

Waarom Samenvoegen

Elke taak heeft een zekere overhead in cpu time en geheugen. Het doel van taken was om dingen overzichtelijker te maken. Door het samenvoegen van taken kan een balans worden gevonden tussen die twee wensen. Neem bijvoorbeeld de snoepautomaat. Die had 40 buttons en 40 actuatoren om de candy bars uit te werpen. Zonder handler objecten zou je daar 80 afzonderlijke taken voor nodig hebben gehad. Met handler objecten kun je dat op nog steeds overzichtelijke manier met 2 taken af.

Taken Samenvoegen

In het voorgaande kwamen we uit op onderstaande tabel:

Objecten	Taaksoort	Periode	Deadline	Prioriteit
InstelControl	Intern, Demand Driven		100ms	2
TrilControl	Intern, Demand Driven		50ms	1
Trilfequentie	Geen			
Display	IO, Demand Driven		100ms	2
PlusKnop	IO, Periodical	50ms	50ms	1
MinKnop	IO, Periodical	50ms	50ms	1
NoodKnop	IO, Periodical	50ms	50ms	1
TrilKnop	IO, Periodical	50ms	50ms	1
RemoteDatabase	IO, Demand Driven		3 minuten	3

Contol cohesie?

RemoteDatabase en Display worden alleen aangestuurd door InstelControl. We kunnen dus even bekijken of control cohesie voor hen van toepassing is.

Voor Display geen control cohesie:
(onderstaande is ook als voorbeeld gebruikt in de voorgaande theorie)

- In het object model zien we dat Display alleen door InstelControl wordt aangestuurd. In de taakstructureringstabel zien we dat de Display een deadline heeft van 200ms. Als het acceptabel zou zijn dat het InstelControl 200ms moet wachten tot de display update klaar is voordat het kan reageren op nieuwe frequentie-aanpassingen, dan zouden we de taak van de display dus kunnen samenvoegen met die van InstelControl.
  Het lijkt mijn echter fijner dat de trilfrequentieaanpassing zo snel mogelijk wordt verwerkt na het indrukken van een knop. Vandaar dat ik ervoor heb gekozen om display zijn eigen taak te laten houden.

Voor RemoteDatabase geen control cohesie:

- De deadline van RemoteDatabase is zelfs groter dan die van Display. Op dezelfde gronden kunnen we voor dit object dus besluiten om geen samenvoeging op basis van control cohesie toe te passen.

Temporele Cohesie

Er zijn 4 periodieke taken die knop-berichten versturen. Ze pollen met dezelfde periode en deadline, en hebben dezelfde prioriteit. In de voorgaande theorie zagen we dat je in zo'n geval de taken kunt samenvoegen in een taak die een nieuw object heeft: een zogenaamd Handler object.
In dit geval komt er dus een ButtonHandler object dat de taak heeft, en elke 60ms voor elke knop de update() functie aanroept.

Nu voegen we een extra kolom aan de linkerkant van de taakstructureringstabel toe waar we de naam van de taak invullen. Normaal gesproken is dat de naam van het object dat de taak heeft, maar dan met het woord "Taak" in de naam. In de kolom Objecten staan alle objecten die worden

uitgevoerd in de betreffende taak. Het eerste object is het object dat de taak heeft (in dit geval van ButtonTaak is dat dus de ButtonHandler).

Taak	Objecten	Taaksoort	Periode	Deadline	Prioriteit	FreeRTOS Prioriteit
InstelTaak	InstelControl	Intern, Demand Driven		100ms	2	2
TrilTaak	TrilControl	Intern, Demand Driven		50ms	1	3
	Trilfequentie	Geen				
DisplayTaak	Display	IO, Demand Driven		100ms	2	2
ButtonTaak	ButtonHandler, PlusKnop, MinKnop, NoodKnop, TrilKnop	IO, Periodical	50ms	50ms	1	3
DatabaseTaak	RemoteDatabase	IO, Demand Driven		3 minuten	3	1

Sidenote: of toetsen nu automatisch terugveren of niet, de berichten die ze verzenden (buttonpressed en buttonreleased) zullen in het algemeen vergelijkbaar zijn.

Prioriteiten in FreeRTOS

Als je in FreeRTOS een taak aanmaakt via xTaskCreatePinnedToCore, dan geldt voor zijn parameter taskPriority: hoe hoger dat getal, hoe **hoger** de prioriteit. Dus in omgekeerde volgorde. Dat zie je bovenstaand terug in de laatste kolom.

(niet te verwarren met interrupt-prioriteiten in FreeRTOS : daarvoor geldt wel weer: kleinere getallen betekenen hogere prioriteiten).

Klassediagram maken

Het opstellen van het klassediagram kan in de volgende stappen worden gedaan:

1. **Initiele Klassen**
   Maak een kopie van het object model. Vervang alle overeenkomstige objecten door klassen.
2. **Publieke operaties**
   Schuif de berichten langs de pijlen in de richting van de pijl door naar de klasse die zich daar bevindt. Het bericht wordt daar geimplementeerd als publieke functie. Met andere woorden: elk bericht wordt een publieke operatie bij zijn ontvanger.
3. **Verwijder dubbele pijlen**
   Dubbele pijlen zijn dubbelop in een klassediagram.
4. **Representeer gelijksoortige objecten door een enkele klasse**
5. **Composition / Member variabelen**
   Het is handig dat wanneer je een object aanmaakt, dat dat object zelf automatisch zoveel mogelijk de objecten aanmaakt die hij nodig heeft. Dat doe je door het toevoegen van de benodigde objecten als private member variabelen in de member sectie van de klasse. Probeer dus als het even kan composition toe te passen.
   Vuistregel: als een object A alleen communiceert met object B, kun je overwegen om B eigenaar te maken van object A. Test dan nog even of “B heeft een A” natuurlijk klinkt.
   Regel: Zodra B eigenaar is van object A, kan een composition-relatie van B naar A worden getekend (het zwarte wiebertje aan de kant van de eigenaar).
   Als er al een pijl van B naar A liep, mag deze dan worden verwijderd.
   Immers: eigenaarschap impliceert kennen: als je een member variabele hebt, kun je zijn member functies aanroepen. Je hebt daar dan geen reference meer voor nodig.
6. **Member references**
   De pijlen in het klassediagram die niet zijn omgezet in composition relaties betekenen alleen nog “kennen”, geen eigenaarschap (want de pijlen die eigenaarschap vertegenwoordigden zijn in de vorige stap al omgezet naar compositie-relaties). Voeg voor elk object dat een klasse “kent” een private member reference variabele toe.
7. **Listener Patterns**
   Klassen die berichten versturen naar andere klassen moeten die andere klassen kennen. Daardoor werken ze alleen met die andere klassen. Als je klassen algemener toepasbaar/herbruikbaar wilt maken, moet je ze hun berichten laten sturen naar een zogenaamde “Listener”. Dat is een interface klasse die de juiste functies voor de berichten bevat. Zodoende kunnen alle klassen die afleiden van zo’n interface klasse (ook toekomstige klassen) probleemloos naar de verstuurde berichten “luisteren”. Buttons zijn bijvoorbeeld typisch klassen die je in meerdere designs wilt kunnen hergebruiken. *(TODO: insert doc over listener patterns).*
8. **Markeer isActive**
   Markeer klassen die een taak hebben met het (extra) stereotype <<isActive>>.
9. **Voeg Handler klassen toe**
   Voeg de extra Handler klassen toe die zijn ontstaan als gevolg van de taaksamenvoeging van periodieke taken.
10. **Weak dependencies**
    Het kan zijn dat een klasse in een van de functie-parameters van een van zijn functie een

klasse gebruikt waarvan hij verder geen member of reference member heeft. In dat geval laat je de afhankelijkheid zien mbv een “weak dependency” relatie (een gestippelde pijl).
Voorbeeld: Stel je voor dat je Display klasse een DrawRect(rect:Rect) functie heeft.
De Rect wordt alleen als tijdelijke variabele meegegeven aan de DrawRect functie. De Display klasse heeft verder geen members of member references van type Rect. In dat geval is de Dislay “weak dependent” van Rect, en loopt er een gestippelde pijl van Display naar Rect om dat aan te geven. Hetzelfde geldt als een tijdelijk object van een klasse in een functiebody wordt aangemaakt. Bijvoorbeeld als je display klasse de functie update() heeft, en die functie in zijn body een tijdelijk Rect object aanmaakt.
Het nut van het ook het weergeven van de weak dependencies is dat als je bijvoorbeeld een library van je klassen wilt maken, dat je ziet van welke andere libraries die allemaal afhankelijk moet zijn en waarom.

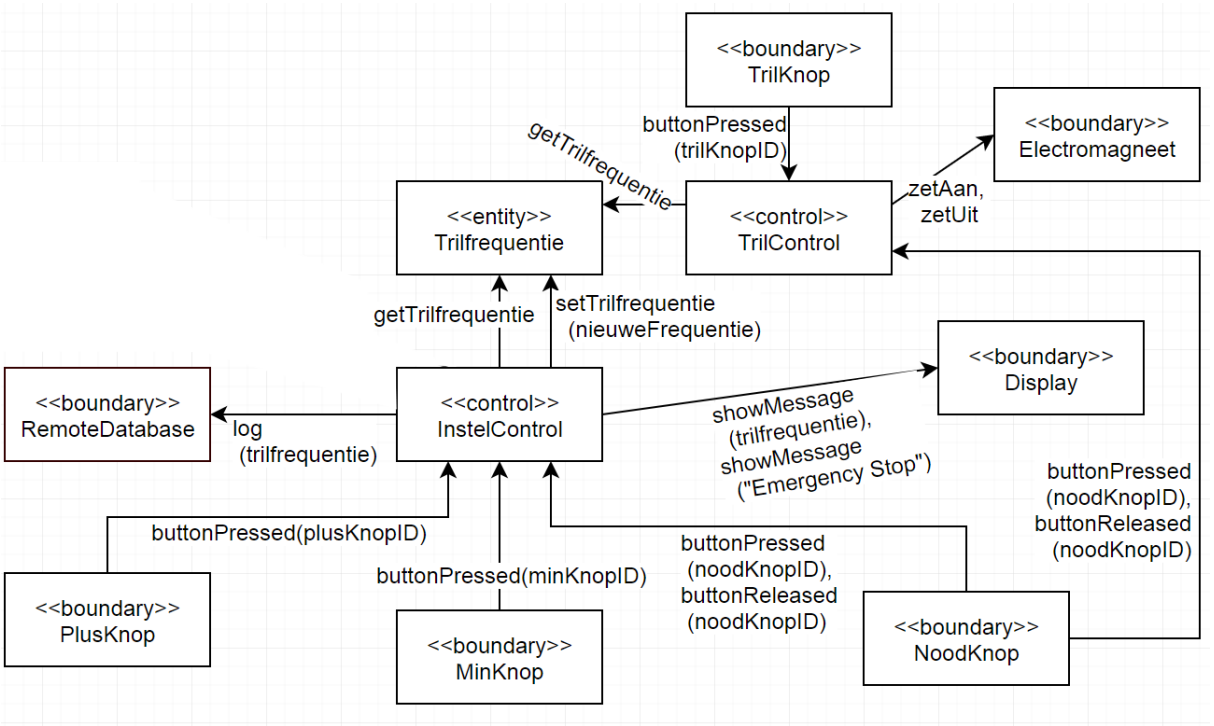
11. **Multiplicity**
    Zorg dat aan het uiteinde van de de associatiepijlen en de compositie-relaties de juiste multipliciteit in het klassediagram is aangegeven.
    Tel eenvoudigweg hoeveel members en reference members een klasse heeft.
    Voorbeelden:
    a. 1..3 betekent: 1 tot 3.
       Bijvoorbeeld in het geval van een array met 3 entries waar pointers in opgeslagen kunnen worden.
    b. * betekent: 0 of meer.
       Bijvoorbeeld in het geval van een vector.
    c. 2..* betekent: 2 of meer.
       Bijvoorbeeld: .. ik kan even niets bedenken. Dit komt niet vaak voor.
    d. Niets aangeven betekent: 1.

Klassediagram maken

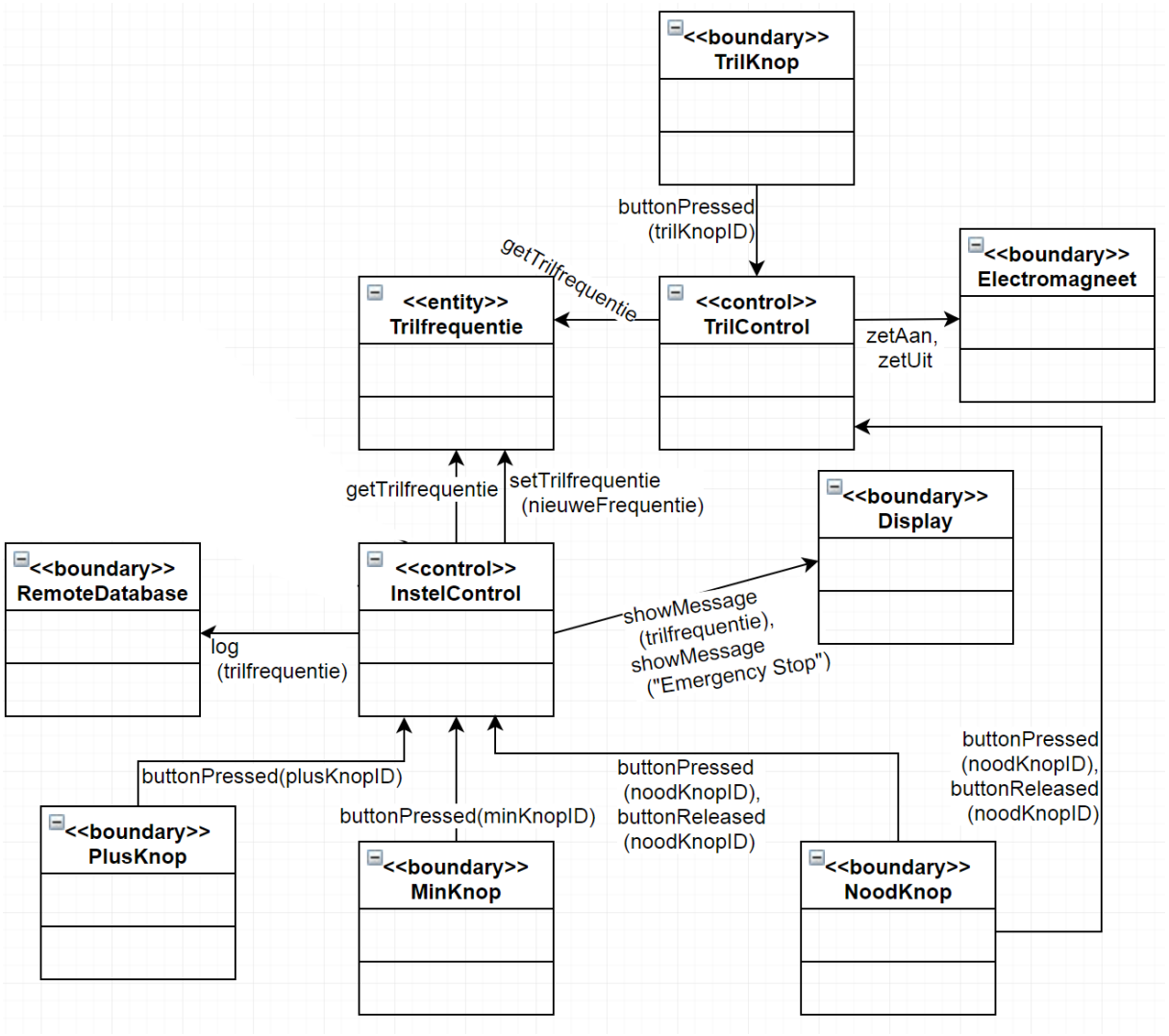
Stap 1: **initiele klassen**

- Maak een kopie van het object model.
- Vervang alle objecten door klassen.

Dit was het object model waar we op waren uitgekomen:



Onderstaand zijn de objecten vervangen door klassen (dwz de blokken krijgen 3 compartimenten per blok). NB: het mag nog geen klasse-diagram heten, maar dit is de eerste stap in de transitie van Object model naar Klasse diagram.



Stap 2: publieke operaties

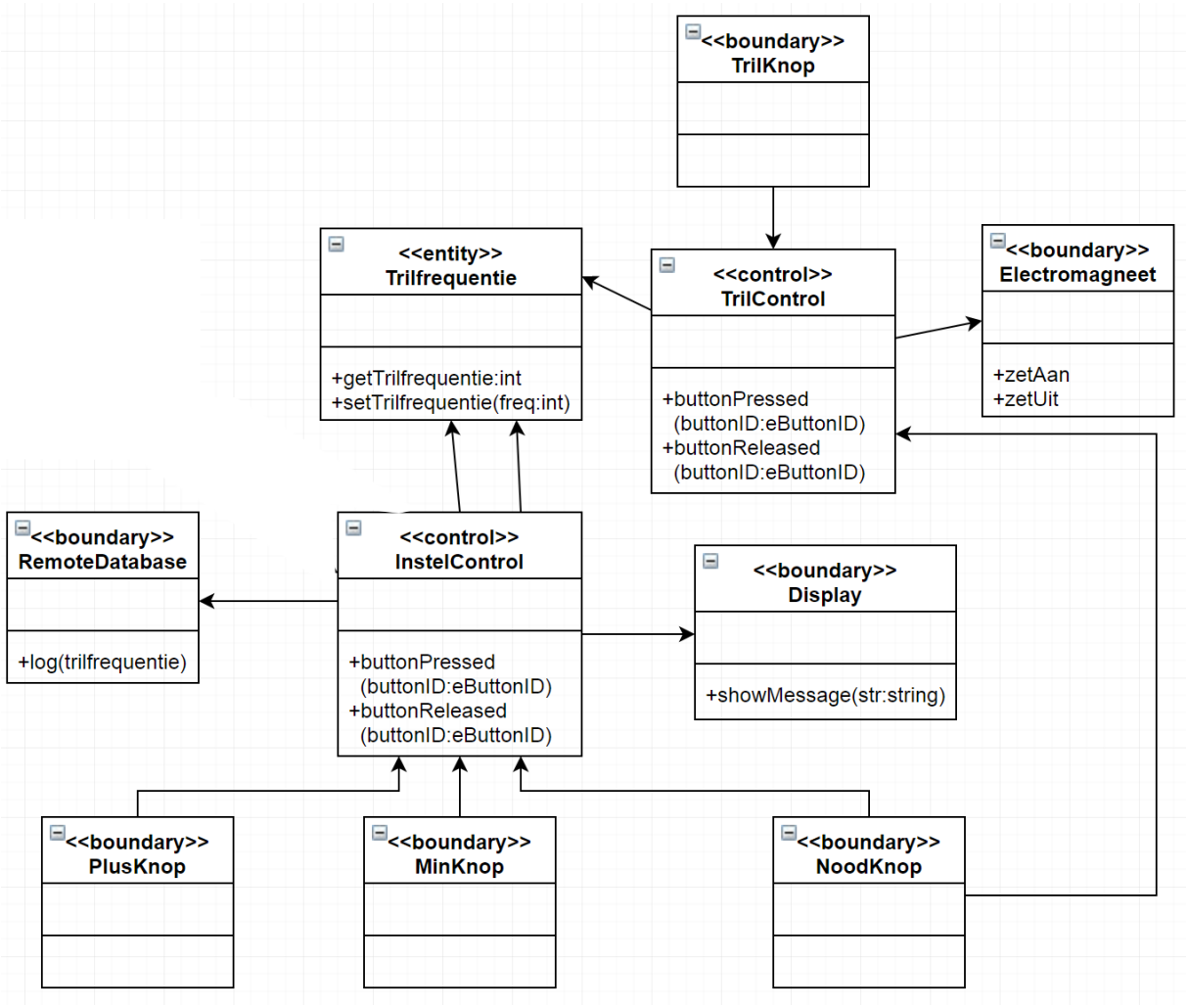
Schuif de berichten langs de pijlen in de richting van de pijl door naar de klasse die zich daar bevindt. Het bericht wordt daar geimplementeerd als publieke functie. Met andere woorden: elk bericht wordt een publieke operatie bij zijn ontvanger.

Voorbeelden:

- In bovenstaande diagram zie je dat InstelControl het bericht getTrilfrequentie stuurt naar de entiteit Trilfrequentie. Daarom geven we de entiteit Trilfrequentie een publieke functie met de naam getTrilfrequentie.
- Je ziet dat de PlusKnop een bericht buttonPressed(plusKnopID) stuurt naar InstelControl. Dat bericht is van dezelfde vorm als het bericht buttonPressed(minKnopID) dat MinKnop stuurt naar InstelControl. We geven InstelControl daarom een publieke functie buttonPressed(buttonID:eButtonID). Voor de functieparameter buttonID kies ik een enum type, genaamd eButtonID: enum eButtonID {PlusKnopID,MinKnopID,NoodKnopID,TrilKnopID}. Zo kan aan de functie buttonpressed een identifier van de knop worden meegegeven. Binnen de functie kan dan middels bijvoorbeeld een switch statement gekeken worden om welke knop het gaat en welke dus actie nodig is.

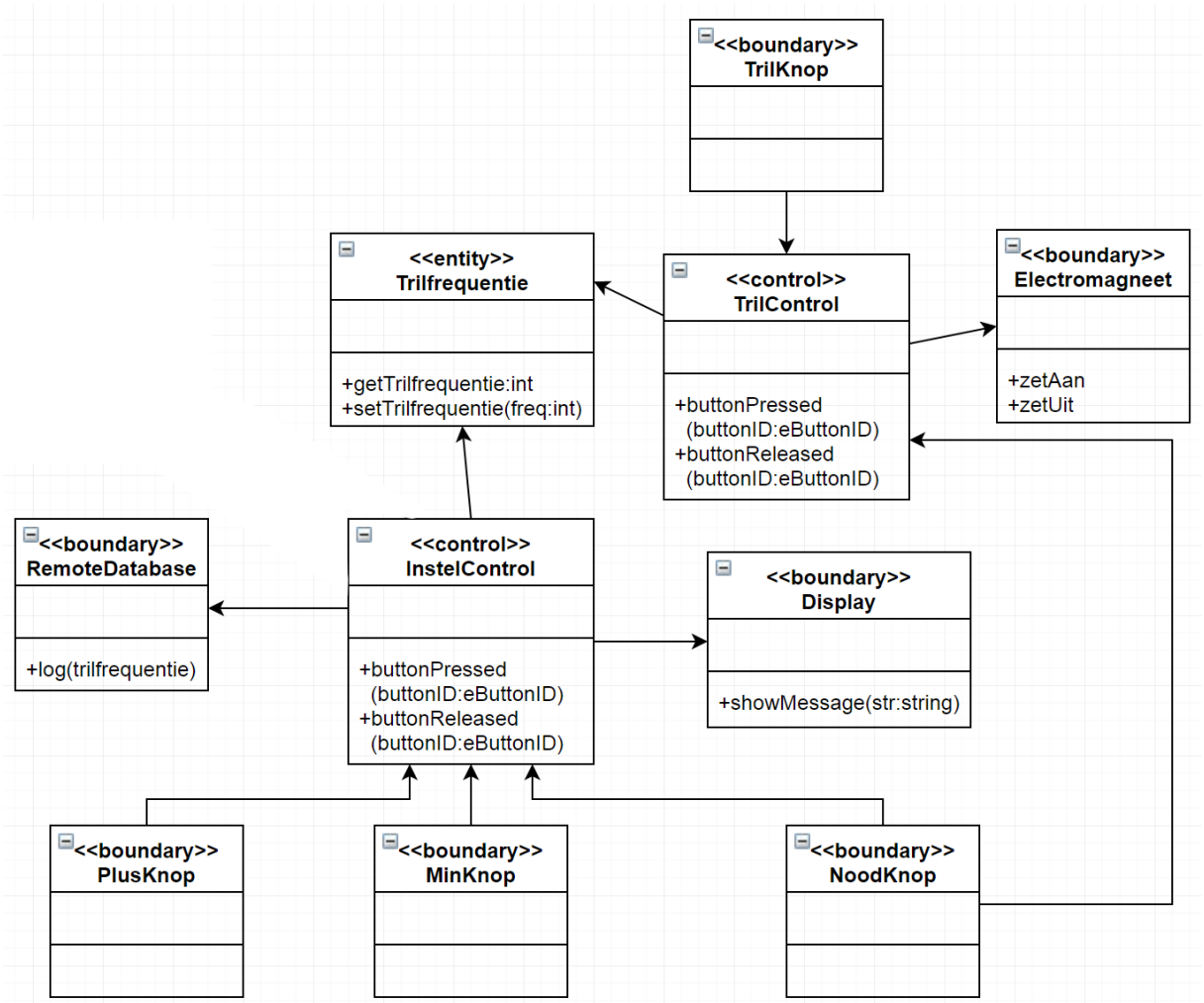
- Je ziet verder dat InstelControl twee verschillende showMessage berichten stuurt naar de Display. Ik kies ervoor om Display een publieke functie showMessage(str:string) te geven. Het string argument is flexibel genoeg om de twee verschillende berichten te kunnen overbrengen.

Na het omzetten van de berichten in publieke operaties ziet ons halffabrikaat klassediagram er zo uit:



Stap 3: verwijder dubbele pijlen

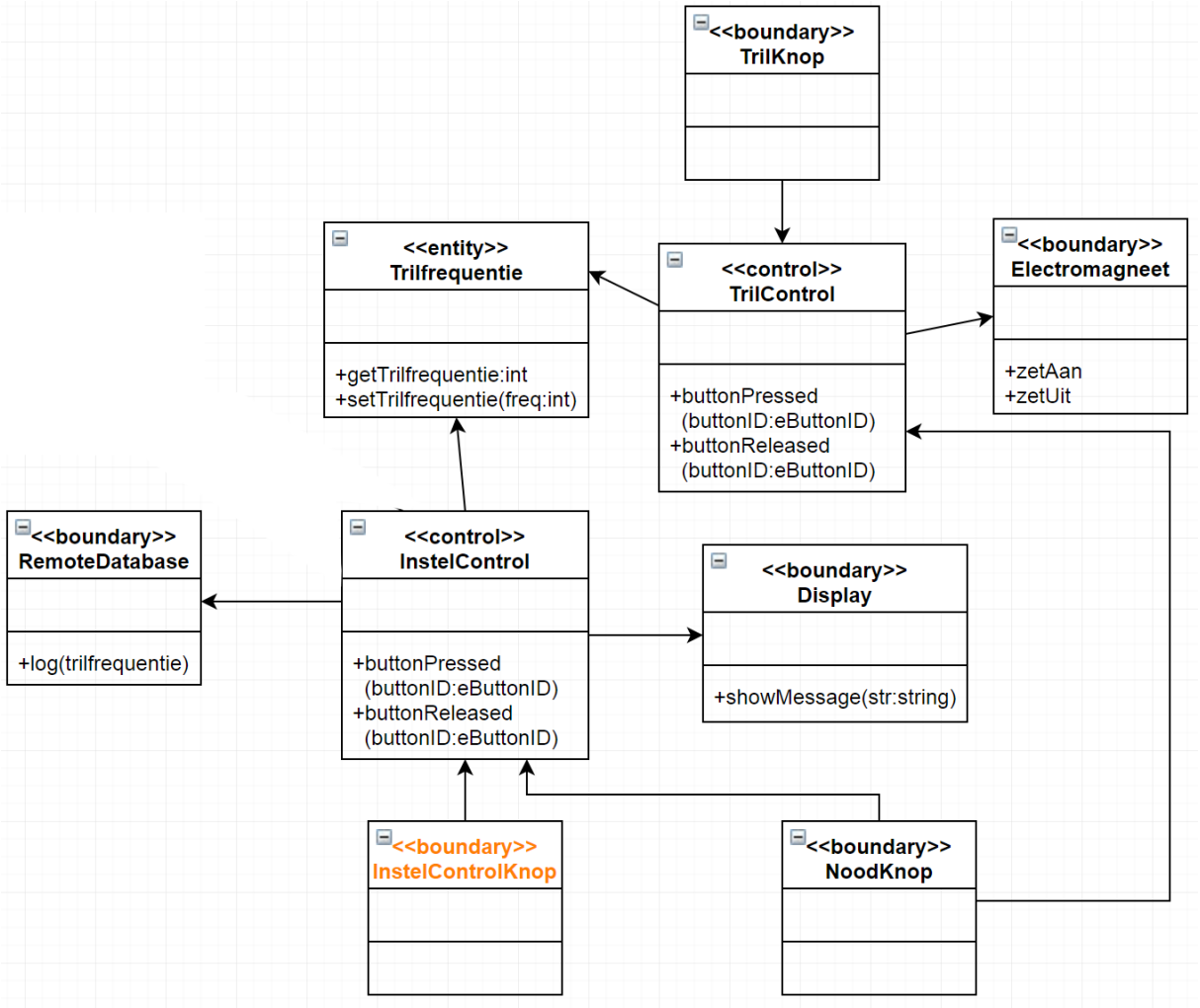
In het bovenstaande diagram zie je vanuit InstelControl twee pijlen lopen naar Trilfrequentie. Een enkele pijl volstaat om aan te geven dat functies uit de klasse Trilfrequentie worden aangeroepen vanuit de klasse InstelControl. Na weglaten van een van beide pijlen blijft dit dus over:



Stap 4: Representeer gelijksoortige objecten door een enkele klasse

In het bovenstaande diagram zijn PlusKnop en MinKnop 100% gelijksoortig: beide roepen ze de functie buttonPressed en buttonReleased van een object van klasse InstelControl aan. We kunnen daarom de objecten PlusKnop en MinKnop prima instanties maken van een enkele klasse. Die klasse noem ik InstelControlKnop.

Na deze samenvoeging ziet het klassediagram-halffabrikaat er zo uit:



NB: NoodKnop kunnen we (nog) niet samenvoegen met InstelControlKnop, want objecten van type NoodKnop sturen hun berichten ook naar een object van type TrilControl.

Stap 5: Composition / Member variabelen

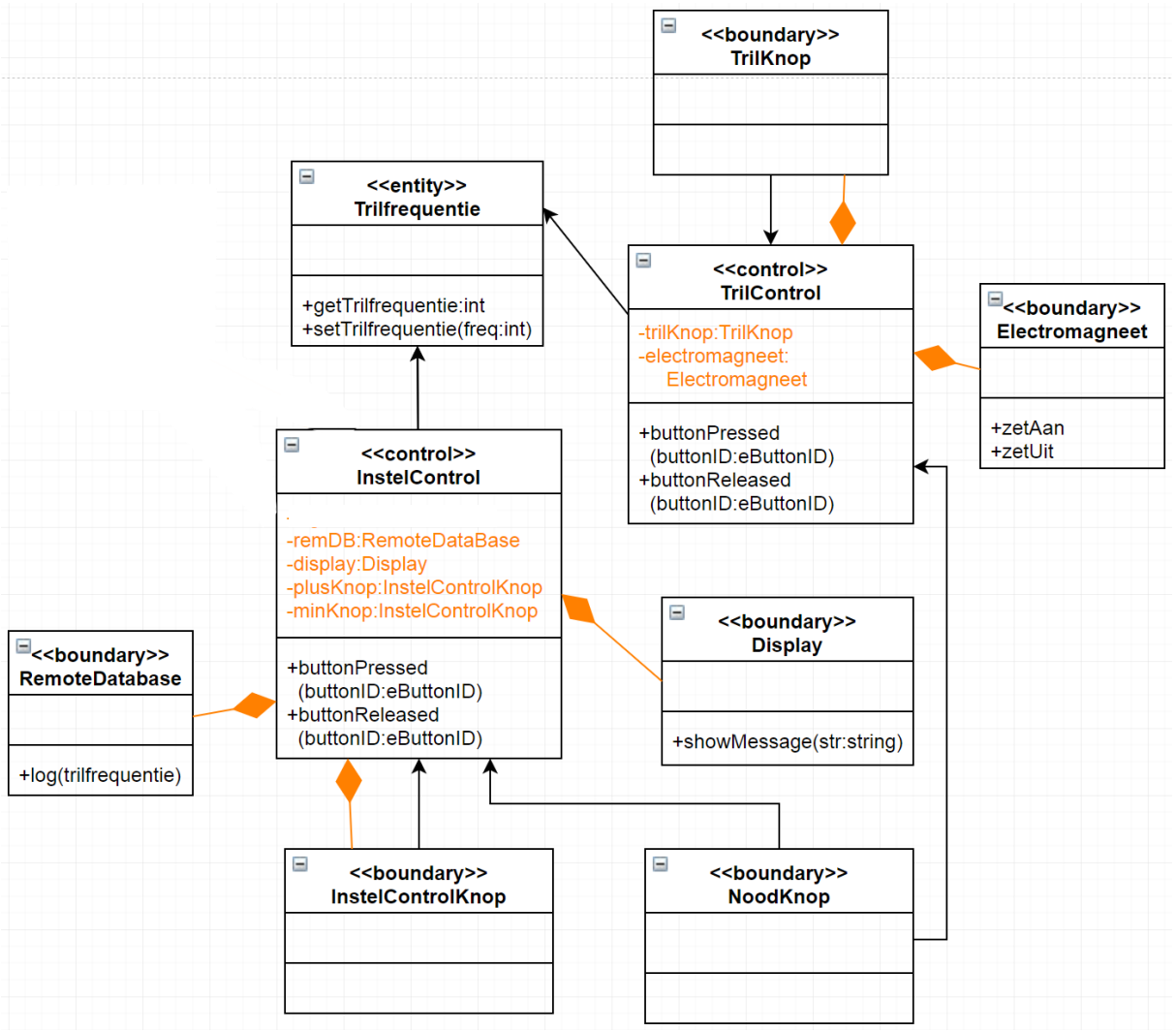
Het is handig dat wanneer je een object aanmaakt, dat dat object zelf automatisch zoveel mogelijk de objecten aanmaakt die hij nodig heeft. Dat doe je door het toevoegen van de benodigde objecten als private member variabelen in de member sectie van de klasse.

Vuistregel: als een object A alleen communiceert met object B, kun je overwegen om B eigenaar te maken van object A. Test dan nog even of “B heeft een A” natuurlijk klinkt.

- Display, InstelControlKnop en RemoteDatabase communiceren alle alleen met InstelControl. Het licht daarom voor de hand om hen allen een member variabele van InstelControl te maken. Zodra je een InstelControl object aanmaakt, worden dan automatisch objecten van de genoemde klassen aangemaakt. Dat hoeft de programmeur die een instantie van InstelControl codeert dan niet meer zelf “handmatig” te doen, waardoor zijn code compacter en overzichtelijker wordt.
  Gevolgen:

    - De pijlen vanuit InstelControl naar Display en RemoteDatabase kunnen worden vervangen door een compositie-relatie.
    - Er komt ook een compositie-relatie van InstelControl naar InstelControlKnop.
      De pijl vanuit InstelControlKnop naar InstelControl wordt daar NIET door vervangen. Immers: composition impliceert alleen dat de eigenaar zijn eigendom kent. Zonder de extra pijl vanuit eigendom naar eigenaar zou niet duidelijk zijn dat in dit geval het eigendom de eigenaar moet kennen. InstelControlKnop moet zijn eigenaar InstelControl immers kennen om er buttonPressed berichten naartoe te kunnen sturen.
      NB: InstelControlKnop is de klasse waarmee de objecten PlusKnop en MinKnop uit het object model worden geinstantieerd. InstelControl krijgt dus de members plusKnop en minKnop, beide van klasse InstelControlKnop.
- TrilKnop en Electromagneet communiceren alleen met TrilControl. Het licht dus voor de hand om die twee objecten als members toe te voegen aan TrilControl.
  Gevolgen:
    - De pijl vanuit TrilControl naar Electromagneet kan worden vervangen door een compositie-relatie.
    - Er komt ook een compositie-relatie van TrilControl naar TrilKnop.
      De pijl vanuit TrilKnop naar TrilControl wordt daar NIET door vervangen. TrilKnop moet zijn eigenaar TrilControl immers kennen om er buttonPressed berichten naartoe te kunnen sturen.
- NoodKnop stuurt zowel berichten naar InstelControl als naar TrilControl. Het is dus niet logisch om InstelControl of TrilControl eigenaar te maken van Noodknop (een object kan immers maar eigendom van een ander object zijn).
- Trilfrequentie ontvangt berichten van zowel InstelControl als van TrilControl. Om dezelfde reden kies ik ervoor om het een globaal object te laten zijn, en het niet in eigendom te geven van InstelControl of TrilControl.
  In het geval van Trilfrequentie zou je echter kunnen betogen dat het meer hoort bij InstelControl dan bij TrilControl: InstelControl is “de bron” van de informatie. TrilControl vraagt die alleen even op. Het zou dus ook verdedigbaar zijn geweest om Trilfrequentie een composite member te maken van InstelControl.
  In dat geval zou TrilControl er niet direct bij kunnen. Je zou dan bijvoorbeeld InstelControl een getTrilfrequentie member functie kunnen geven die dan door TrilControl wordt aangeroepen en de aanroep doorspeelt naar Trilfrequentie.

Na het toepassen van bovenstaande compositie ziet het klassediagram er zo uit:



Stap 6: Member References

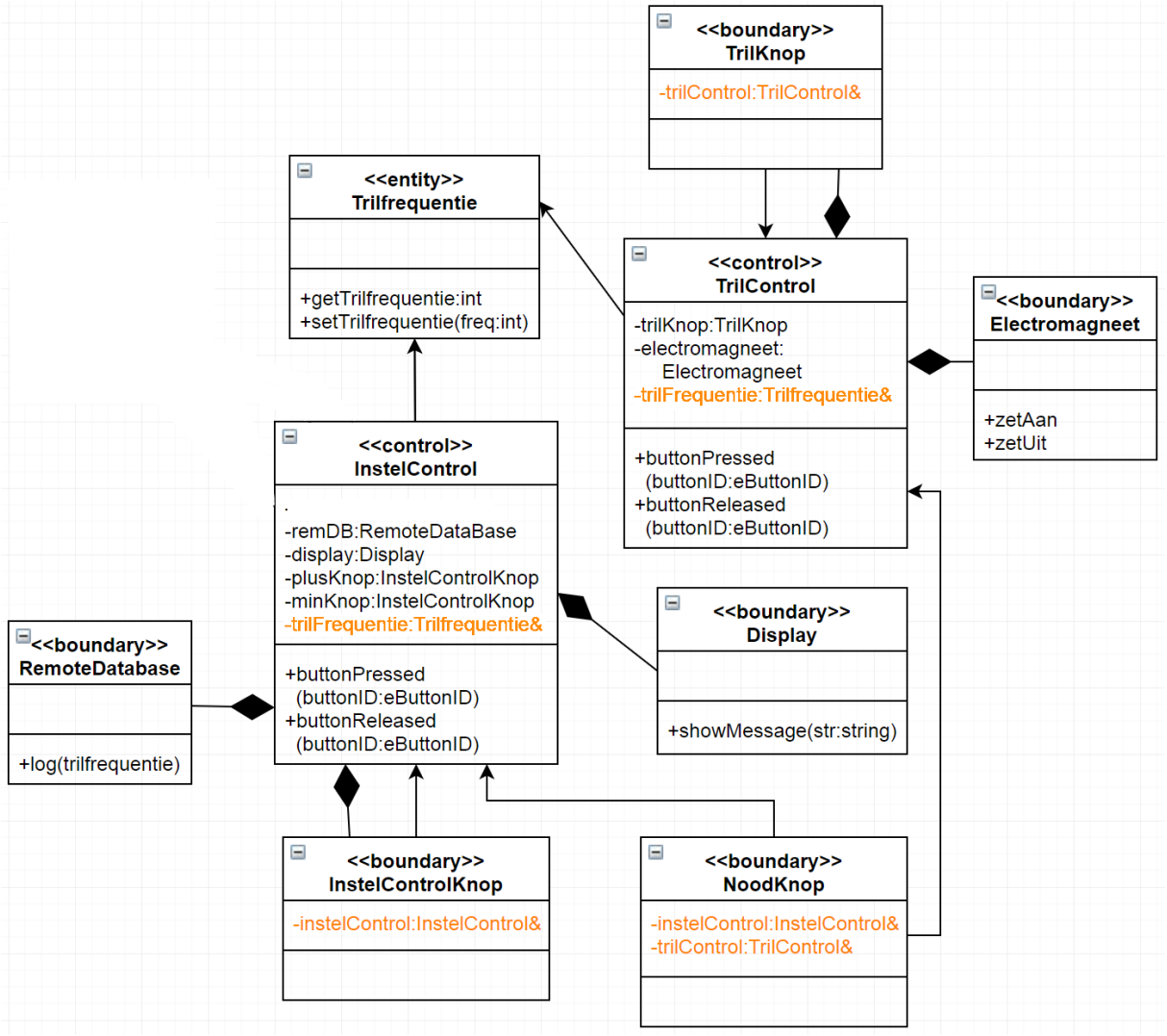
De pijlen in het klassediagram die niet zijn omgezet in composition relaties betekenen alleen nog "kennen", geen eigenaarschap (want de pijlen die eigenaarschap vertegenwoordigden zijn in de vorige stap al omgezet naar compositie-relaties). Voeg voor elk object dat een klasse "kent" een private member reference variabele toe.

Bijvoorbeeld:

- InstelControl kent een object van type Trilfrequentie. We voegen dus een reference member variabele van het type Trilfrequentie& toe aan InstelControl.

Etc..

Na deze exercitie ziet het klassediagram er als volgt uit:



Stap 7: Listener Patterns toepassen

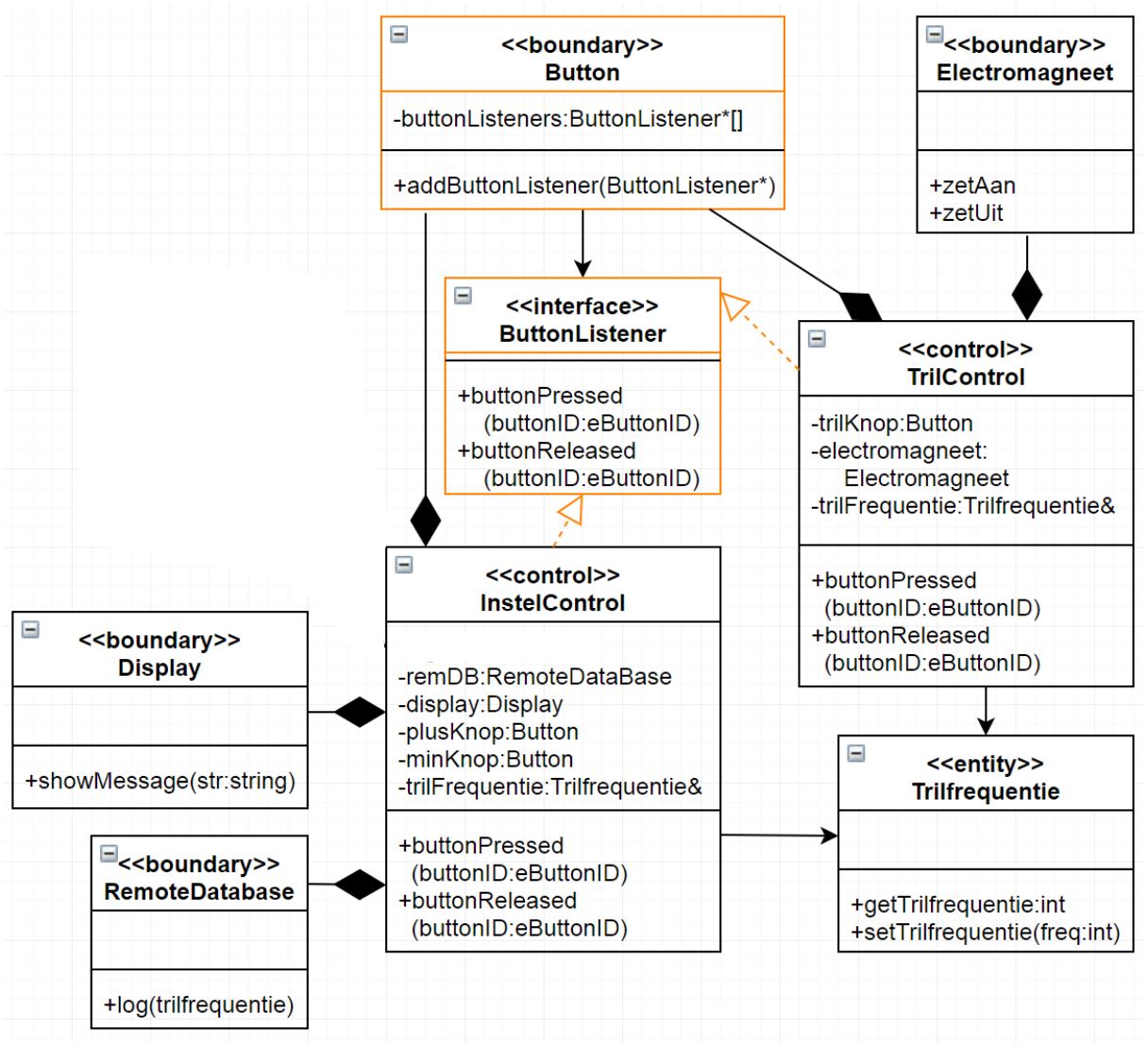
Klassen die berichten versturen naar andere klassen moeten die andere klassen kennen. Daardoor werken ze alleen met die andere klassen. Als je klassen algemener toepasbaar/herbruikbaar wilt maken, moet je ze hun berichten laten sturen naar een zogenaamde “Listener”. Dat is een interface klasse die de juiste functies voor de berichten bevat. Zodoende kunnen alle klassen die afleiden van zo’n interface klasse (ook toekomstige klassen) probleemloos naar de verstuurde berichten “luisteren”. Buttons zijn bijvoorbeeld typisch klassen die je in meerdere designs wilt kunnen hergebruiken.

- **ButtonListener**
  In bovenstaand diagram zien we verschillende klassen die alle bedoeld zijn om knop-instanties te maken. Ze fungeren hetzelfde. Alleen zijn de klassen die naar hun berichten luisteren anders. De belangrijkste regel van bijna alle patterns is: “abstraheer het veranderlijke”. Anders gezegd: verschuil alles wat kan veranderen achter een abstracte interface. Bij een listener pattern is dat niet anders. Het veranderlijke is de listener. Dus moet er een abstracte interface komen voor de listener. In dit geval noemen we die “ButtonListener”. Zodra die interface er is, kunnen de knoppen voortaan altijd tegen

diezelfde interface praten in plaats van tegen een specifieke klasse als InstelControl of TrilControl. We hoeven dan dus slechts een maal een Button klasse te maken met bijbehorende ButtonListener interface. Die kunnen we dan in alle situaties hergebruiken.

- **Button.addButtonListener**
  Sommige Buttons, zoals NoodKnop, sturen berichten naar meerdere listeners. In het algemeen zul je aan een knop daarom meerdere listeners willen kunnen opgeven waar hij zijn berichten naar moet versturen. Dat kan bijvoorbeeld door het toevoegen van een pointer naar een ButtonListener aan een array van pointers naar ButtonListener. De functie die dat doet noem ik addButtonListener.
- **Alleen voor klassen die berichten versturen**
  NB: Het listener pattern kun je alleen toepassen op klassen die berichten versturen.
  Een actuator als bijvoorbeeld een Servo, die alleen berichten ontvangt, kun je ook wel abstraheren met een interface. Maar dat is dan geen listener pattern (de Servo hoeft geen lijst met listeners bij te houden).

In onderstaande klassediagram is het listener pattern toegepast op de knoppen.



Nu valt op:

- Alle Knop Klassen zijn vervangen door een one and only knop klasse, genaamd Button.
- InstelControl en TrilControl krijgen nog steeds dezelfde berichten binnen voor de buttons, maar dit keer lopen die berichten via de ButtonListener.
- Door middel van het stereotype <<interface>> is aangegeven dat ButtonListener een interface klasse is.
- Het is ook te zien aan de inheritance relatie via welke van ButtonListener wordt geerft: die heeft om die reden (erven van een interface) een gestippelde lijn.
- Voordat het listener pattern was toegepast, kenden bijvoorbeeld TrilControl en TrilKnop elkaar. Er was sprake van een circular dependency. Dat geeft bij het programmeren onhandig gedoe met forward declarations (om het kip-ei probleem te doorbreken voor de compiler).
- Nadat het listener pattern is toegepast, is er geen circular dependency meer: TrilControl kent de Button klasse en de ButtonListener klasse waarvan hij afleidt. De Button kent alleen de ButtonListener klasse. Er is geen circeltje van afhankelijkheden meer voor de compiler.

Een weak dependency – of toch een normale associatie voor het noodKnop object?

In een klassediagram gaat een deel van de informatie uit het object model verloren. Toch kun je aan de member variabelen plusKnop, minKnop en trilKnop nog zien welke knoppen InstelControl en TrilControl heeft, en er dus vermoedelijk naar luistert.

Zoals gezegd, het mooist is als elke klasse die luistert naar objecten, aan die objecten zichzelf als listener toevoegt (via een aanroep van addButtonListener). De geeigende plek daarvoor is de constructor.
Nu was er ook een knop object, genaamd NoodKnop, die geen eigendom is van InstelControl en TrilControl, maar waarvan InstelControl en TrilControl toch berichten willen ontvangen.
Je kunt dan bijvoorbeeld een reference naar NoodKnop meegeven aan de constructor van InstelControl. Die kan dan die reference gebruiken om van dat NoodKnop object addButtonListener aan te roepen:

```
class InstelControl : ButtonListener
{
   Button plusKnop;
   Button minKnop;
   InstelControl(Button& noodKnop)
   {
       plusKnop.addButtonListener(this);
       minKnop.addButtonListener(this);
       noodKnop.addButtonListener(this);
   }
   …etc
```

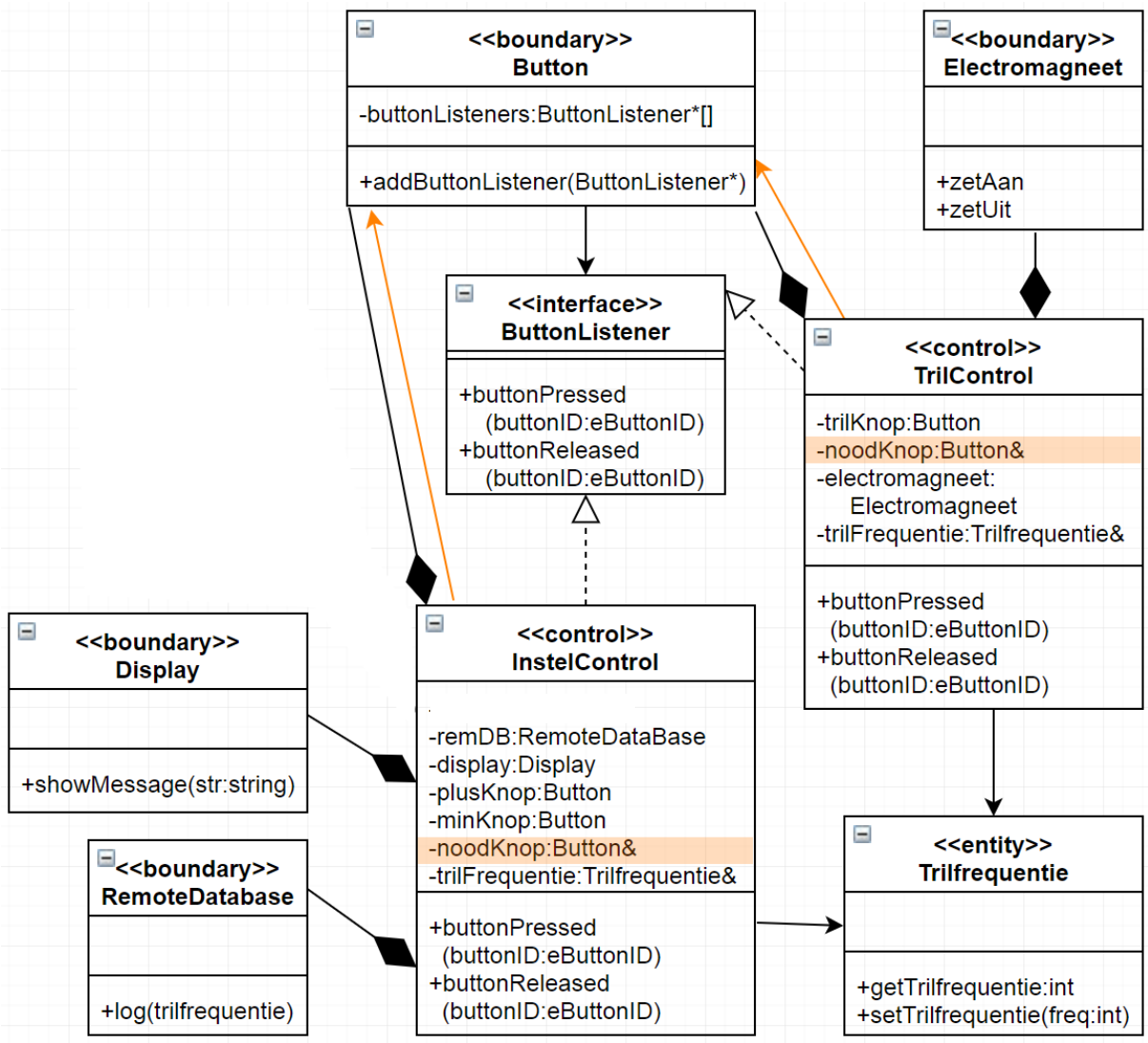
Naar het noodknop object zou InstelControl dan een weak dependency hebben.
Aan een weak dependency kun je alleen niet zien of de dependency een functieparameter betreft of de functiebody. Het dwingt ook niet de aanwezigheid van een reference variabele af.

Je kunt de reference member ook opslaan (hoewel InstelControl hem later niet meer nodig heeft):

```
class InstelControl : ButtonListener
{
```

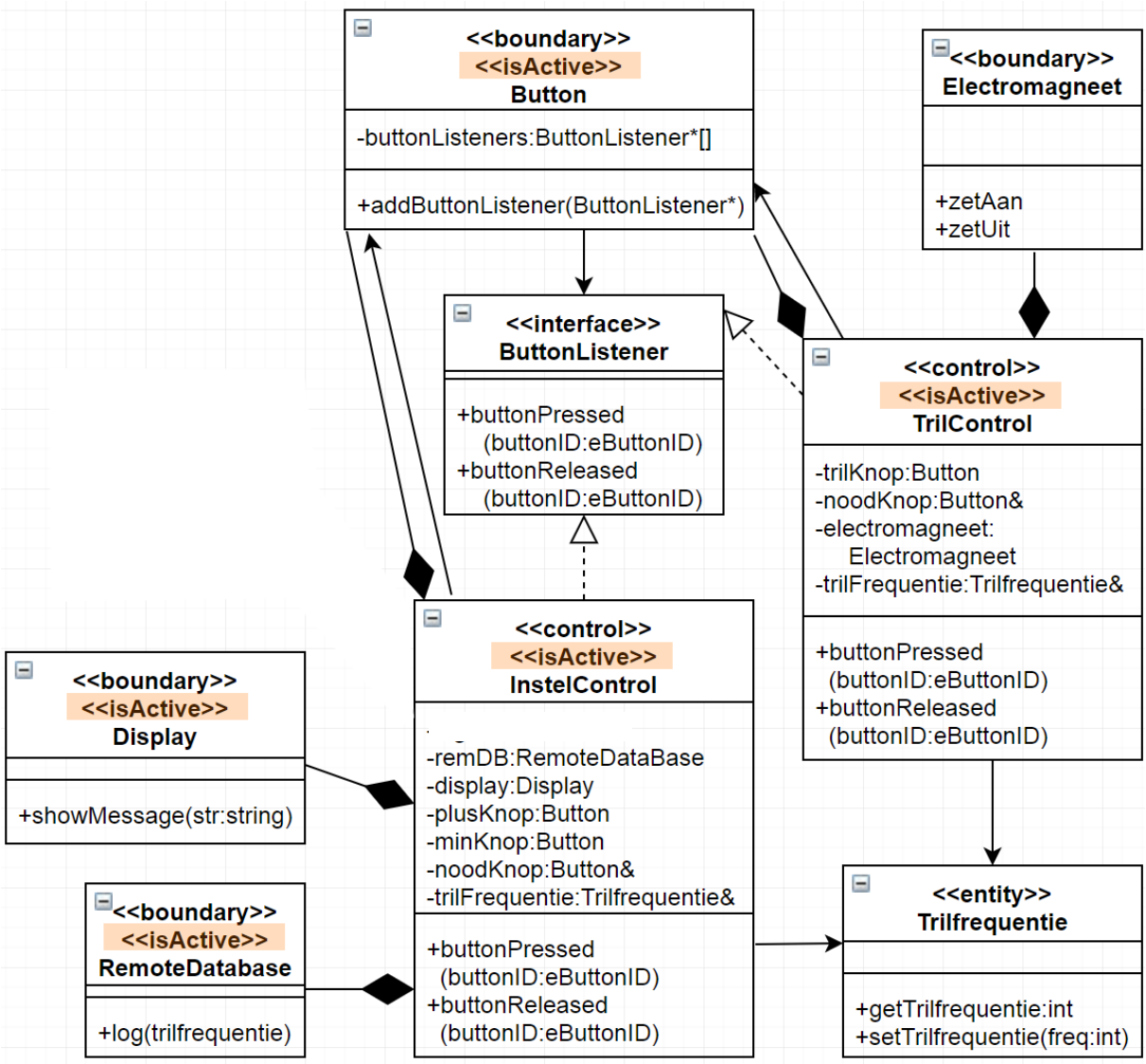
```
Button  plusKnop;
Button  minKnop;
Button& noodKnop;
InstelControl(Button& noodKnop):noodKnop(noodKnop)
{
    plusKnop.addButtonListener(this);
    minKnop.addButtonListener(this);
    noodKnop.addButtonListener(this);
}
…etc
```

In dat geval is het zowel in de member sectie in de code van de klasse als in de member sectie van het klasse diagram duidelijker dat InstelControl ook berichten verwacht van een permanent aanwezige knop met de naam noodKnop. Om die reden kies ik voor die oplossing. InstelControl en TrilControl krijgen dus ook een reference member noodKnop:Button&. InstelControl heeft dus sommige buttons, maar refereert alleen naar andere. Om dat te weer te geven voegen we naast de compositiepijl tussen TrilControl en Button ook nog een gewone associatiepijl toe:



Stap 8: Markeer isActive

Markeer klassen die een taak hebben met het (extra) stereotype <<isActive>>.

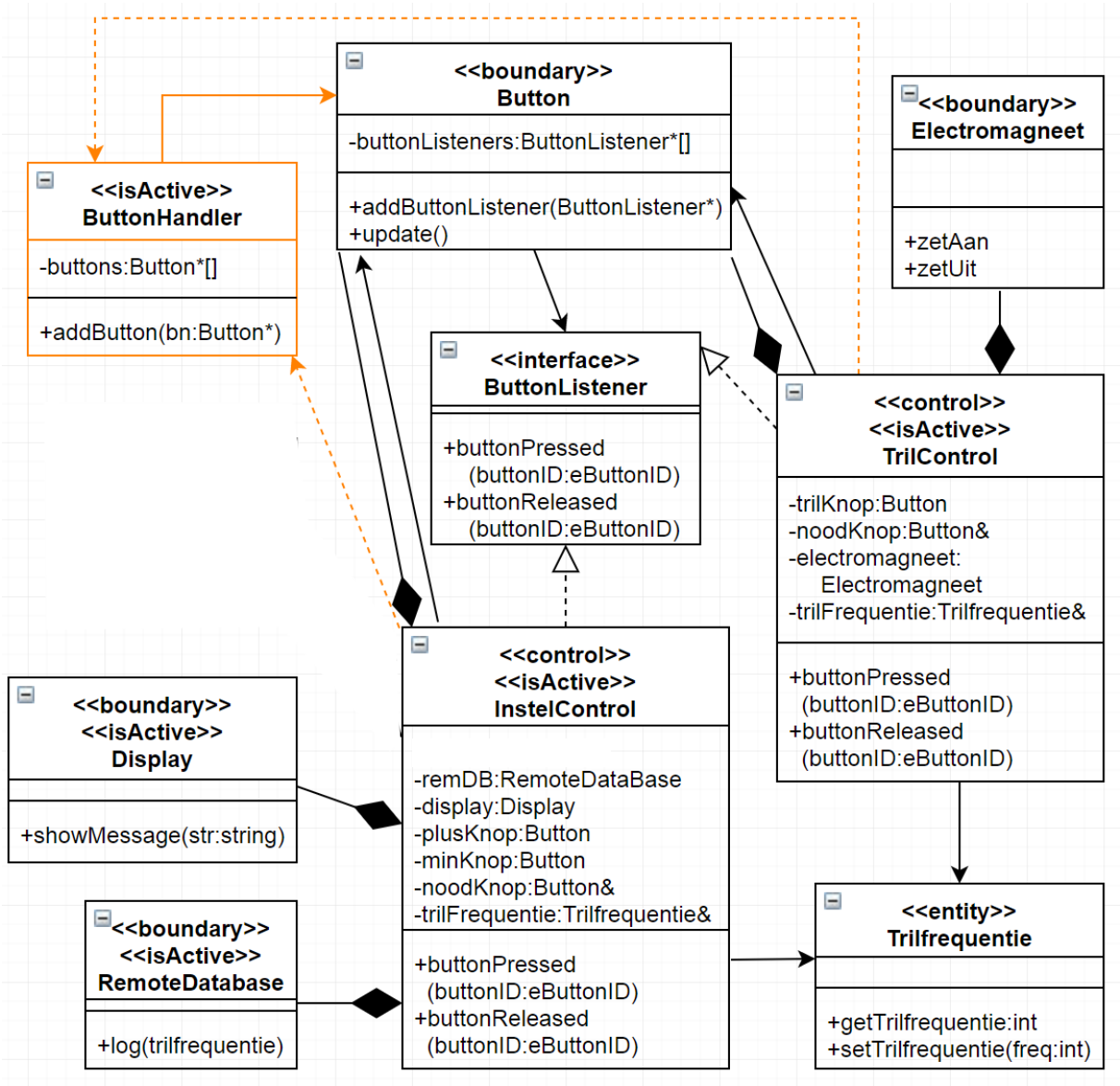


Stap 9: Voeg Handler klassen toe

Voeg de extra Handler klassen toe die zijn ontstaan als gevolg van de taaksamenvoeging van periodieke taken.

Bij de stap taakstructurering / taaksamenvoeging is voorzien dat het een goed idee is om een extra <<isActive>> ButtonHandler klasse aan te roepen die voor alle Button objecten een update() functie aanroept. De Button objecten hebben dan geen eigen taak meer nodig.

Zo gezegd, zo gedaan:



Merk op: omdat we nu een ButtonHandler gebruiken, konden we <<isActive>> bij de Button weghalen.

Stap 10: Weak dependencies

Weak dependencies

Het kan zijn dat een klasse in een van de functie-parameters of in zijn functiebody van een van zijn functie een klasse gebruikt waarvan hij verder geen member of reference member heeft. In dat geval laat je de afhankelijkheid zien mbv een “weak dependency” relatie (een gestippelde pijl).

InstelControl en TrilControl kennen de ButtonHandler. Het idee is dat in hun constructor er de knoppen aan toevoegen die ze in hun eigendom hebben. Ze hoeven ButtonHandler dus alleen te kennen binnen de functiebody van hun constructor. Het meegeven van een refrenence van de ButtonHandler aan de constructor van InstelControl en TrilControl volstaat dus. Het is daarom niet nodig dat InstelControl en TrilControl zelf een reference member hebben naar de buttonhandler.

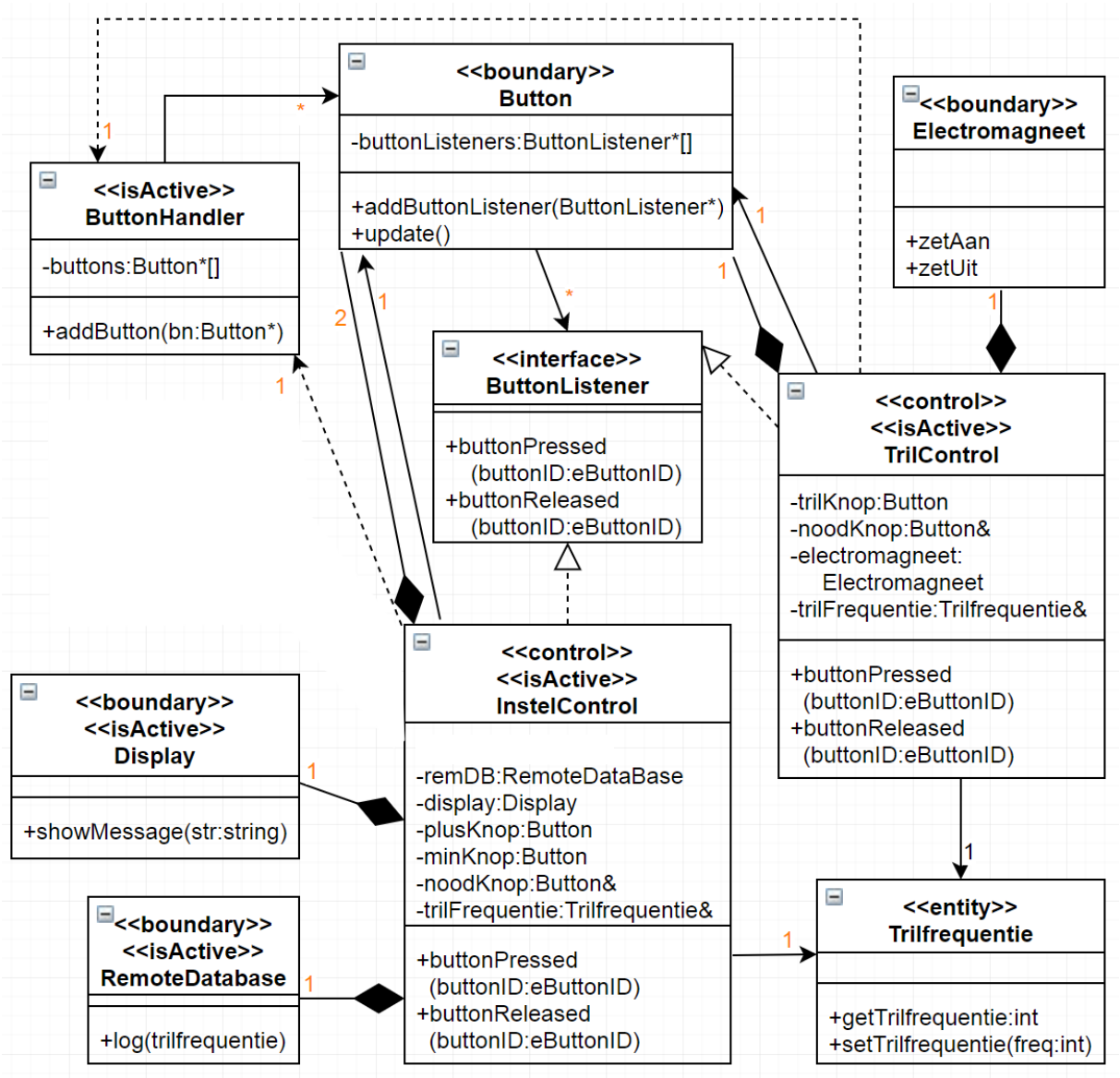
Voor dit ontwerp hebben we eerder gezien dat de afhankelijkheid van het noodknop object met een weak dependency gemodelleerd had kunnen worden. Ik heb er echter bewust voor gekozen (zie eind van het hoofdstuk over Listeners) om in plaats daarvan een member variabele noodKnop:Button& aan TrilControl en InstelControl toe te voegen. Daarmee werd het een “gewone” associatie.

Stap 10: Multiplicity

Zorg dat aan het uiteinde van de de associatiepijlen en de compositie-relaties de juiste multipliciteit in het klassediagram is aangegeven.
Tel eenvoudigweg hoeveel members en reference members een klasse heeft.

- InstelControl heeft 2 Buttons (plusKnop en minKnop). Aan het eind van zijn compositierelatie naar de klasse Button moet dus een 2 komen te staan.
- TrilControl heeft 1 Button (trilKnop). Aan het eind van zijn compositierelatie naar de klasse Button moet dus een 1 komen te staan.. of worden weggelaten (dat betekent ook 1). Ik noteer overal netjes een 1. Dan is voor mezelf duidelijk dat ik geen tak ben vergeten te beschouwen.
- InstelControl en TrilControl hebben beide een reference naar een Button (noodKnop). Naast de “gewone” associatiepijl naar Button vanuit die klassen noteer ik daarom een 1.
- De klasse ButtonHandler heeft een array (misschien wel een vector) van niet nader genoemde grootte. Je kunt er via zijn addButton functie een willekeurig aantal Buttons aan toevoegen. Naast zijn associatiepijl naar Button noteren we daarom een *.
  NB: dat er run-time in ons design 4 buttons aan worden toegekend, doet er niet toe.
- Iets soortgelijks geldt voor de Button klasse. Die heeft een array met ButtonListeners (misschien wel een vector) van niet nader genoemde grootte. Je kunt er via zijn addButtonListener functie een willekeurig aantal ButtonListeners aan toevoegen. Naast zijn associatiepijl naar ButtonListener noteren we daarom een *.

Met de multipliciteiten verwerkt krijgen we dus:



Synchronisatiemechanismen

Het is goed gebruik om taken alleen te laten communiceren en data te laten uitwisselen via synchronisatiemechanismen.

De voordelen van het gebruik van synchronisatiemechanismen zijn:

1. Taken kunnen dingen aan elkaar doorgeven en vervolgens zelf doorgaan met hun eigen taak (= asynchroniteit).
2. Data wordt veilig doorgegeven tussen de taken. Het RTOS voorkomt bijvoorbeeld door middel van het gebruik van mutexen waar nodig dat data nooit gelijktijdig door meerdere taken wordt bepoteld (anders zou het bijvoorbeeld kunnen voorkomen dat terwijl een struct nog wordt gelezen door de ene taak, dat de andere taak al een deel ervan aan het overschrijven is, waardoor de data niet meer consistent is.

De synchronisatiemechanismen die we binnen FreeRTOS gebruiken zijn:

- **Event group en eventbits**
  Een taak kan met xEventGroupWaitBits() wachten op een of meer eventbits uit zijn eigen

event group (een EventGroupHandle_t, hier steeds eventGroup genoemd). Elk eventbit, bijvoorbeeld EventBits_t eventBitAan = (1 << 0), staat voor een ding waar de taak op kan wachten. Een event group hoort bij een enkele taak. Alleen die ene taak wacht erop; andere taken zetten er bits in. Een bit dat gezet is, blijft staan totdat het gewist wordt: met xClearOnExit = pdTRUE wist xEventGroupWaitBits() de bits waarop gewacht werd zelf, met pdFALSE wis je een bit zelf (met xEventGroupClearBits()) zodra je het hebt afgehandeld.
Dit zijn de FreeRTOS-mechanismen die tot je beschikking staan:

- **Eventbit (als flag)**
  De ene taak zet een eventbit: xEventGroupSetBits(eventGroup, eventBitAan). De andere taak kan er op wachten. Het "afgaan" van een eventbit betekent "let op". Niet meer, niet minder. Een eventbit is dus niet hetzelfde als een bool. Als iemand tegen je zegt "let op", weet je niet of hij "true" of "false" bedoelt.
- **Queue**
  Een queue (QueueHandle_t) is een FIFO (first in, first out) buffer. Een andere taak schrijft er iets in:
  xQueueSend(tempQueue, &temperature, 0);
  De taak die de queue bezit, leest eruit. Hoeft hij verder nergens op te wachten, dan kan dat blokkerend: hij wacht dan tot er iets in de queue staat:
  int newTemperature;
  xQueueReceive(tempQueue, &newTemperature, portMAX_DELAY);
  Moet de taak tegelijk ook op iets anders wachten (een eventbit of een timer), dan geef je de queue een eigen eventbit, bijvoorbeeld eventBitTempQueue. De schrijvende taak zet dat bit direct na het schrijven, en de lezende taak wacht op dat bit en leest daarna niet-blokkerend, met de hulpfunctie tryReadQueue() (zie het STD van InstelControl).
- **Timer**
  Je kunt een software timer (TimerHandle_t) gebruiken als een soort kookwekker. Je maakt hem een keer aan met xTimerCreate(), met een callback-functie die wordt aangeroepen als de timer afloopt. Die callback zet een eventbit. Na xTimerStart() (of xTimerChangePeriod(), als de wachttijd per keer verschilt) ga je normaal gesproken gelijktijdig ook op andere eventbits wachten. Als je alleen op de timer zelf zou wachten, had je in het algemeen ook uitgekund met een eenvoudige vTaskDelay() aanroep.

- **Queue van lengte 1 (pool)**
  Een variabele die meerdere taken veilig moeten kunnen lezen en schrijven, berg je op in een queue van lengte 1.
  Neem bijvoorbeeld:
  QueueHandle_t temperatuurQueue = xQueueCreate(1, sizeof(int));
  Schrijven doe je met xQueueOverwrite(temperatuurQueue, &temperatuur), lezen met xQueuePeek(temperatuurQueue, &temperatuur, 0). Overwrite vervangt de oude waarde, en Peek laat de waarde in de queue staan.
  Op zo'n queue wacht je niet. Elke taak kan er naar believen uit lezen of naartoe schrijven.
- **Mutexen**
  Als je voor FreeRTOS designt, heb je veelal genoeg aan de bovengenoemde synchronisatiemechanismen. In een uitzonderlijk geval wil je misschien dat een reeks

activiteiten altijd gegroepeerd moet worden uitgevoerd, dus dat het klaar moet zijn voordat een andere taak dezelfde reeks start. In dat geval kun je die reeks activiteiten beschermen met een mutex (xSemaphoreCreateMutex()). Daar ga ik in dit document niet verder op in.

STDs ontwerpen

Objecten die code uitvoeren binnen een taak volgen meestal een toestandsdiagram (een STD).
In het algemeen zijn dat de <<control>> objecten. Maar ook veel boundary objecten die periodieke taken verrichten doorlopen ook een STD.

Hoe kom je vanuit een gegeven klassediagram tot de STDs?

Dat gaat in de volgende stappen:

- Maak de interface sectie van het STD
- Schrijf voor jezelf in woorden de reeks van acties en beslissingen op die horen bij de uitvoering van de taak van het betreffende object.
- Zet je beschrijving om in een STD:
    - Als je ergens op **wacht**, dan maak je daar een event van langs de uitgaande pijl van een toestand. Bedenk ook een passende naam voor de toestand. Op de timer events na zijn events altijd afkomstig uit de STD interface (zie verderop).
    - Bijvoorbeeld als je wacht tot de “aan-knop” is ingedrukt, maak dan een toestand met de naam “uit”, en verlaat die via een uitgaande pijl met de event “eventBitAanButton”.
    - Bedenk je of er actions zijn die altijd moeten gebeuren bij het binnengaan van de toestand (entry events), bij het verlaten van de toestand (exit events) of tijdens een event langs een uitgaande pijl.
      De acties kunnen voor een deel bestaan uit **aanroepen van publieke memberfuncties** van andere objecten. In het klassediagram kun je zien welke andere objecten je klasse kent en heeft, en dus welke publieke functies er aangeroepen kunnen worden.
    - Verwerk je **logische beslissingen** met behulp van guards en eventueel decision nodes.
    - **Voorkom duplicate actions en guards** eventueel met het gebruik van merge nodes.
    - Gebruik eventueel een **samengestelde toestand** (composite state) als het gedrag uit meerdere lagen bestaat.

Timers

Net als bij de overige dingen waar je op kunt wachten (eventbits, queues) – zie hierboven – zouden er nu in het STD ook timers kunnen worden gebruikt. Timers zijn een vreemde eend in de bijt. Wanneer gebruik je daar nu wel of niet een aparte timer voor in je STD? De vuistregel is: Benoem in je STD **geen** aparte timer als:

- Je timer **altijd wordt gestart in het entry-event** van een enkele, specifieke toestand, en de toestand **altijd verlaten wordt bij het vuren** van zijn timer-event

In zo’n geval kun je bijvoorbeeld **after**(1000ms) naast de uitgaande pijl zetten (dat impliceert dat de betreffende timer is gestart tijdens het entry-event). Wacht de toestand daarnaast nog op iets anders, dan wordt zo’n after() in de FreeRTOS-code toch een eigen software timer met een eventbit. In het STD hoef je die timer dan niet te noemen.

In de overige gevallen geef je de timer een naam, bijvoorbeeld myTimer, met eventbit eventBitMyTimer. Je maakt hem aan bij de pijl uit de initial node (myTimer = xTimerCreate(...)), en

start hem expliciet, met bijvoorbeeld xEventGroupClearBits(eventGroup, eventBitMyTimer); xTimerStart(myTimer, portMAX_DELAY). De timer-event kan eventueel afgehandeld worden door een event-handler in de body-sectie (het deel tussen entry en exit event) van de betreffende toestand. Voorbeeld: eventBitMyTimer / doAction().

Ontwerp van de STD interface

- Het ontwerpen van STDs begint met het ontwerp van zijn interface sectie.
  Het STD communiceert met andere taken via de interface functies in zijn interface sectie.
- Die interface functies worden uitgevoerd in de thread van de aanroepende taak.

Je kunt de STD interface ontwerpen met de volgende vuistregels:

1. Voeg elke publieke functie van de klasse waarvoor je een STD maakt, en die wordt aangeroepen door een andere taak, toe aan de STD interface.
2. Je kunt de publieke functies in twee soorten verdelen:
   - **Functies die "even snel" een waarde opvragen**:
     Bijvoorbeeld getTemperature() en isTemperatureTooHigh().
     Die functies implementeer je door de waarde uit een queue van lengte 1 te retourneren.
     Voorbeeld van zo'n functie:
     float getTemperature() {float t; xQueuePeek(temperatureQueue, &t, 0); return t;}
     De betreffende queue van lengte 1 maak je een member van de klasse van het STD.
     De notatie van zo'n functie in de interface sectie is zoiets:
     getTemperature / float t; xQueuePeek(temperatureQueue, &t, 0); return t
     De **types** van parameters en return values noteer je daar dus **niet**.
   - **Overige functies** – in het algemeen functies die een bericht doorgeven of expliciet opdracht geven om iets te gaan doen. Die functies kun je ook weer in twee categorien verdelen:
     - **Functies zonder argument**
       Functies zonder argument, zoals bijvoorbeeld zetAan() implementeer je door een eventbit te zetten:
       zetAan / xEventGroupSetBits(eventGroup, eventBitAan)
     - **Functies met argument**
       - **Als alleen de laatst binnengekomen waarde ertoe doet**
         Als alleen de laatst binnengekomen waarde ertoe doet, implementeer je zo'n functie met een **queue van lengte 1** en een **eventbit**. De waarde wordt met xQueueOverwrite in de queue geschreven, en het eventbit wordt gezet om te signaleren dat er een nieuwe, nog niet verwerkte waarde is. Voorbeeld:
         buttonPressed(buttonID) / xQueueOverwrite(buttonIdQueue, &buttonID);
         xEventGroupSetBits(eventGroup, eventBitButton)
       - **Als het belangrijk is dat je geen voorgaande waarde misloopt**
         Als het belangrijk is dat je geen voorgaande waarde misloopt, kun je in plaats van een combinatie van eventbit en queue van lengte 1 zoals hierboven beter kiezen voor een gewone **queue**:

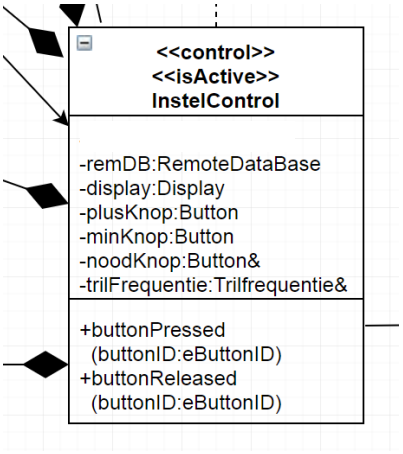
buttonPressed(buttonID) / xQueueSend(buttonQueue, &buttonID, 0)
Wacht de taak in een toestand ook nog op iets anders, dan krijgt de queue een eigen eventbit, dat direct na het schrijven gezet wordt:
buttonPressed(buttonID) / xQueueSend(buttonQueue, &buttonID, 0);
xEventGroupSetBits(eventGroup, eventBitButtonQueue)
De wachttijd 0 bij xQueueSend zorgt ervoor dat de aanroepende taak niet blijft hangen als de queue vol is.

Ontwerp van het STD van InstelControl

We beginnen met het ontwerp van de STD interface

Ontwerp van de STD interface

We beginnen met een blik op onze klasse in het klassediagram:



Er zijn welgeteld twee publieke functies. Dat worden dus de functies van de STD interface.
In beide gevallen wordt er een parameter meegegeven. Nu moet er een knoop worden doorgehakt over of we voor een eventbit met een queue van lengte 1 of voor een gewone queue kiezen om de nieuw binnengekomen waarde mee te bufferen.
Als de noodKnop wordt ingedrukt, willen we dat absoluut niet missen. Je zou dat kunnen borgen door aan de Button een IsButtonPressed functie toe te voegen die altijd gepollt kan worden. Je kunt het ook borgen door alle binnenkomende buttonPresses te bufferen in een queue.
Verder is er ook weer niet veel risico dat er uberhaupt iets wordt gemist: InstelControl heeft behalve het updaten van de frequentie na zo'n buttonpress toch niets te doen.
Werken met events heeft in het algemeen de voorkeur boven pollen, dus ik kies ervoor om ook de noodKnop via events af te handelen. Om dat veilig te kunnen doen, kies ik voor het gebruik van een queue.
InstelControl wacht straks in elke toestand behalve op de knoppen ook op een timer (zie verderop). Daarom krijgen beide queues een eigen eventbit: eventBitBnPressQueue en eventBitBnReleaseQueue. buttonPressed en buttonReleased zetten dat bit direct na het schrijven in de queue.

De STD interface van InstelControl komt er dus zo uit te zien:



NB: de horizontale lijn (in andere dictaten soms weggelaten) verduidelijkt dat deze STD werkt met meerdere threads tegelijk: het gedeelte boven de lijn wordt uitgevoerd in de aanroepende thread. Het gedeelte onder de lijn (de eigenlijke STD, hier nog niet ingevuld) wordt uitgevoerd door de thread van de taak waar de STD toe behoort.

Actiereeks van InstelControl in woorden

Wat InstelControl moet doen, staat beschreven in de **beschrijving van de use case** Instellen.

Er blijkt een uitzonderingssituatie te zijn:
Na indrukken van de noodknop moet “Emergency” op het scherm getoond worden. Na intrekken van de noodknop mag de normale use case weer hervat worden.

De normale use case wacht op het indrukken van een button. Dat wordt gedaan vanuit een toestand die ik maar “Normaal” noem.
Is het de plusKnop, dan moet de frequentie verhoogd worden, tenzij 100 is bereikt.
Is het de minKnop, dan moet de frequentie verlaagd worden, tenzijn 1 is bereikt.
Is het de noodKnop, dan moet naar de NoodToestand gegaan worden.

Een minuut nadat de laatste frequentie update plaatsvond, moet dat worden gelogd.

Hetzelfde verhaal, maar nu met toestanden, events en acties er aan toegevoegd.

Er blijkt een uitzonderingssituatie te zijn:
Na indrukken van de noodknop moet “Emergency” op het scherm getoond worden. Zoals de STD interface laat zien, kom je daar achter door te wachten op **eventBitBnPressQueue**, en daarna met tryReadQueue uit **bnPressQueue** te lezen. Het tonen van “Emergency” op het scherm kan doormiddel van de aanroep van **display.showMessage**.
De betreffende toestand waar we dan in gekomen zijn noem ik “**Noodtoestand**”. Na **intrekken** van de noodknop mag de use case weer **hervat** worden. Dus vanuit de Noodtoestand moet weer op **eventBitBnReleaseQueue** gewacht, en vervolgens uit **bnReleaseQueue** gelezen worden.

De **normale** (zonder uitzonderingen) use case **wacht** ook op het indrukken van een button. Dat wordt gedaan vanuit een **toestand** die ik maar “**Normaal**” noem.

Is het de plusKnop, dan moet de frequentie verhoogd worden, tenzij 100 is bereikt.
Is het de minKnop, dan moet de frequentie verlaagd worden, tenzijn 1 is bereikt.
Is het de noodKnop, dan moet naar de NoodToestand gegaan worden.

We zijn hier alleen geinteresseerd zijn in de inhoud van **bnPressQueue**. Toch moet ook bnReleaseQueue gelezen worden, zodat die niet vol met oude rommel staat wanneer we weer naar de toestand NoodToestand gaan.

In de noodtoestand zijn we dus alleen geinteresseerd in de inhoud van de bnReleaseQueue.
Toch moet daar ook de bnPressQueue gelezen worden, zodat die niet vol met oude rommel staat wanneer we weer naar de toestand Normaal gaan.

Een **minuut nadat** de laatste frequentie update plaatsvond, moet dat worden gelogd. Dus bij de **frequentie update** moeten we als actie een **timer start** uitvoeren: xEventGroupClearBits(eventGroup, eventBitLogTimer); xTimerStart(logTimer, portMAX_DELAY). Voor wat betreft het **loggen**: laten we zeggen dat het niet uitmaakt of het systeem zich ten tijde van dat loggen in de Normaal toestand of in de Noodtoestand bevindt. **Vanuit beide toestanden** moet dus worden **geluisterd** naar het eventbit van de **timer** (eventBitLogTimer), en in dat geval de actie **remoteDataBase.log**(trilFrequentie.getTrilFrequentie()) uitvoeren. In het STD gebruiken we **geen after()** voor deze timer, maar benoemen we hem **expliciet**, omdat hij **in meerdere toestanden** wordt gebruikt. We maken hem aan bij de pijl uit de initial node, met xTimerCreate(); zijn callback logTimerCallback zet eventBitLogTimer.

Op meer dingen tegelijk wachten: eventbits en tryReadQueue

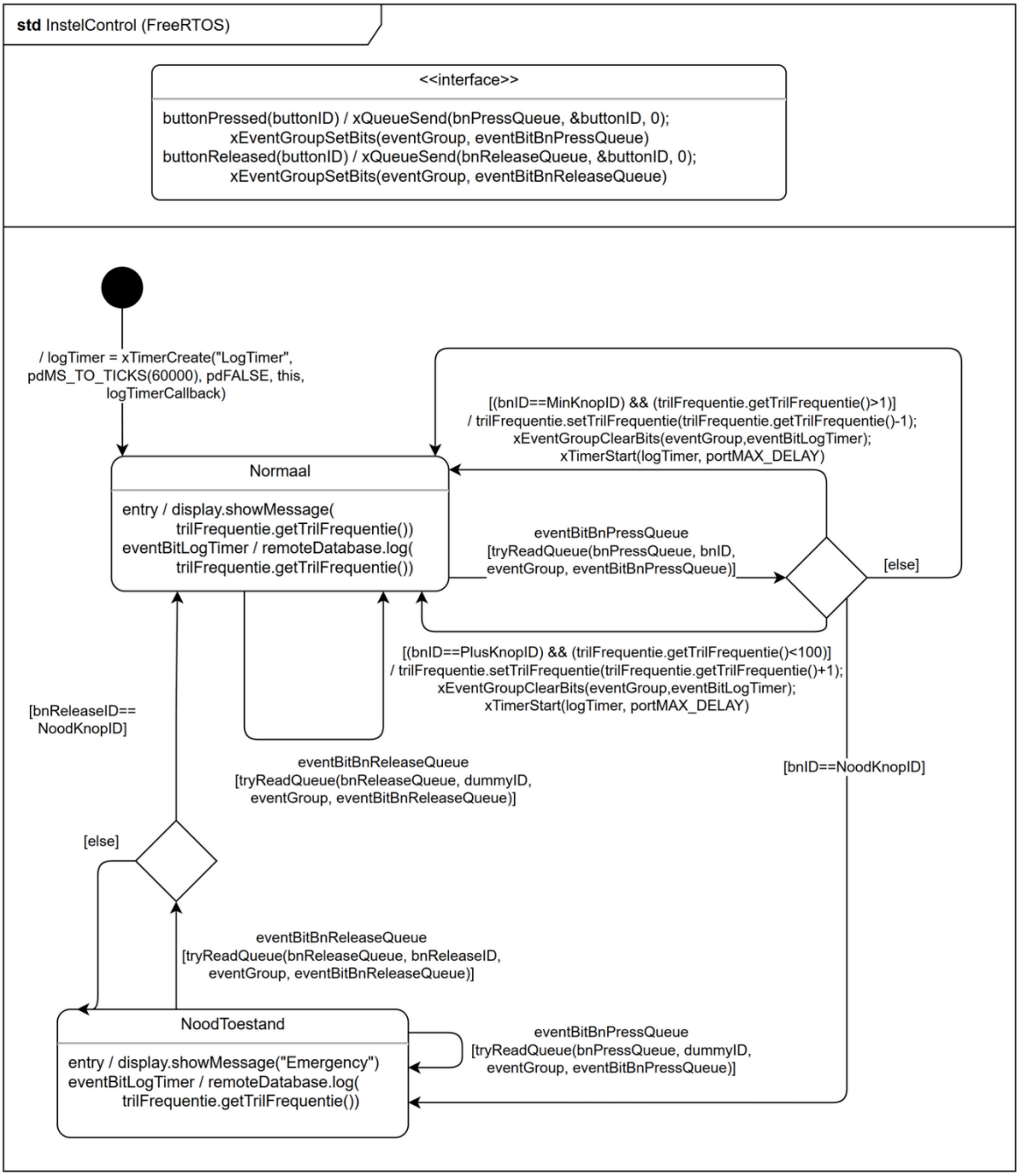
In beide toestanden wacht InstelControl op drie dingen tegelijk: eventBitBnPressQueue, eventBitBnReleaseQueue en eventBitLogTimer. In de code gaat dat met een enkele xEventGroupWaitBits() op die drie bits samen, met xClearOnExit = pdFALSE. Daarna kijk je welk bit gezet is, en handel je een event af. Waren er meer bits tegelijk gezet, dan blijven de andere staan voor de volgende wait. Een timer-bit wis je zelf, zodra je het afhandelt.

Het eventbit van een queue is een aanwijzing, geen belofte: het bit en de inhoud van de queue lopen niet vanzelf gelijk. Lees daarom na het wachten altijd niet-blokkerend, met de hulpfunctie tryReadQueue(). Die leest een bericht (als er een is), en trekt daarna het eventbit gelijk met wat er werkelijk nog in de queue staat. In het STD zie je dat terug als een guard: eventBitBnPressQueue [tryReadQueue(bnPressQueue, bnID, eventGroup, eventBitBnPressQueue)]

De manier van wachten, de code van tryReadQueue en de callback van logTimer staan als notes bij het STD, op de pagina na het STD.

Het STD van InstelControl

Na verwerking van het bovenstaande ontstaat dus het volgende STD van InstelControl


std InstelControl (FreeRTOS)
<<interface>>
buttonPressed(buttonID) / xQueueSend(bnPressQueue, &buttonID, 0);
xEventGroupSetBits(eventGroup, eventBitBnPressQueue)
buttonReleased(buttonID) / xQueueSend(bnReleaseQueue, &buttonID, 0);
xEventGroupSetBits(eventGroup, eventBitBnReleaseQueue)
/ logTimer = xTimerCreate("LogTimer", pdMS_TO_TICKS(60000), pdFALSE, this, logTimerCallback)
Normaal
entry / display.showMessage(trilFrequentie.getTrilFrequentie())
eventBitLogTimer / remoteDatabase.log(trilFrequentie.getTrilFrequentie())
[(bnID==MinKnopID) && (trilFrequentie.getTrilFrequentie()>1)] / trilFrequentie.setTrilFrequentie(trilFrequentie.getTrilFrequentie()-1); xEventGroupClearBits(eventGroup,eventBitLogTimer); xTimerStart(logTimer, portMAX_DELAY)
eventBitBnPressQueue [tryReadQueue(bnPressQueue, bnID, eventGroup, eventBitBnPressQueue)]
[else]
[(bnID==PlusKnopID) && (trilFrequentie.getTrilFrequentie()<100)] / trilFrequentie.setTrilFrequentie(trilFrequentie.getTrilFrequentie()+1); xEventGroupClearBits(eventGroup,eventBitLogTimer); xTimerStart(logTimer, portMAX_DELAY)
[bnReleaseID== NoodKnopID]
eventBitBnReleaseQueue [tryReadQueue(bnReleaseQueue, dummyID, eventGroup, eventBitBnReleaseQueue)]
[bnID==NoodKnopID]
[else]
eventBitBnReleaseQueue [tryReadQueue(bnReleaseQueue, bnReleaseID, eventGroup, eventBitBnReleaseQueue)]
NoodToestand
entry / display.showMessage("Emergency")
eventBitLogTimer / remoteDatabase.log(trilFrequentie.getTrilFrequentie())
eventBitBnPressQueue [tryReadQueue(bnPressQueue, dummyID, eventGroup, eventBitBnPressQueue)]

Bij het STD van InstelControl horen de volgende notes:

```
Het wachten in toestand Normaal gaat zo:

uxBits = xEventGroupWaitBits(eventGroup,
            eventBitBnPressQueue | eventBitBnReleaseQueue |
            eventBitLogTimer,
            pdFALSE, pdFALSE, portMAX_DELAY);
if ((uxBits & eventBitLogTimer) != 0)
{
    xEventGroupClearBits(eventGroup, eventBitLogTimer);
    // interne transitie: geen nieuwe entry
    remoteDatabase.log(...);
}
else if ((uxBits & eventBitBnPressQueue) != 0)
{
    if (tryReadQueue(bnPressQueue, bnID,
                     eventGroup, eventBitBnPressQueue))
    {
        ... guards [bnID==...] ...
    }
}
else if ((uxBits & eventBitBnReleaseQueue) != 0)
{
    ... tryReadQueue(bnReleaseQueue, dummyID, ...) ...
}

NB: xClearOnExit = pdFALSE. Zijn er meer bits
tegelijk gezet, dan wordt er één afgehandeld en
blijven de andere staan voor de volgende wait.
Een timer-bit wis je dus zelf; het bit van een
queue trekt tryReadQueue gelijk met de queue.
```

logTimer wordt niet als after() genoteerd: hij wordt gestart bij een frequentie-update en in beide toestanden afgehandeld. Daarom is het een expliciet genoemde FreeRTOS software timer, waarvan de callback eventBitLogTimer zet.

```
// Leest niet-blokkerend één bericht uit queue en trekt daarna het
// event-bit gelijk met de werkelijke queue-inhoud. Geeft true als er een
// bericht gelezen is.
template<typename T>
bool tryReadQueue(QueueHandle_t& queue, T& item,
                  EventGroupHandle_t& eventGroup,
                  const EventBits_t& eventBitQueue)
{
    // Niet-blokkerend lezen (timeout 0 ticks).
    bool bSucces = (xQueueReceive(queue, &item, 0) == pdPASS);

    // Event-bit gelijktrekken met de werkelijke queue-inhoud.
    if (uxQueueMessagesWaiting(queue) > 0)
    {
        xEventGroupSetBits(eventGroup, eventBitQueue); // er staat nog iets: bit aan
    }
    else
    {
        xEventGroupClearBits(eventGroup, eventBitQueue); // leeg: bit uit
        if (uxQueueMessagesWaiting(queue) > 0) // schrijver tussendoor gekomen?
        {
            xEventGroupSetBits(eventGroup, eventBitQueue);
        }
    }
    return bSucces;
}
```

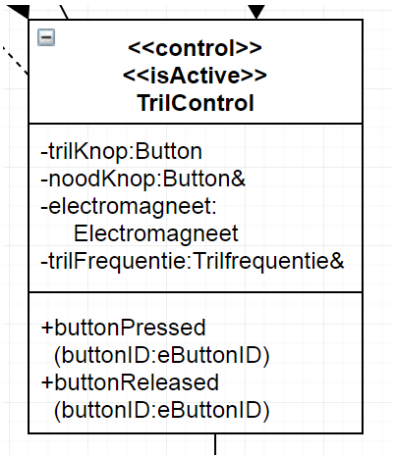
```
void logTimerCallback(TimerHandle_t xTimer)
{
  InstelControl *instance = static_cast<InstelControl *>(pvTimerGetTimerID(xTimer));
  xEventGroupSetBits(instance->eventGroup, instance->eventBitLogTimer);
}
```

Ontwerp van het STD van TrilControl

We beginnen met het ontwerp van de STD interface

Ontwerp van de STD interface

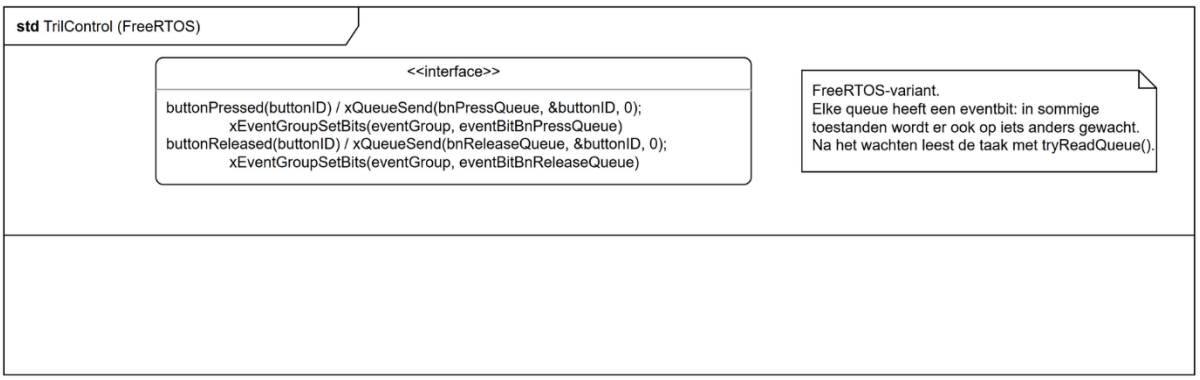
We beginnen met een blik op onze klasse in het klassediagram:



Er zijn welgeteld twee publieke functies: buttonPressed en buttonReleased. Dat worden dus de functies van de STD interface. In beide gevallen wordt er een parameter meegegeven. Nu moet er een knoop worden doorgehakt over of we voor een eventbit met een queue van lengte 1 of voor een gewone queue kiezen om de nieuw binnengekomen waarde mee te bufferen. De overweging is hier hetzelfde als bij InstelControl:
Als de noodKnop wordt ingedrukt, willen we dat absoluut niet missen doordat bijvoorbeeld de trilknop net erna is ingedrukt. Je zou dat kunnen borgen door aan de Button een IsButtonPressed functie toe te voegen die altijd gepollt kan worden. Je kunt het ook borgen door alle binnenkomende buttonPresses te bufferen in een queue.

Werken met events heeft in het algemeen de voorkeur boven pollen, dus ik kies ervoor om ook de noodKnop via events af te handelen. Om dat veilig te kunnen doen, kies ik voor het gebruik van een queue.
Ook TrilControl wacht op meer dingen tegelijk: in de toestand Trillen op de knoppen en op het verstrijken van een halve trilperiode. Daarom krijgen ook hier beide queues een eigen eventbit.

De STD interface van TrilControl komt er dus hetzelfde uit te zien als die van InstelControl:



Actiereeks van TrilControl in woorden

Wat TrilControl moet doen, staat beschreven in de beschrijving van de use case Trillen.

Het trillen bestaat uit het wachten tot een halve trilperiode voorbij is. De duur van een trilperiode is 1 gedeeld door de trilfrequentie. Voorbeeld: als de trilfrequentie 100Hz (Hertz) is, dat wil zeggen: 100 trillingen per seconde, dan duurt een complete trilling 1/100 seconde. Binnen die tijd is de electromagneet aan en weer uit gezet. Tussen het aan en uit zetten zit dan dus de helft van 1/100 seconde.

Met het indrukken van de trilknop kan dat trillen getoggeld worden.

Er blijkt een uitzonderingssituatie te zijn:
Na indrukken van de noodknop moet het trillen stoppen, en wordt er niet meer naar de trilknop geluisterd. Na intrekken van de noodknop mag de normale use case weer hervat worden.

Hetzelfde verhaal, maar nu met toestanden, events en acties er aan toegevoegd.

Wat TrilControl moet doen, staat beschreven in de beschrijving van de use case Trillen.

Het trillen bestaat uit het wachten tot een halve trilperiode voorbij is. De duur van een trilperiode is 1 gedeeld door de trilfrequentie. Voorbeeld: als de trilfrequentie 100Hz (Hertz) is, dat wil zeggen: 100 trillingen per seconde, dan duurt een complete trilling 1/100 seconde. Binnen die tijd is de electromagneet aan en weer uit gezet. Tussen het aan en uit zetten zit dan dus de helft van 1/100 seconde.

Met het indrukken van de trilknop kan dat trillen getoggeld worden.

Er blijkt een uitzonderingssituatie te zijn:
Na indrukken van de noodknop moet het trillen stoppen, en wordt er niet meer naar de trilknop geluisterd. Na intrekken van de noodknop mag de normale use case weer hervat worden.

De normale use case wacht dus op het indrukken van een button. Dat wordt gedaan vanuit een toestand die ik maar “Normaal” noem.
Is het de noodKnop, dan moet naar de NoodToestand gegaan worden.
Is het de trilKnop, dan moet het trillen getoggeld worden. De toestand Normaal heeft dus twee subtoestanden, die ik maar Trillen en NietTrillen noem.

De toestand Trillen zou je ook weer twee subtoestanden kunnen geven: ElectromagneetAan en ElectromagneetUit. In totaal kom je dan om 3 toestand-nestings-levels. Dat is misschien weer wat overkill in onze effort om het gedrag duidelijk te modelleren.
In plaats daarvan kies ik ervoor om de toestand trillen geen subtoestanden te geven. Na het verstrijken van de periodetijd wordt als actie een boolean bMagneetEnabled getoggeld, en wordt overeenkomstig electromagneet.zetAan() of electromagneet.zetUit() aangeroepen.
De extra laag van toestanden is dan dus gerealiseerd door middel van een boolean.

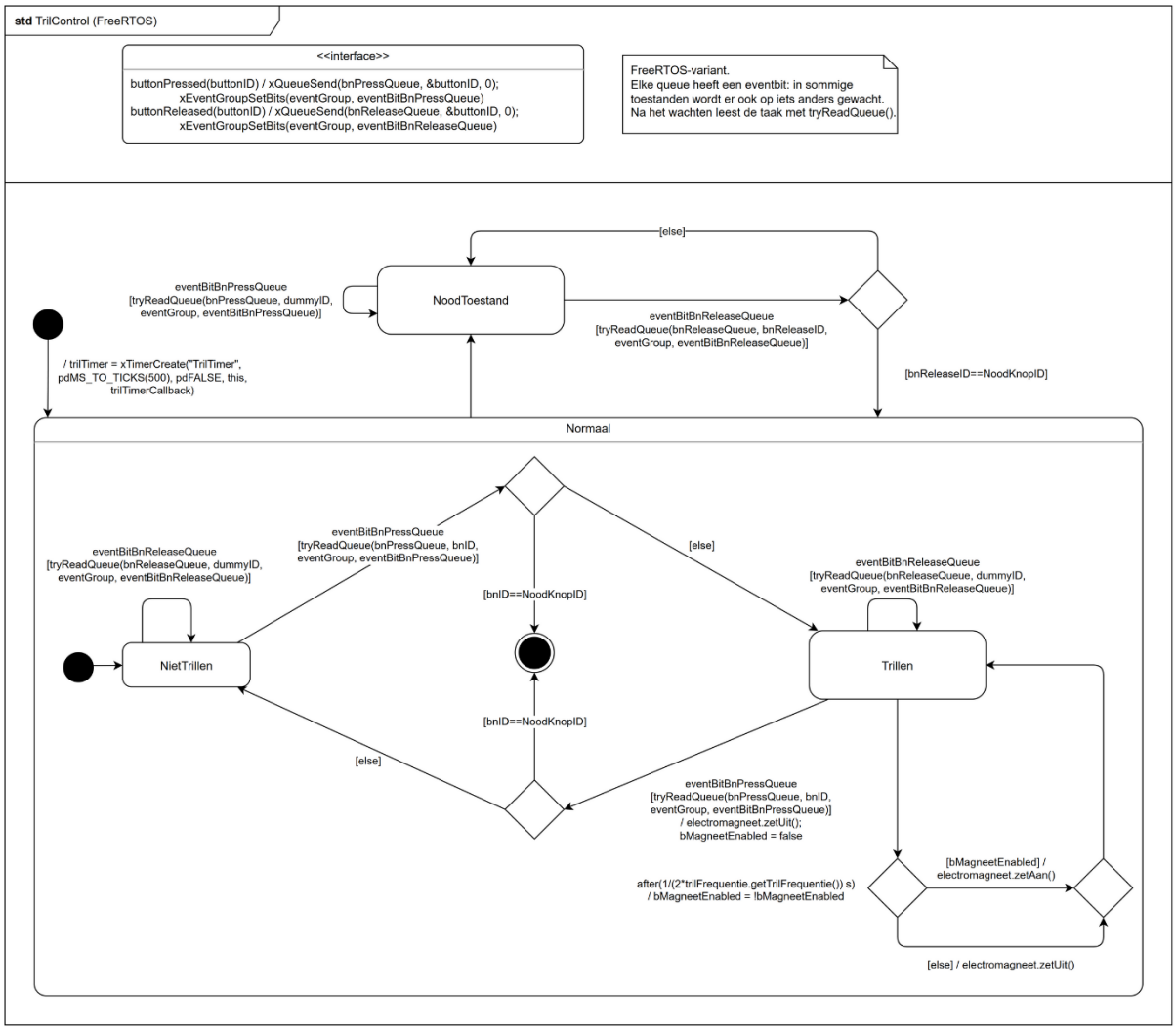
In de noodtoestand zijn we alleen geinteresseerd in de inhoud van de bnReleaseQueue.
Toch moet daar ook de bnPressQueue gelezen worden, zodat die niet vol met oude rommel staat wanneer we weer naar de toestand Normaal gaan.

Omgekeerd geldt dat wel in toestand Normaal alleen geinteresseerd zijn in de inhoud van bnPressQueue. Toch moet daar ook de bnReleaseQueue gelezen worden, zodat die niet vol met oude rommel staat wanneer we weer naar de toestand NoodToestand gaan.

Wordt er vanuit Trillen een knop ingedrukt (de trilknop of de noodknop), dan stopt het trillen. De electromagneet moet dan uit, anders kan hij aan blijven staan. Daarom wordt langs die pijl electromagneet.zetUit() aangeroepen en wordt bMagneetEnabled op false gezet. Dat gebeurt bewust niet in een exit event van Trillen: ook de after()-lus verlaat Trillen, en daar moet de magneet juist blijven togglen.

Het STD van TrilControl

Na verwerking van het bovenstaande ontstaat dus het volgende STD van TrilControl



NB:

- Het “after”/timer event waarmee de toestand Trillen kan worden verlaten impliceert een entry event bij Trillen waarbij zijn wachttijd wordt gezet en de timer wordt gestart:
  xEventGroupClearBits(eventGroup, eventBitTrilTimer);
  xTimerChangePeriod(trilTimer, pdMS_TO_TICKS(500 / trilFrequentie.getTrilFrequentie()), portMAX_DELAY)

Omdat de wachttijd van de trilfrequentie afhangt, gebruik je xTimerChangePeriod() in plaats van xTimerStart(). xTimerChangePeriod() start de timer ook.

- De toestand Trillen wacht op drie events: dat timer event (eventBitTrilTimer) en de eventbits van de twee queues.
  Omdat er op meerdere dingen tegelijk gewacht wordt, moet er voor “after” een aparte software timer (trilTimer) met een eigen eventbit worden aangemaakt, waar dus binnen een enkele xEventGroupWaitBits() samen met de andere eventbits op gewacht kan worden. Die timer maken we aan bij de pijl uit de initial node.
- Wordt Trillen via een ander event verlaten, dan geldt de after() niet meer. Stop de timer dan, en gooi een eventueel al gezet timer-bit weg:
  xTimerStop(trilTimer, portMAX_DELAY); xEventGroupClearBits(eventGroup, eventBitTrilTimer);
- Bij 100 Hz duurt een halve trilperiode 5 ms. Dat vraagt een tick van 1 ms (configTICK_RATE_HZ = 1000). Bij een tick van 10 ms (de ESP-IDF-standaard van 100 Hz) wordt pdMS_TO_TICKS(5) nul, en die periode weigert xTimerChangePeriod().

Bij het STD van TrilControl horen de volgende notes (tryReadQueue is dezelfde als bij InstelControl):

```
after(...) is een verkorte notatie voor:

entry / xEventGroupClearBits(eventGroup, eventBitTrilTimer);
        xTimerChangePeriod(trilTimer,
            pdMS_TO_TICKS(500 / trilFrequentie.getTrilFrequentie()),
            portMAX_DELAY)

met langs de uitgaande pijl het event eventBitTrilTimer.
xTimerChangePeriod, en niet xTimerStart: de periode hangt
af van de trilfrequentie. Hij start de timer ook.

Trillen wacht op drie dingen tegelijk:
xEventGroupWaitBits(eventGroup,
    eventBitBnPressQueue | eventBitBnReleaseQueue |
    eventBitTrilTimer, pdFALSE, pdFALSE, portMAX_DELAY);
Daarom is er een expliciete timer met eventbit nodig,
en volstaat een eenvoudige vTaskDelay niet.

Wordt Trillen via een ander event verlaten, dan geldt
de after() niet meer:
xTimerStop(trilTimer, portMAX_DELAY);
xEventGroupClearBits(eventGroup, eventBitTrilTimer);

electromagneet.zetUit() staat bewust niet als exit
event bij Trillen: de after()-lus verlaat Trillen ook,
en daar moet de magneet juist blijven togglen. Daarom
staat die actie bij de pijl naar de beslisnode onder
Trillen.
```

```
// Leest niet-blokkerend één bericht uit queue en trekt daarna het
// event-bit gelijk met de werkelijke queue-inhoud. Geeft true als er een
// bericht gelezen is.
template<typename T>
bool tryReadQueue(QueueHandle_t& queue, T& item,
                  EventGroupHandle_t& eventGroup,
                  const EventBits_t& eventBitQueue)
{
    // Niet-blokkerend lezen (timeout 0 ticks).
    bool bSucces = (xQueueReceive(queue, &item, 0) == pdPASS);

    // Event-bit gelijktrekken met de werkelijke queue-inhoud.
    if (uxQueueMessagesWaiting(queue) > 0)
    {
        xEventGroupSetBits(eventGroup, eventBitQueue); // er staat nog iets: bit aan
    }
    else
    {
        xEventGroupClearBits(eventGroup, eventBitQueue); // leeg: bit uit
        if (uxQueueMessagesWaiting(queue) > 0) // schrijver tussendoor gekomen?
        {
            xEventGroupSetBits(eventGroup, eventBitQueue);
        }
    }
    return bSucces;
}
```

```
void trilTimerCallback(TimerHandle_t xTimer)
{
  TrilControl *instance = static_cast<TrilControl *>(pvTimerGetTimerID(xTimer));
  xEventGroupSetBits(instance->eventGroup, instance->eventBitTrilTimer);
}
```

Van STD naar Code

Het omzetten van een STD naar Code gaat vrij rechttoe rechtaan. In FreeRTOS is een taak een functie die door de scheduler wordt gestart. Dit zijn de stappen voor het omzetten van STD naar code:

1. Creeer een **klasse** voor de taak, met een **TaskHandle_t** en een **statische functie taskWrapper**, die de **main()** van het object aanroept.
2. Creeer een **enum class** waarmee de **toestanden** geduid kunnen worden (en zo nodig een tweede voor de substates).
3. Geef **alles waarop de taak wacht** een **eventbit** in een event group:
   static constexpr EventBits_t eventBitXxx = (1 << n);
4. Voeg de **member variabelen en references** toe zoals aangegeven in het klassediagram, plus de **handles** van de synchronisatiemechanismen (EventGroupHandle_t, QueueHandle_t, TimerHandle_t).
   (normaliter zijn die private).
5. Voeg een **constructor** toe die:
   a. De **reference members** initialiseert.
   b. De **synchronisatiemechanismen aanmaakt**: xEventGroupCreate, xQueueCreate en xTimerCreate (met **this** als timer-ID).
   c. Als de klasse een **listener** is, het **eigen object** als listener toevoegt aan de objecten waar het naar luistert.
   d. De members die een **Handler** nodig hebben toevoegt aan de betreffende Handler.
   e. Als **laatste** de taak start met **xTaskCreatePinnedToCore**, met **this** als parameter. Dan bestaat alles wat main() gebruikt al.
6. Maak voor **elke STD interface** een gelijknamige **publieke functie** aan. De code kun je 1 op 1 overnemen, bijvoorbeeld xQueueSend gevolgd door xEventGroupSetBits.
7. Maak voor elke timer een **statische callback**, die het object terugvindt met pvTimerGetTimerID en het **eventbit van de timer** zet.
8. Implementeer **de main() functie**:
   a. Creeer een **switch statement** dat springt naar de code voor de huidige toestand.
   b. Voer daar als eerste de **entry event code** uit.
   c. Vervolgens eventuele **“do” code**.
   d. **Wacht op** het optreden van **events**:
      uxBits = xEventGroupWaitBits(eventGroup, eventBitA | eventBitB, pdFALSE, pdFALSE, portMAX_DELAY);
   e. Interpreteer de event, en laat de bijbehorende **guard, actie en/of transitie** uitvoeren:
      if ((uxBits & eventBitA) != 0) {if (tryReadQueue(...)) {state = State::IDLE;}}

Timers in de code

Net als bij de overige synchronisatiemechanismen (eventbits, queues) zouden er nu **in de code** timers kunnen worden aangemaakt. Timers zijn een vreemde eend in de bijt. Wanneer maak je daar nu wel of niet een timer voor aan in je code? De vuistregel is: als je **alleen** op de timer gaat wachten en ondertussen **niets** anders doet, hoef je geen aparte timer aan te maken. Je gebruikt dan in de code bijvoorbeeld een vTaskDelay(pdMS_TO_TICKS(1000)) call. De code gaat pas verder na het verstrijken van die tijd.

Je maakt wel expliciet een timer aan bij “time-out” situaties. Een software timer (xTimerCreate) is dan als een kookwekker die na enige tijd een callback aanroept. Laat die callback het eventbit van de timer zetten. In de code kan dan worden gewacht op **zowel** die timer als op andere events, in één xEventGroupWaitBits. Daardoor kan het object tot de timer afgaat door blijven gaan met andere dingen.

Van STD naar code van InstelControl

De eerste stappen (klasse, enums, eventbits, members, constructor en STD interface) zijn vrij rechttoe rechtaan en leiden tot de volgende code:

```
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/event_groups.h"
#include "freertos/queue.h"
#include "freertos/timers.h"

// Onderstaande enum wordt zowel door TrilControl en InstelControl
// Gebruikt. Hij moet dus in een aparte header file staan die door
// beide wordt geinclude. Voor het overzicht zet ik het toch even
// erbij, hier.
enum eButtonID {MinKnopID, PlusKnopID, TrilKnopID, NoodKnopID};

class InstelControl : public ButtonListener
{
    enum class State {NORMAAL, NOODTOESTAND};

    // Er wordt in een toestand op meer dingen tegelijk gewacht. Daarom
    // krijgt alles waarop gewacht wordt een eventbit, ook de queues.
    static constexpr EventBits_t eventBitBnPressQueue   = (1 << 0);
    static constexpr EventBits_t eventBitBnReleaseQueue = (1 << 1);
    static constexpr EventBits_t eventBitLogTimer       = (1 << 2);

private:
    State               state = State::NORMAAL;

    TaskHandle_t        taskHandle;
    EventGroupHandle_t  eventGroup;
    QueueHandle_t       bnPressQueue;
    QueueHandle_t       bnReleaseQueue;
    TimerHandle_t       logTimer;

    Trilfrequentie&     trilFrequentie;
    Button&             noodKnop;
    Button              plusKnop;
    Button              minKnop;
    Display             display;
    RemoteDatabase      remoteDatabase;

public:
    InstelControl(const char* taskName, UBaseType_t taskPriority,
                  uint32_t taskStackSizeBytes, BaseType_t taskCoreNumber,
                  Trilfrequentie& trilFrequentie, Button& noodKnop,
                  ButtonHandler& buttonHandler) :
        trilFrequentie(trilFrequentie), noodKnop(noodKnop)
    {
        eventGroup = xEventGroupCreate();
        configASSERT(eventGroup != NULL);
        bnPressQueue = xQueueCreate(10, sizeof(eButtonID));
        configASSERT(bnPressQueue != NULL);
        bnReleaseQueue = xQueueCreate(10, sizeof(eButtonID));
        configASSERT(bnReleaseQueue != NULL);

        // Actie van de pijl uit de initial node. One shot timer; zijn
        // callback zet eventBitLogTimer, zodat er tegelijk op andere
        // dingen gewacht kan worden.
        logTimer = xTimerCreate("LogTimer", pdMS_TO_TICKS(60000), pdFALSE,
                                this, logTimerCallback);
        configASSERT(logTimer != NULL);

        // Meld objecten die door een handler aangezwengeld worden
        // aan bij de betreffende handler.
        buttonHandler.addButton(&plusKnop);
```

```
        buttonHandler.addButton(&minKnop);

        buttonHandler.addButton(&noodKnop);
                        // Aanname hierbij: de handler negeert
                        // herhaaldelijk aanmelden van dezelfde knop.

        // Meld mezelf als ButtonListener aan bij de buttons waar ik
        // naar luister.
        plusKnop.addButtonListener(this);
        minKnop.addButtonListener(this);
        noodKnop.addButtonListener(this);

        // start de thread van deze taak
        xTaskCreatePinnedToCore(taskWrapper, taskName, taskStackSizeBytes,
                                this, taskPriority, &taskHandle,
                                taskCoreNumber);
    }

    // De STD interface. Deze functies worden uitgevoerd in de thread
    // van de aanroepende taak.
    void buttonPressed(eButtonID buttonID)
    {
        xQueueSend(bnPressQueue, &buttonID, 0);
        xEventGroupSetBits(eventGroup, eventBitBnPressQueue);
    }
    void buttonReleased(eButtonID buttonID)
    {
        xQueueSend(bnReleaseQueue, &buttonID, 0);
        xEventGroupSetBits(eventGroup, eventBitBnReleaseQueue);
    }
```

Zoals eerder besproken, wordt het paradigma gehanteerd dat de eigenaar van een Button die button ook aanmeldt bij de ButtonHandler.

Merk op dat de constructor de taak **als laatste** start. FreeRTOS kan geen member functie als taak starten. Daarom krijgt xTaskCreatePinnedToCore de **statische** functie taskWrapper mee, met **this** als parameter. taskWrapper zet die parameter weer om naar het object, en roept daarvan main() aan. Op dezelfde manier vindt de **statische** timer callback het object terug met pvTimerGetTimerID: bij xTimerCreate is **this** als timer-ID meegegeven. De callback doet niets anders dan het eventbit van de timer zetten. Hij draait in de timer-taak van FreeRTOS, en moet dus kort zijn.

```
private:
    static void taskWrapper(void* pParam)
    {
        static_cast<InstelControl*>(pParam)->main();
    }

    static void logTimerCallback(TimerHandle_t xTimer)
    {
        InstelControl* instance =
            static_cast<InstelControl*>(pvTimerGetTimerID(xTimer));
        xEventGroupSetBits(instance->eventGroup, instance->eventBitLogTimer);
    }
```

Een queue heeft hier dus een eigen eventbit, dat de schrijver zet. De lezer leest de queue met de hulpfunctie **tryReadQueue**: niet-blokkerend (timeout 0), waarna het eventbit wordt gelijkgetrokken met de werkelijke inhoud van de queue. Staat er nog een bericht in, dan blijft het bit staan, en wordt dat bericht bij de volgende wait opgepikt.

```
    // Leest niet-blokkerend een bericht uit queue en trekt daarna het
    // eventbit gelijk met de werkelijke queue-inhoud. Geeft true als er
    // een bericht gelezen is.
    template<typename T>
    static bool tryReadQueue(QueueHandle_t& queue, T& item,
                             EventGroupHandle_t& eventGroup,
                             const EventBits_t& eventBitQueue)
    {
        // Niet-blokkerend lezen (timeout 0 ticks).
        bool bSucces = (xQueueReceive(queue, &item, 0) == pdPASS);

        // Eventbit gelijktrekken met de werkelijke queue-inhoud.
        if (uxQueueMessagesWaiting(queue) > 0)
        {
            xEventGroupSetBits(eventGroup, eventBitQueue);   // nog iets: aan
        }
        else
        {
            xEventGroupClearBits(eventGroup, eventBitQueue); // leeg: uit
            if (uxQueueMessagesWaiting(queue) > 0)   // schrijver tussendoor?
            {
                xEventGroupSetBits(eventGroup, eventBitQueue);
            }
        }
        return bSucces;
    }
```

Zoals vrijwel altijd bij rtos taken implementeer je de taak door middel van een oneindige loop binnen de functie main(). Aan het begin van die functie zijn enkele variabelen gedeclareerd die dienen om tijdelijk de uit een queue gelezen waarde in op te slaan:

```
    void main()
    {
        eButtonID   bnID;
        eButtonID   bnReleaseID;
        eButtonID   dummyID;
        EventBits_t uxBits;
        bool        bTransitie;

        while (true)
        {
            switch (state)
            {
                // ... hier de cases van de toestanden (zie hieronder) ...
                default: break;
            }
        }
    }
};
```

Ook de vervolgstap, de uitwerking van de STD in code, is rechttoe rechtaan. Dit is de implementatie van de toestand Normaal:

```
case State::NORMAAL:
    // entry event
    display.showMessage(trilFrequentie.getTrilFrequentie());

    // Wacht tot er een transitie optreedt. De logTimer-event is een
    // interne transitie: geen nieuw entry event.
    bTransitie = false;
    while (!bTransitie)
    {
        // xClearOnExit = pdFALSE: is er meer dan een bit gezet, dan wordt
        // er een afgehandeld en blijven de andere staan voor de volgende wait.
        uxBits = xEventGroupWaitBits(eventGroup,
                    eventBitBnPressQueue | eventBitBnReleaseQueue
                                         | eventBitLogTimer,
                    pdFALSE, pdFALSE, portMAX_DELAY);
        if ((uxBits & eventBitLogTimer) != 0)
        {
            xEventGroupClearBits(eventGroup, eventBitLogTimer);
            remoteDatabase.log(trilFrequentie.getTrilFrequentie());
        }
        else if ((uxBits & eventBitBnPressQueue) != 0)
        {
            if (tryReadQueue(bnPressQueue, bnID,
                             eventGroup, eventBitBnPressQueue))
            {
                bTransitie = true;
                if (bnID == NoodKnopID)
                {
                    state = State::NOODTOESTAND;
                }
                else if ((bnID == MinKnopID) &&
                         (trilFrequentie.getTrilFrequentie() > 1))
                {
                    trilFrequentie.setTrilFrequentie(
                        trilFrequentie.getTrilFrequentie() - 1);
                    xEventGroupClearBits(eventGroup, eventBitLogTimer);
                    xTimerStart(logTimer, portMAX_DELAY);
                }
                else if ((bnID == PlusKnopID) &&
                         (trilFrequentie.getTrilFrequentie() < 100))
                {
                    trilFrequentie.setTrilFrequentie(
                        trilFrequentie.getTrilFrequentie() + 1);
                    xEventGroupClearBits(eventGroup, eventBitLogTimer);
                    xTimerStart(logTimer, portMAX_DELAY);
                }
                else
                {
                    state = State::NORMAAL;
                }
            }
        }
        else if ((uxBits & eventBitBnReleaseQueue) != 0)
        {
            // maak bnReleaseQueue leeg (zelf-transitie naar Normaal).
            if (tryReadQueue(bnReleaseQueue, dummyID,
                             eventGroup, eventBitBnReleaseQueue))
            {
                bTransitie = true;
            }
        }
    }
    break;
```

Een paar dingen vallen op:

- Het aflopen van de logTimer is een **interne transitie**: die verlaat de toestand niet. Daarom staat het wachten in een binnenste **while (!bTransitie)** lus. Na het loggen wordt er opnieuw gewacht, zonder dat het entry event (display.showMessage) opnieuw wordt uitgevoerd.
- Alle andere pijlen zijn wel transities, ook de **zelf-transities** ([else] en het leeghalen van bnReleaseQueue). Die zetten bTransitie op true: de toestand wordt verlaten en weer binnengegaan, dus het entry event wordt opnieuw uitgevoerd. Dat klopt met het STD.
- xEventGroupWaitBits wist met **pdFALSE** zelf geen bits. Het eventbit van een timer wis je dus zelf (xEventGroupClearBits); dat van een queue wordt gelijkgetrokken door tryReadQueue.
- Bij een frequentie update wordt de logTimer **herstart**. Wis eerst zijn eventbit: een eerdere afloop die nog niet verwerkt is, telt niet meer. xTimerStart op een timer die al loopt, start hem opnieuw.

Dit is nog de implementatie van de toestand Noodtoestand:

```
case State::NOODTOESTAND:
    // entry event
    display.showMessage("Emergency");
    // Wacht tot er een transitie optreedt (logTimer: interne transitie).
    bTransitie = false;
    while (!bTransitie)
    {
        uxBits = xEventGroupWaitBits(eventGroup,
                    eventBitBnPressQueue | eventBitBnReleaseQueue
                                         | eventBitLogTimer,
                    pdFALSE, pdFALSE, portMAX_DELAY);
        if ((uxBits & eventBitLogTimer) != 0)
        {
            xEventGroupClearBits(eventGroup, eventBitLogTimer);
            remoteDatabase.log(trilFrequentie.getTrilFrequentie());
        }
        else if ((uxBits & eventBitBnReleaseQueue) != 0)
        {
            if (tryReadQueue(bnReleaseQueue, bnReleaseID,
                             eventGroup, eventBitBnReleaseQueue))
            {
                bTransitie = true;
                if (bnReleaseID == NoodKnopID)
                {
                    state = State::NORMAAL;
                }
                else
                {
                    state = State::NOODTOESTAND;
                }
            }
        }
        else if ((uxBits & eventBitBnPressQueue) != 0)
        {
            // maak bnPressQueue leeg (zelf-transitie naar NoodToestand).
            if (tryReadQueue(bnPressQueue, dummyID,
                             eventGroup, eventBitBnPressQueue))
            {
                bTransitie = true;
            }
        }
    }
    break;
```

Van STD naar code van TrilControl

De eerste stappen leiden weer tot vergelijkbare code. Nieuw ten opzichte van InstelControl zijn de enum class voor de substates van Normaal, de trilTimer en de boolean bMagneetEnabled:

```
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/event_groups.h"
#include "freertos/queue.h"
#include "freertos/timers.h"

// Onderstaande enum wordt zowel door TrilControl en InstelControl
// Gebruikt. Hij moet dus in een aparte header file staan die door
// beide wordt geinclude. Voor het overzicht zet ik het toch even
// erbij, hier.
enum eButtonID {MinKnopID, PlusKnopID, TrilKnopID, NoodKnopID};

class TrilControl : public ButtonListener
{
    enum class State           {NORMAAL, NOODTOESTAND};
    enum class SubstateNormaal {NIET_TRILLEN, TRILLEN};

    // Er wordt in een toestand op meer dingen tegelijk gewacht. Daarom
    // krijgt alles waarop gewacht wordt een eventbit, ook de queues.
    static constexpr EventBits_t eventBitBnPressQueue   = (1 << 0);
    static constexpr EventBits_t eventBitBnReleaseQueue = (1 << 1);
    static constexpr EventBits_t eventBitTrilTimer      = (1 << 2);

private:
    State               state           = State::NORMAAL;
    SubstateNormaal     substateNormaal = SubstateNormaal::NIET_TRILLEN;

    TaskHandle_t        taskHandle;
    EventGroupHandle_t  eventGroup;
    QueueHandle_t       bnPressQueue;
    QueueHandle_t       bnReleaseQueue;
    TimerHandle_t       trilTimer;

    Trilfrequentie&     trilFrequentie;
    Button&             noodKnop;
    Button              trilKnop;
    Electromagneet      electromagneet;

    bool                bMagneetEnabled = false;

public:
    TrilControl(const char* taskName, UBaseType_t taskPriority,
                uint32_t taskStackSizeBytes, BaseType_t taskCoreNumber,
                Trilfrequentie& trilFrequentie, Button& noodKnop,
                ButtonHandler& buttonHandler) :
        trilFrequentie(trilFrequentie), noodKnop(noodKnop)
    {
        eventGroup = xEventGroupCreate();
        configASSERT(eventGroup != NULL);
        bnPressQueue = xQueueCreate(10, sizeof(eButtonID));
        configASSERT(bnPressQueue != NULL);
        bnReleaseQueue = xQueueCreate(10, sizeof(eButtonID));
        configASSERT(bnReleaseQueue != NULL);

        // De timer van de after() in Trillen. One shot; zijn callback zet
        // eventBitTrilTimer. De periode wordt bij elke entry van Trillen
        // opnieuw gezet (zie daar), dus 500 ms is maar een beginwaarde.
        trilTimer = xTimerCreate("TrilTimer", pdMS_TO_TICKS(500), pdFALSE,
                                 this, trilTimerCallback);
        configASSERT(trilTimer != NULL);
```

```
    // Meld objecten die door een handler aangezwengeld worden
    // aan bij de betreffende handler.
    buttonHandler.addButton(&trilKnop);
    buttonHandler.addButton(&noodKnop);
                // Aanname hierbij: de handler negeert
                // herhaaldelijk aanmelden van dezelfde knop.

    // Meld mezelf als ButtonListener aan bij de buttons waar ik
    // naar luister.
    trilKnop.addButtonListener(this);
    noodKnop.addButtonListener(this);

    // start de thread van deze taak
    xTaskCreatePinnedToCore(taskWrapper, taskName, taskStackSizeBytes,
                            this, taskPriority, &taskHandle,
                            taskCoreNumber);
}

// De STD interface. Deze functies worden uitgevoerd in de thread
// van de aanroepende taak.
void buttonPressed(eButtonID buttonID)
{
    xQueueSend(bnPressQueue, &buttonID, 0);
    xEventGroupSetBits(eventGroup, eventBitBnPressQueue);
}
void buttonReleased(eButtonID buttonID)
{
    xQueueSend(bnReleaseQueue, &buttonID, 0);
    xEventGroupSetBits(eventGroup, eventBitBnReleaseQueue);
}
```

taskWrapper en tryReadQueue zijn gelijk aan die van InstelControl (met TrilControl in plaats van InstelControl). De timer callback zet nu eventBitTrilTimer:

```
static void trilTimerCallback(TimerHandle_t xTimer)
{
    TrilControl* instance =
        static_cast<TrilControl*>(pvTimerGetTimerID(xTimer));
    xEventGroupSetBits(instance->eventGroup, instance->eventBitTrilTimer);
}
```

Ook de vervolgstap, de uitwerking van de STD in code, is weer rechttoe rechtaan. Dit is de implementatie van de toestand Normaal, NietTrillen:

```
void main()
{
    eButtonID   bnID;
    eButtonID   bnReleaseID;
    eButtonID   dummyID;
    EventBits_t uxBits;

    while (true)
    {
        switch (state)
        {
        case State::NORMAAL:
            switch (substateNormaal)
            {
            case SubstateNormaal::NIET_TRILLEN:
                uxBits = xEventGroupWaitBits(eventGroup,
                             eventBitBnPressQueue | eventBitBnReleaseQueue,
                             pdFALSE, pdFALSE, portMAX_DELAY);
                if ((uxBits & eventBitBnPressQueue) != 0)
                {
                    if (tryReadQueue(bnPressQueue, bnID,
                                     eventGroup, eventBitBnPressQueue))
                    {
                        if (bnID == NoodKnopID)
                        {
                            // via de exit node naar NoodToestand.
                            state = State::NOODTOESTAND;
                        }
                        else // bnID moet TrilKnopID zijn
                        {
                            substateNormaal = SubstateNormaal::TRILLEN;
                        }
                    }
                }
                else if ((uxBits & eventBitBnReleaseQueue) != 0)
                {
                    // maak bnReleaseQueue leeg.
                    tryReadQueue(bnReleaseQueue, dummyID,
                                 eventGroup, eventBitBnReleaseQueue);
                }
                break;
```

Dan Normaal, Trillen:

```
case SubstateNormaal::TRILLEN:
    // Er is een "after" event. Dat impliceert dat deze toestand een
    // entry event heeft waarbij de timer wordt gestart.
    // Een halve trilperiode is 1/(2*trilfrequentie) seconde, oftewel
    // 500/trilfrequentie milliseconden. Die hangt af van de trilfrequentie:
    // daarom xTimerChangePeriod in plaats van xTimerStart
    // (xTimerChangePeriod start de timer ook).
    xEventGroupClearBits(eventGroup, eventBitTrilTimer);
    xTimerChangePeriod(trilTimer,
        pdMS_TO_TICKS(500 / trilFrequentie.getTrilFrequentie()),
        portMAX_DELAY);

    uxBits = xEventGroupWaitBits(eventGroup,
                eventBitBnPressQueue | eventBitBnReleaseQueue
                                     | eventBitTrilTimer,
                pdFALSE, pdFALSE, portMAX_DELAY);
    if ((uxBits & eventBitTrilTimer) != 0)
    {
        xEventGroupClearBits(eventGroup, eventBitTrilTimer);
        bMagneetEnabled = !bMagneetEnabled;
        if (bMagneetEnabled)
        {
            electromagneet.zetAan();
        }
        else
        {
            electromagneet.zetUit();
        }
    }
    else
    {
        // Elke andere event verlaat de toestand Trillen. Een after()
        // geldt alleen zolang je in de toestand bent: stop de timer dus,
        // en gooi een timer-event weg dat intussen al was opgetreden.
        xTimerStop(trilTimer, portMAX_DELAY);
        xEventGroupClearBits(eventGroup, eventBitTrilTimer);

        if ((uxBits & eventBitBnPressQueue) != 0)
        {
            if (tryReadQueue(bnPressQueue, bnID,
                             eventGroup, eventBitBnPressQueue))
            {
                // Het trillen stopt: laat de magneet niet aan staan.
                electromagneet.zetUit();
                bMagneetEnabled = false;

                if (bnID == NoodKnopID)
                {
                    // via de exit node naar NoodToestand.
                    state = State::NOODTOESTAND;
                }
                else // bnID moet TrilKnopID zijn
                {
                    substateNormaal = SubstateNormaal::NIET_TRILLEN;
                }
            }
        }
        else if ((uxBits & eventBitBnReleaseQueue) != 0)
        {
            // maak bnReleaseQueue leeg.
            tryReadQueue(bnReleaseQueue, dummyID,
                         eventGroup, eventBitBnReleaseQueue);
        }
    }
    break;
```

```
default: break;
} // end switch substateNormaal
break;
```

Anders dan bij de logTimer van InstelControl is het aflopen van de trilTimer **geen** interne transitie. De after()-pijl verlaat Trillen en komt er weer in. Daarom is er hier geen binnenste wachtlus: na elke event begint de case opnieuw, en wordt het entry event (de timer opnieuw zetten) opnieuw uitgevoerd. Precies wat we willen: na elke halve trilperiode start de volgende.

Ook het leeghalen van bnReleaseQueue is een zelf-transitie: de timer wordt gestopt en bij het weer binnengaan van Trillen opnieuw gestart. De magneet blijft daarbij in dezelfde stand.

NB: bij 100 Hz duurt een halve trilperiode 5 ms. Dat vraagt een tick van 1 ms (configTICK_RATE_HZ = 1000). Bij de ESP-IDF-default van 100 Hz is een tick 10 ms, en wordt pdMS_TO_TICKS(5) nul.

En tot slot, de NoodToestand:

```
case State::NOODTOESTAND:
    uxBits = xEventGroupWaitBits(eventGroup,
                 eventBitBnPressQueue | eventBitBnReleaseQueue,
                 pdFALSE, pdFALSE, portMAX_DELAY);
    if ((uxBits & eventBitBnReleaseQueue) != 0)
    {
        if (tryReadQueue(bnReleaseQueue, bnReleaseID,
                         eventGroup, eventBitBnReleaseQueue))
        {
            if (bnReleaseID == NoodKnopID)
            {
                state = State::NORMAAL;
                substateNormaal = SubstateNormaal::NIET_TRILLEN; // initieel
            }
            else
            {
                state = State::NOODTOESTAND;
            }
        }
    }
    else if ((uxBits & eventBitBnPressQueue) != 0)
    {
        // maak bnPressQueue leeg.
        tryReadQueue(bnPressQueue, dummyID,
                     eventGroup, eventBitBnPressQueue);
    }
    break;

default: break;
```

Enum prefixes

In de code hierboven zijn de toestanden een **enum class**. Daardoor moet je de naam van een toestand altijd prefixen met de enum type naam: State::NORMAAL, SubstateNormaal::TRILLEN. Dat voorkomt nameclashes: NORMAAL kan zonder problemen ook in een andere enum voorkomen. Bovendien zet de compiler een enum class niet stilzwijgend om naar een int.

eButtonID is omwille van het overzicht nog een gewone enum. Ook daar is het een betere gewoonte om de namen te prefixen met de enum type naam, of er een enum class van te maken.

STD naar Code bij meer geavanceerde varianten

Je hebt nu voorbeelden gezien voor het omzetten van STD naar code bij het Trilapparaat.
Er zijn echter situaties die extra aandacht vergen:

- Een entry event in een state die nested substates heeft.
- Een action bij de uitgaande pijl van een state die nested substates heeft.
- Een exit event in een state die nested substates heeft.
- Een event bij de uitgaande pijl van een state die nested substates heeft.

Een entry event in een state die nested substates heeft

Kijk bijvoorbeeld nog eens naar de huidige code van TrilControl:

```
while (true)
{
    switch (state)
```

```
{
case State::NORMAAL:
    switch (substateNormaal)
    {
    case SubstateNormaal::NIET_TRILLEN:
        uxBits = xEventGroupWaitBits(eventGroup,
                    eventBitBnPressQueue | eventBitBnReleaseQueue,
                    pdFALSE, pdFALSE, portMAX_DELAY);
        if ((uxBits & eventBitBnPressQueue) != 0)
        {
            if (tryReadQueue(bnPressQueue, bnID,
                             eventGroup, eventBitBnPressQueue))
            {
                if (bnID == NoodKnopID)
                {
                    state = State::NOODTOESTAND;
                }
                else // bnID moet TrilKnopID zijn
                {
                    substateNormaal = SubstateNormaal::TRILLEN;
                }
```

Stel nu dat de onderste else case wordt uitgevoerd: substateNormaal wordt gezet op TRILLEN.
Dan is de case afgehandeld, en komen we weer terug bij het begin van de while (true) loop, bij het bovenste switch statement. Aangezien de hoofdtoestand nog steeds NORMAAL is, gaat het programma vervolgens case State::NORMAAL in.

So far, geen probleem. Maar wat als NORMAAL entry event code zou hebben gehad, waar zetten we dat dan neer? Als we de entry event code simpelweg onder case State::NORMAAL: neerzetten, wordt die code steeds opnieuw uitgevoerd als er een substate verandert (zoals in bovenstaand voorbeeld). Maar entry event code van de state NORMAAL mag natuurlijk alleen uitgevoerd worden bij het binnengaan van NORMAAL vanuit een andere (main-) state.

Je zou in het algemeen dit op de volgende manier kunnen oplossen:

```
while (true)
{
    switch (state)
    {
    case State::NORMAAL:
        if (prevState != State::NORMAAL)
        {
            // begin entry event stuff
            // ...
            // end of entry event stuff
            prevState = State::NORMAAL;
        }
        switch (substateNormaal)
        {
        case SubstateNormaal::NIET_TRILLEN:
```

Daarbij moeten de andere hoofdtoestanden prevState ook bijwerken (bijvoorbeeld prevState = State::NOODTOESTAND onder case State::NOODTOESTAND:). Anders wordt de entry event code na een terugkeer naar NORMAAL niet opnieuw uitgevoerd.
In InstelControl zag je een verwante oplossing voor een verwant probleem: de binnenste while (!bTransitie) lus, waardoor een interne transitie het entry event niet opnieuw uitvoert.

Een action bij de uitgaande pijl van een state die nested substates heeft, of een exit-event van een state die nested substates heeft

In het STD van TrilControl kun je zien dat er naast de **uitgaande pijl van de toestand NORMAAL** geen event staat, en ook geen actions. Stel nu dat er een **action** zou hebben gestaan, dan zou die action uitgevoerd moeten worden door **elk van de substates** op het event dat van hen naar de exit node leidt. Hetzelfde geldt voor het geval dat de toestand NORMAAL exit event code had gehad. Ook die zou moeten worden uitgevoerd door **elk van de substates** op het event dat van hen naar de exit node leidt.

Iets vergelijkbaars zie je bij de substate Trillen. Die heeft geen exit event, maar de actie electromagneet.zetUit() staat bij de pijl die Trillen op een knopdruk verlaat, zowel richting NietTrillen als richting de exit node. Een exit event kan hier niet: ook de after()-pijl verlaat Trillen (en komt er weer in), en daar moet de magneet juist blijven togglen.

Een event bij de uitgaande pijl van een state die nested substates heeft.

In het STD van TrilControl is ervoor gekozen om de toestand NORMAAL te verlaten via een exit-node, in combinatie met een uitgaande pijl zonder event.

Alternatief had ervoor gekozen kunnen worden om de toestand NORMAAL te verlaten via een pijl naar Noodtoestand met naast die pijl een event “eventBitNoodKnopPressed” waarop wordt gewacht. Je kunt niet xEventGroupWaitBits(eventGroup, eventBitNoodKnopPressed, ...) onder case State::NORMAAL: zetten, omdat dan alle substates eronder worden geblokkeerd. De oplossing is **elk van de substates** (ook) op eventBitNoodKnopPressed te laten wachten: neem dat eventbit op in het masker van hun xEventGroupWaitBits.